BTK AKADEMİ EĞİTİM AKIŞI (2.Bölüm)

Sayı ve Kodlama sistemleri

Hash Fonksiyonlarının Tanımı ve Kullanım Alanları

Depolama Ortamları

Dosya Sitemleri

SAYI SİSTEMLERİ

Sayı Sistemleri Nedir?

Sayı Sistemleri: Sayıları adlandırmaya ve temsil etmeye yarayan ifade şeklidir.

I I I

Ondalık Sayı Sistemi (Decimal Number System)

*Bir sayıyı temsil etmek için 0 ila 9 arasındaki 10 rakam kullanır. Bu rakam sayısı aynı zamanda kullanılan tabanı ifade eder.

Örnek: 571 sayısını 10 tabanına göre açarsak:

$1 \times 10^0 = 1 \times 1 = 1$

$7 \times 10^1 = 7 \times 10 = 70$

$5 \times 10^2 = 5 \times 100 = 500$

TOPLAM = 571

Kaç Tane Sayı Sistemi Vardır?

İhtiyaca göre bir çok sayı sistemleri ifade edilebilir. En yaygın kullanılan sayı sistemi Decimal (10 tabanlı) sayı sistemidir.

Ancak genel olarak bilgisayar sistemlerinde kullanılan sayı sistemleri **2'li, 8'li, 16'lı tabanlı** sayı sistemleridir.

İkili Sayı Sistemi (Binary Number System)

*Bir sayıyı temsil etmek için 0 ve 1 olmak üzere 2 adet rakam kullanır. Bu rakam sayısı aynı zamanda kullanılan tabanı ifade eder.

İkili Sayı Sistemi (Binary Number System)

Bilgisayar sistemleri için temel olarak iki elektronik durum söz konusudur.
1.Durum: Açık=1
2. Durum: Kapalı=0

Binary (ikili) Sayı sisteminde bulunan her ‘0’ veya ‘1’ rakamları BIT (Binary Digit) adı ile tanımlanır.

Her karakter 8 bit’in birleşiminden oluşur. Örneğin **C** harfi **01100011** ile ifade edilir.

8 bit = 1 byte eşittir. Yapı dikkate alındığında üzerinde en fazla 2^8 değerinde veri taşıyabilir. 10‘luk tabanda bu değer 256 ya denk gelmektedir.

$2^8 = 2x2x2x2x2x2x2x2 = 256$

İkili Sayı Sistemi (Binary Number System)

8 bit = 1 byte eşittir. Yapı dikkate alındığında üzerinde en fazla 2^8 değerinde veri taşıyabilir. 10'luk tabanda bu değer 256 ya denk gelmektedir.

$2^8 = 2x2x2x2x2x2x2x2 = 256$

İkili Sayı Sistemi (Binary Number System)

Örnek: Binary Sistemde yazılabilecek 4 haneli kaç sayı yazılabilir ve Bu sayılardan en büyüğü kaçtır ?

- n tabanında k haneli yazılabilecek sayı miktarı = **Taban** $^{\textbf{hane}}$ yani $\mathbf{n}^{\mathbf{k}}$ şeklinde bulunur.

Taban =2 , Hane = 4 Sonuç = 2^4=16

- n tabanında k haneli yazılabilecek sayılardan en büyüğü = **Taban** $^{\textbf{hane}}$ **- 1** yani $\mathbf{n}^{\mathbf{k}}$ **- 1** şeklinde bulunur.

Taban =2 , Hane = 4 Sonuç = 2^4- 1 =15 (F)

Binary	Decimal	Binary	Decimal
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A=10
0011	3	1011	B=11
0100	4	1100	C=12
0101	5	1101	D=13
0110	6	1110	E=14
0111	7	1111	F=15

İkili Sayı Sistemi (Binary Number System) Toplama

Onluk tabanda toplama işleminde 10 ve katlarından sonraki sayılar bir sonrakine ekleniyorsa 2 tanında yapılan işlemler içinde 2 ve 2'nin katları bir sonrakine eklenir.

$$
\begin{array}{rcl}
1011 & = & 11 \\
1100 & = & 12 \\
+\ 1101 & = & 13 \\
\hline
\mathbf{100100} & \mathbf{=} & \mathbf{36}
\end{array}
$$

İkili Sayı Sistemi (Binary Number System)
Çıkarma

$$
\begin{array}{rcr}
1101 & = & 13 \\
- \; 1011 & = & 11 \\
\hline
\mathbf{0010} & \mathbf{=} & \mathbf{02}
\end{array}
$$

İkili Sayı Sistemi (Binary Number System)
Çarpma

```
        1101011      107
x          1011    x  11
        1101011      107
       1101011     +107
      0000000       1177
+ 1101011
 10010011001
```

İkili Sayı Sistemi (Binary Number System)
Bölme

$$11000 \div 100 = ?$$

11000	100
100	
01000	110

Binary to Decimal

$$(1\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 1\ )_2 = (?)_{10}$$

$2^9\ 2^8\ 2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$

$$(1\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 1\ )_2 = 1x2^9 + 0x2^8 + 0x2^7 + 0x2^6 + 1x2^5 + 1x2^4\ 1x2^3 + 0x2^2 + 1x2^1 + 1x2^0$$

$$= 512 + 0 + 0 + 0 + 32 + 16 + 8 + 0 + 2 + 1$$

$$= \mathbf{571}$$

Sekizli Sayı Sistemi (Octal Number System)

*Bu sistemde sayı gösterimi için 0 ile 8 arasında 8 rakam kullanır. Diğer sistemlere göre pek yaygın olmayan bir kullanımı vardır.

Örneğin, ondalık sayı sistemindeki 10, sekizlik sayı sisteminde 12 olarak gösterilir.

$$(\overset{3}{1}\overset{2}{0}\overset{1}{7}\overset{0}{3})_8 = 1x8^3 + 0x8^2 + 7x8^1 + 3x8^0 = (571)_{10}$$

On altılı Sayı Sistemi (Hexadecimal Number System)

*Bu sistem, sayı gösterimi için 0 ila 15 arasındaki 16 rakamı kullanır.

Ancak 9 dan sonra diğer rakamlar : 10=A, 11=B, 12=C, 13=D, 14=E, 15=F sembolleri ile ifade edilir.

$$(23B)_{16} = 2x16^2 + 3x16^1 + Bx16^0 = (571)_{10}$$

DİKKAT !!!

*571 sayısının 2 , 8 ve 16 tabanlarındaki karşılığına bakalım !

$(571)_{10} = (1\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 1\ )_2$ Binary

$(571)_{10} = (1073)_8$ Octal

$(571)_{10} = (23B)_{16}$ Hexadecimal

2’lik sistemden diğer sistemlere dönüşüm

$$(1\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 1)_2 = (?)_8$$

$$(1 - 0\ 0\ 0 - 1\ 1\ 1 - 0\ 1\ 1)_2 = (\underline{0\ 0\ 1} - \underline{0\ 0\ 0} - \underline{1\ 1\ 1} - \underline{0\ 1\ 1})$$

$$\downarrow \qquad \downarrow \qquad \downarrow \qquad \downarrow$$

$$1 \qquad 0 \qquad 7 \qquad 3$$

$$=(1073)_8$$

2’lik sistemden diğer sistemlere dönüşüm

$$(1\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 1\ )_2 = (?)_{16}$$

$$(1\ 0 - 0\ 0\ 1\ 1 - 1\ 0\ 1\ 1\ )_2 = (\underline{0\ 0\ 10} - \underline{0\ 0\ 1\ 1} - \underline{1\ 0\ 1\ 1})$$

2 3 B

$$=(23B)_{16}$$

VERİ KAVRAMI

İkili sistemlerde ifade edilen bit dizileri bizim için bir veri niteliğindedir. Bu verinin ham hali 0 ve 1'lerden oluşsa da aynı veri farklı veri kodlama yöntemleri kullanılarak farklı bir veriye dönüşebilmektedir.

Örneğin, 01000010 01010100 01001011 şeklindeki bit dizisi

- ASCII formatında 'BTK' ya karşılık gelirken,
- Hexadecimal formatında 0x42 0x54 0x4b sayısına karşılık gelir.
- BSD kodlamasında 42544B sayısına karşılık gelir.

VERİ KAVRAMI

- İkili sistemlerde ifade edilen bit dizileri bizim için bir veri niteliğindedir.

- Bu verinin ham hali 0 ve 1'lerden oluşsa da aynı veri farklı veri kodlama yöntemleri kullanılarak farklı bir veriye dönüşebilmektedir.

Örneğin, 01000010 01010100 01001011 şeklindeki bit dizisi

- ASCII formatında 'BTK' ya karşılık gelirken,
- BSD olarak ifade edildiğinde 42544B sayısına karşılık gelir.

VERİ YAPILARI

Veri Yapıları: bilgisayar ortamında verilerin etkin olarak saklanması ve işlenmesi için kullanılan yapılardır.

•***Temel Veri Yapıları:*** Karakter, Tamsayı ve Gerçel Sayı gibi değişkenler temel veri yapısı olarak kabul edilir.

•Karakterler bir araya gelerek sözceleri (string), sayılar bir araya gelerek dizileri (array) oluşturur.

Temel Veri Yapıları

- **<u>Karakter:</u>** a, c, z, !, >
- **<u>Tamsayı:</u>** 571, 44, 4
- **<u>Gerçel Sayı:</u>** 61,14 , 1,0002
- **<u>Sözce:</u>** ‘BTK’ , ‘AKADEMİ’’
- **<u>Dizi/Matris:</u>** [1 3 0], $\begin{matrix} 1 & 0 \\ 0 & 1 \end{matrix}$

Yazı Karakterleri Kodlama Sistemleri

Yazı karakterleri göstermede kullanılan birkaç yöntem vardır ancak bunlardan en çok kullanılanları;

- ASCII(American Standart Code For Information Interchange),
- Unicode (Unicode Transformation Format) dir. (UTF-8,UTF-16,UTF-32)

ASCII(American Standart Code For Information Interchange)

ASCII birtakım sayıların birtakım karakterlerle eşleştirildiği basit bir tablodan ibarettir.

- ASCII 7 bitlik bir sistemdir.
- (2^7=128) olduğundan toplam 128 karakter oluşturulabilir.
- Bu 7 bit ile gösterilebilecek en son sayı 127‘dir.
- ASCII tablosunu şöyle bir incelediğimizde ilk 32 karakter kontrol karakteri, geri kalan 96 karakter ise basım karakteridir.

ASCII TABLOSU

0	00	NUL
1	01	SOH
2	02	STX
3	03	ETX
4	04	EOT
5	05	ENQ
6	06	ACK
7	07	BEL
8	08	BS
9	09	HT
10	0A	LF
11	0B	VT
12	0C	FF
13	0D	CR
14	0E	SO
15	0F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM

26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US
32	20	space
33	21	!
34	22	"
35	23	#
36	24	$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3

52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M

78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]
94	5E	^
95	5F	_
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g

104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	|
125	7D	}
126	7E	~
127	7F	DEL

Genişletilmiş ASCII(American Standart Code For Information Interchange)

- ASCII 8 bitlik bir sistemdir.
- (2^8=256) olduğundan toplam 256 karakter oluşturulabilir.
- Bu 8 bit ile gösterilebilecek en son sayı 255‘dir.
- ASCII tablosunu şöyle bir incelediğimizde ilk 32 karakter kontrol karakteri, geri kalan 96 karakter ise basım karakteridir. Standart ASCII ile aynı ancak 129 – 255 arası karakter farklı ülke ve durumlara göre özelleştirilmiş karakterler olarak kullanılmaktadır.

Unicode (Universal Code)

- Universal" ve "Code" kelimelerinin bir araya getirilmesiyle oluşan Unicode standartı 1980'li yıllarda geliştirilmiştir.
- Unicode'un geliştirilmesinin arında yatan temel neden ASCII (American Standart Code for Information Interchange) karakter kodlamasının daha gelişmiş ve stratejik bir sürümünün oluşturulabilmesidir.
- ASCII karakterler sadece İngilizce üzerinde etkili olurken, Unicode tamamen evrenseldir.
- İbranice, Arapça ve Çince gibi kompleks diller ile karmaşık semboller ve karakterler kolayca meydana getirilebilirler.

Unicode Transformatin Format (UTF)

- UTF-16, UTF-32 ve UTF-8, kod noktalarının baytlarla nasıl eşleştirildiğini açıklayan Unicode dönüşümleri geliştirilmiştir .
- UTF, her Unicode kod noktasını benzersiz bir bayt dizisine eşlemek için bir algoritma tanımlayan Unicode Dönüşümü anlamına gelir.
- örneğin UTF-16 için 16 bit, UTF-32 için 32 bit ve UTF-8 için 8 bit alan kullanılmaktadır.
- UTF-8, UTF-16 ve UTF-32 karakter kodlaması arasındaki temel fark, bellekteki bir karakteri temsil etmek için kaç bayta ihtiyaç duyduğudur.

Unicode Transformatin Format (UTF)

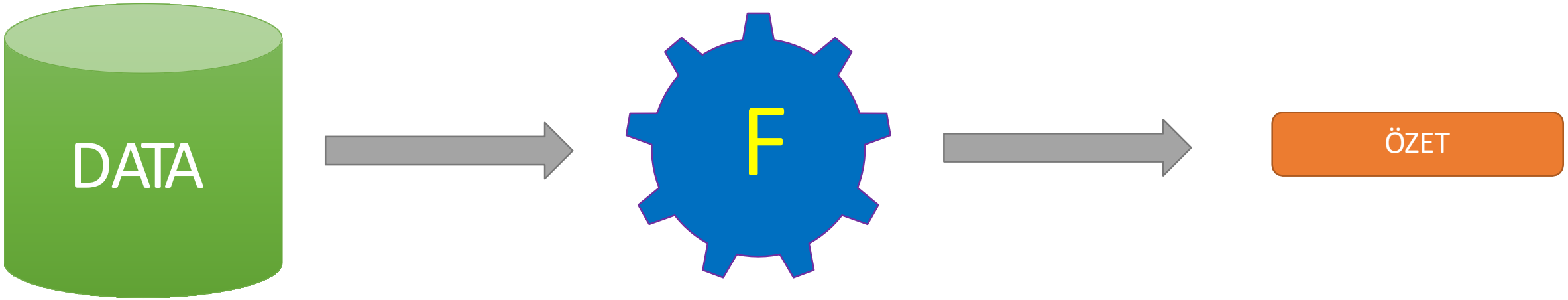
- UTF-8 en az bir bayt kullanırken UTF-16 en az 2 bayt kullanır. Karakterin kod noktasının baytın maksimum değeri olan 127'den büyükse, o zaman UTF-8 2, 3 veya 4 bayt alabilir,
- UTF-16 yalnızca iki veya dört bayt olabilir.
- UTF-32 sabit genişlikli bir kodlama şemasıdır ve bir Unicode kod noktasını kodlamak için her zaman 4 bayt kullanır.

Unicode ile ASCII Farkları

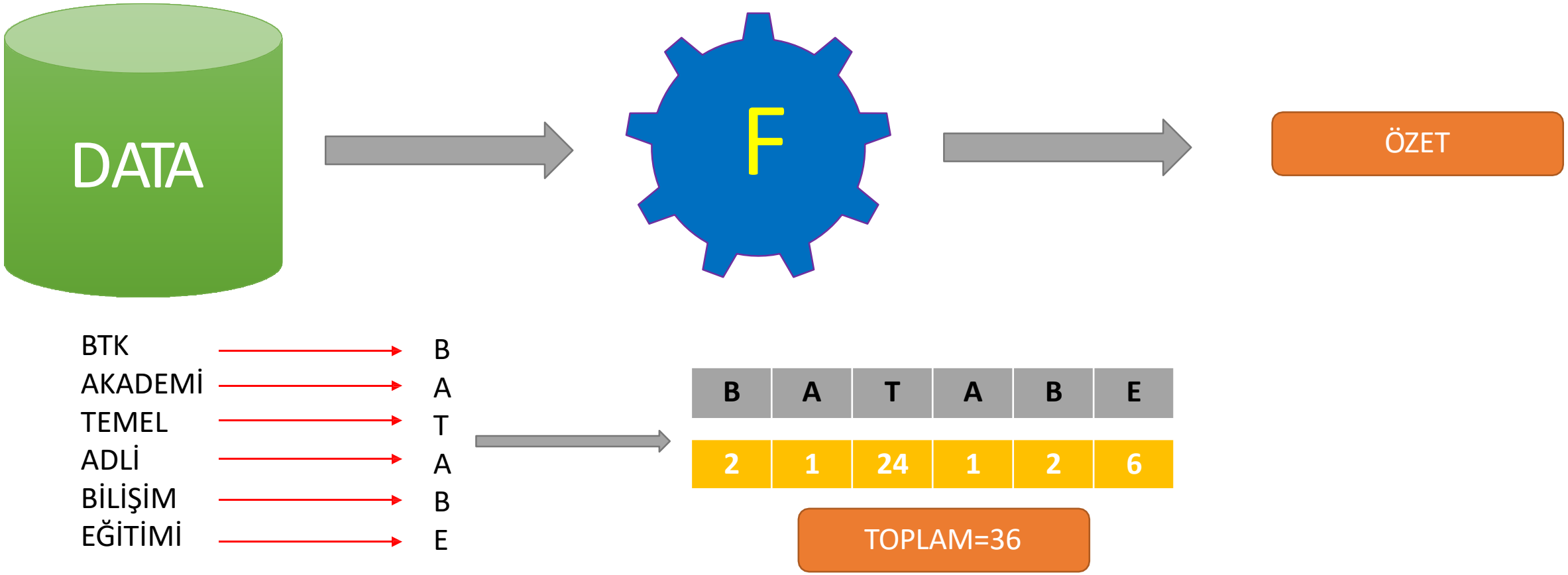
- ASCII yalnızca Latin alfabesi için kullanılabilir. Unicode ise evrensel olarak tüm dillerin kullanımına uygun şekilde tasarlanmıştır.
- Unicode hala Unicode Konsoriyum'u tarafından kar amacı gütmeden geliştirilmeye devam edilmektedir. ASCII'nin geliştirilmesi yıllar önce durdurulmuştur.

HASH FONKSİYONLARI

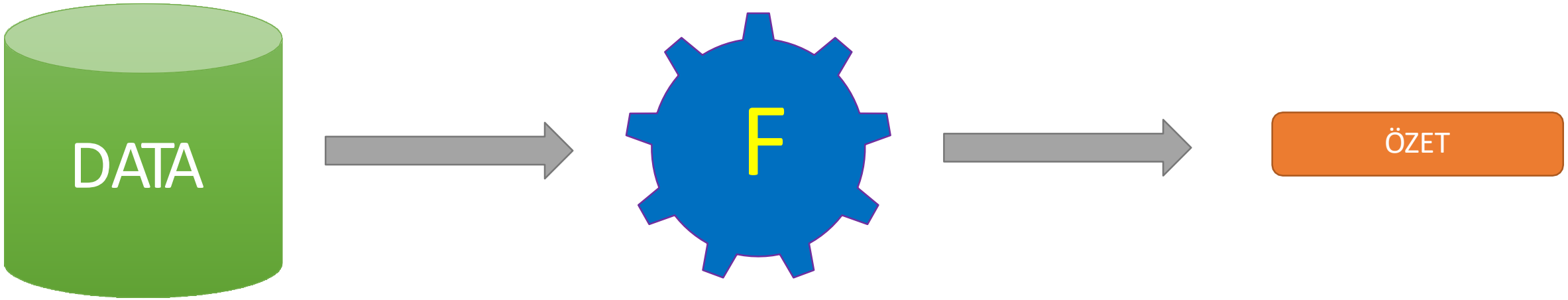
Hash Fonksiyonları Nedir ?



Hash Fonksiyonları Nasıl Çalışır?



Hash Fonksiyonları Nasıl Çalışır?



	Mod 7	
25	→	4
87	→	**3**
65	→	2
78	→	1
98	→	0
66	→	**3**

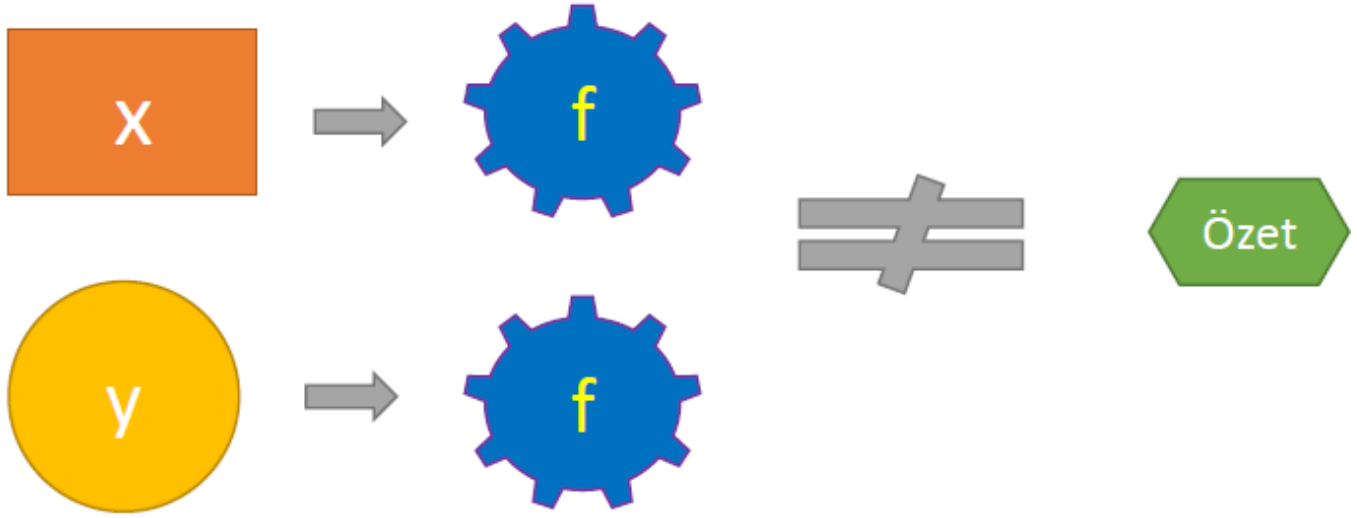
0	1	2	3	4	5	6
0	**1**	**2**	**3**	**4**		

3

İyi Bir Hash Fonksiyonun Özelliği Nasıl Olmalı ?

Çakışma Direnci (collision): Hash fonksiyonuna giren herhangi iki farklı girdinin aynı özeti çıktı olarak üretmemesi gerekir.

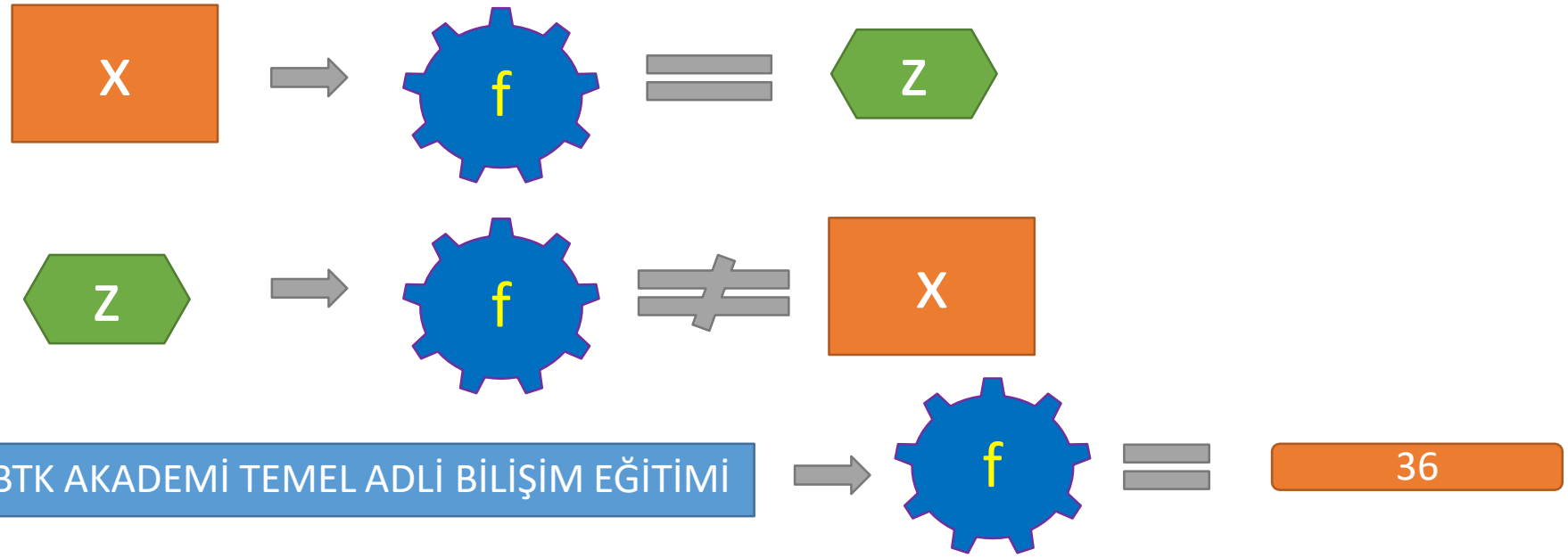
- Bir karma işlevi h için, h(x) = h(y) olacak şekilde herhangi iki farklı x ve y girdisi bulmak zordur.



İyi Bir Hash Fonksiyonun Özelliği Nasıl Olmalı?

Ön Görüntü Direnci (Pre-Image Resistance): Hash fonksiyonundan çıkan özet değerden orijinal mesaja ulaşılamaması gerekir.

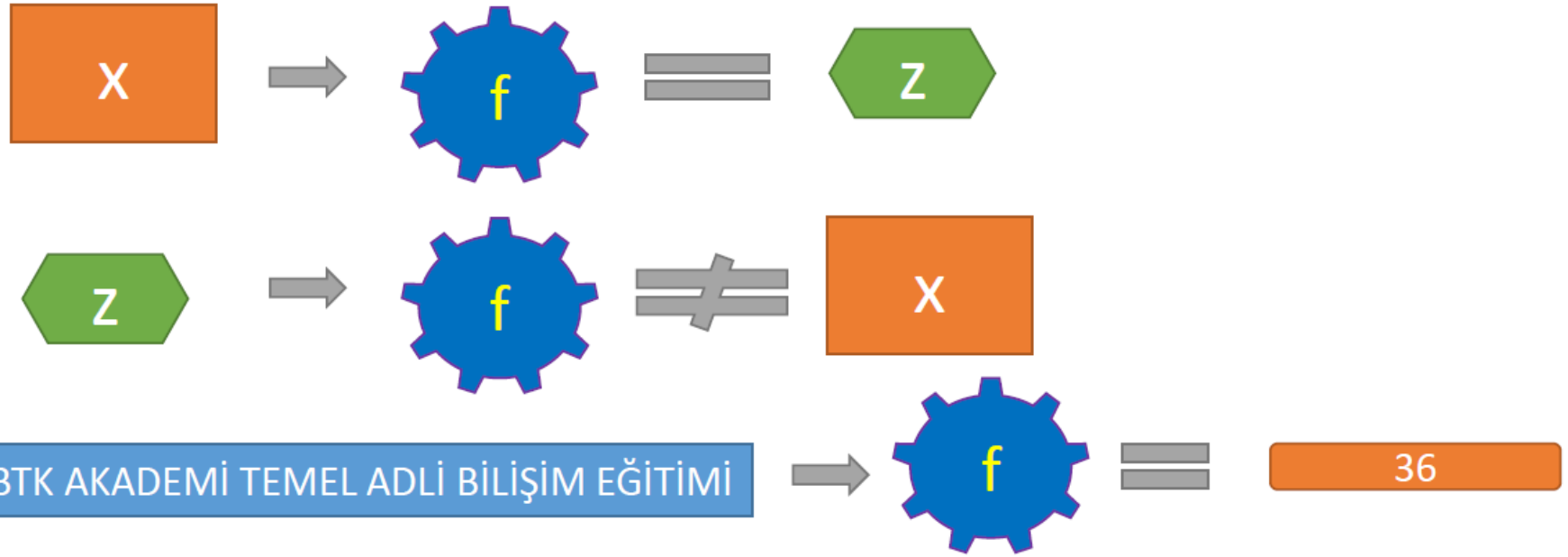
❖ Eğer bir hash fonksiyonu h bir hash değeri z ürettiyse, o zaman z'ye hash olan herhangi bir x girdi değerini bulmak zor bir süreç olmalıdır.



İyi Bir Hash Fonksiyonun Özelliği Nasıl Olmalı ?

Ön Görüntü Direnci (Pre-Image Resistance): Hash fonksiyonundan çıkan özet değerden orijinal mesaja ulaşılamaması gerekir.

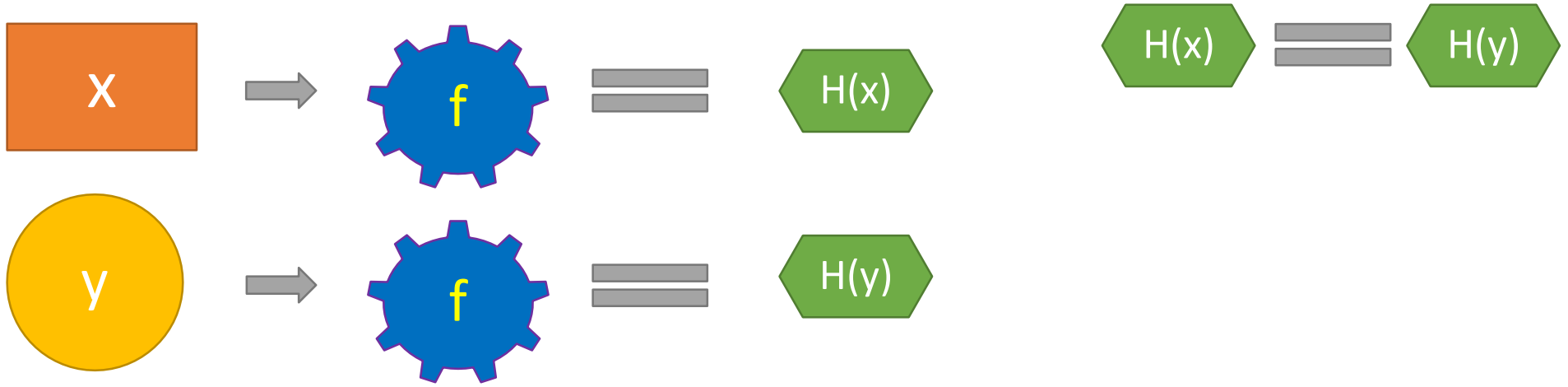
- Eğer bir hash fonksiyonu h bir hash değeri z ürettiyse, o zaman z'ye hash olan herhangi bir x girdi değerini bulmak zor bir süreç olmalıdır.



İyi Bir Hash Fonksiyonun Özelliği Nasıl Olmalıdır?

İkincil Ön Görüntü Direnci (Second Pre-Image Resistance): İki ayrı mesajın aynı özet değerine denk gelmesi çok zor olmalıdır.

- Eğer bir x girişi için bir hash fonksiyonu (h), h(x) hash değerini üretiyorsa, o zaman h(y) = h(x) olacak şekilde başka bir y girdi değeri bulmak zor olmalıdır.

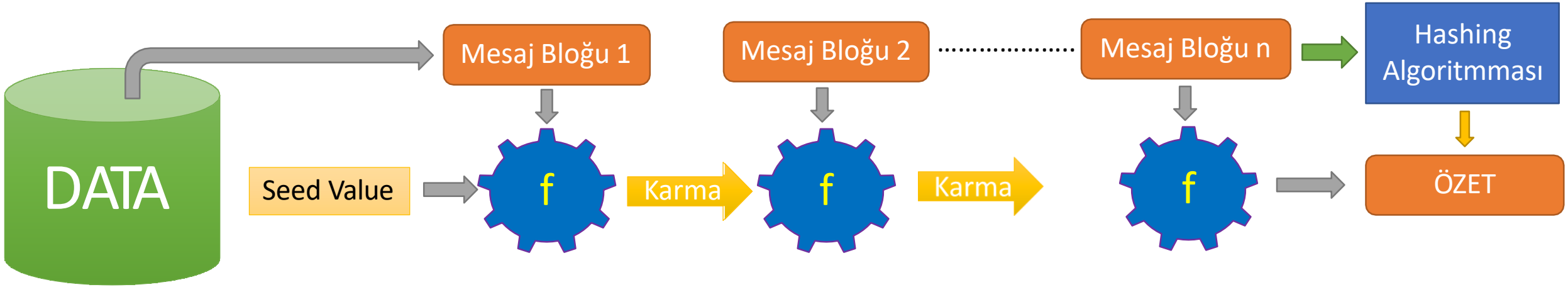


İyi Bir Hash Fonksiyonun Özelliği Nasıl Olmalı ?

İkincil Ön Görüntü Direnci (Second Pre-Image Resistance): İki ayrı mesajın aynı özet değerine denk gelmesi çok zor olmalıdır.

- Eğer bir x girişi için bir hash fonksiyonu (h), h(x) hash değerini üretiyorsa, o zaman h(y) = h(x) olacak şekilde başka bir y girdi değeri bulmak zor olmalıdır.

Hash Fonksiyonları Nasıl Çalışır?



Birinci mesaj bloğunun karma değeri, çıktısı üçüncü işlemin sonucunu değiştiren ikinci karma işlemin girdisi haline geldiğinden, vb. **Bu etki, karmanın çığ etkisi olarak bilinir .**

Çığ etkisi, tek bir veri biti ile bile farklılık gösteren iki mesaj için önemli ölçüde farklı karma değerlerle sonuçlanır.

Kullanım Alanları

- Mesaj Güvenliği
- Şifre Güvenliği
- Adli Bilişim
- Kripto Para

Kullanım Alanları

BTK

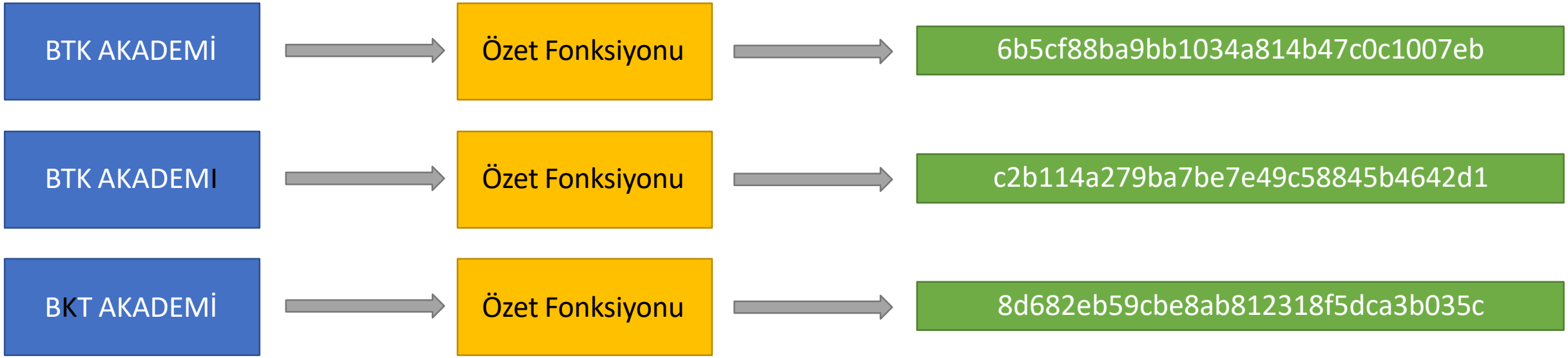
AKADEMİ

USER NAME	PASSWORD
8dca7e74ef73b0265911fec31e519e83	b0956973857efb53188c30bc51ca9508

Kullanım Alanları

HASH VALUE
58188088b894afd312f3148f014f3289

Kullanım Alanları



Yaygın Kullanılan Özet Fonksiyonları

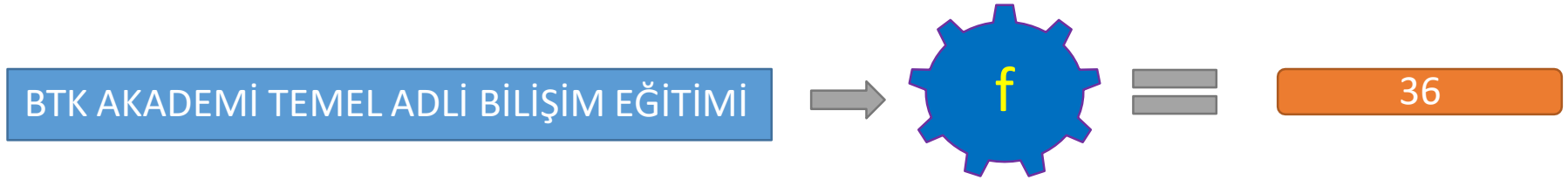
1- MD (Message-Digest algorithm) Ailesi: MD5 karma işlevi, orijinal olarak, dijital imzaların kimliğini doğrulamak için güvenli bir şifreleme karma algoritması olarak kullanılmak üzere tasarlanmıştır

MD2, MD4, MD5 ve MD6 karma işlevlerinden oluşur. İnternet Standardı RFC 1321 olarak kabul edilmiştir. 128 bitlik bir karma işlevidir.

MD2 8 bit makineler için,
MD4 VE MD5 32 bit makineler için tasarlanmıştır.
MD5, MD4 den daha yavaş ancak daha güvenilirdir.

MD5 Nedir?

MD5 (message digest algorithm) karma algoritması, herhangi bir uzunluktaki mesajı girdi olarak kabul eden ve çıktı olarak orijinal mesajın kimliğini doğrulamak için kullanılacak sabit uzunlukta bir özet değeri döndüren tek yönlü bir şifreleme işlevidir.

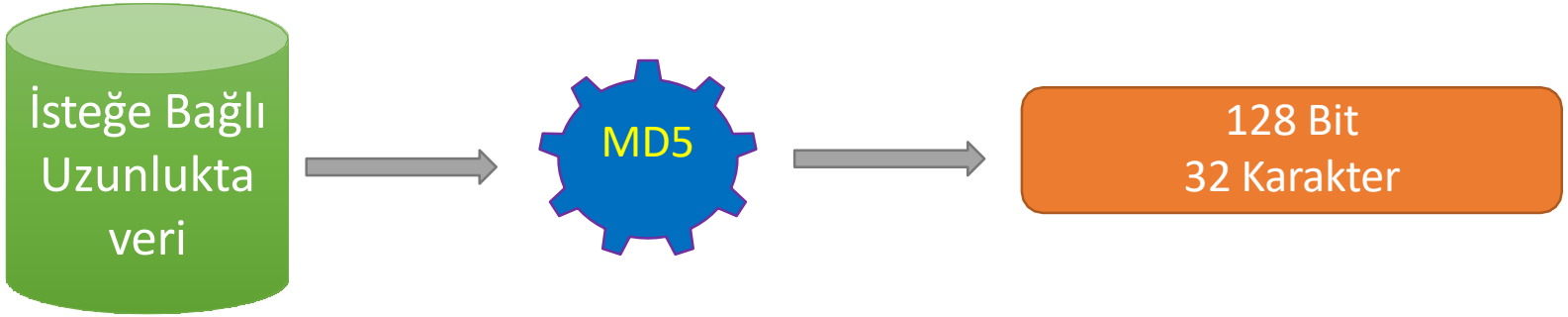


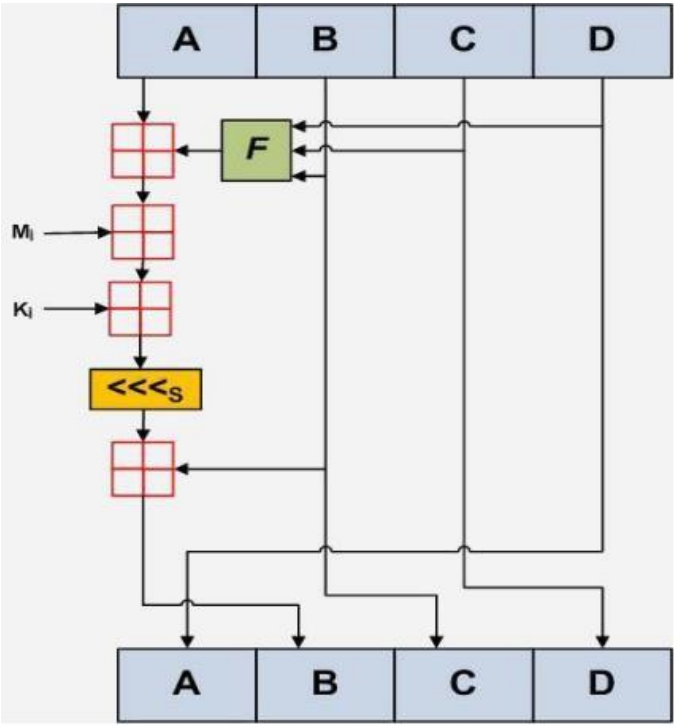
MD5 Kullanım Alanları?

- MD5 Adli Bilişim alanında veri bütünlüğünü doğrulamak ve kasıtsız veri bozulmasını tespit etmek için kullanılır.

- Mesaj Güvenliği
- Şifre Güvenliği
- Veri Bütünlüğünü Doğrulama

MD5 Yapısı


İsteğe Bağlı Uzunlukta veri
MD5
128 Bit
32 Karakter


A
B
C
D
F
Mi
Ki
<<<s
A
B
C
D

Yaygın Kullanılan Özet Fonksiyonları

2- SHA-1 :

- Mesaj özet algoritmalarının tasarımında MD5'inkilere benzer teknikler kullanılır.

- MD5’e göre daha güvenli olan160 bitlik değer üretir.

Yaygın Kullanılan Özet Fonksiyonları

2- SHA-2 :

- SHA-1’ e göre SHA-2 mesaj özet algoritmaları daha uzun ve kırılması daha zordur.
- SHA-2 algoritma ailesi, 224, 256, 384 ve 512 bit uzunluğunda karma değerler sunar.
- Mesaj özet uzunluklarının adlarıyla bilinirler - örneğin, SHA-224 ve SHA-256.

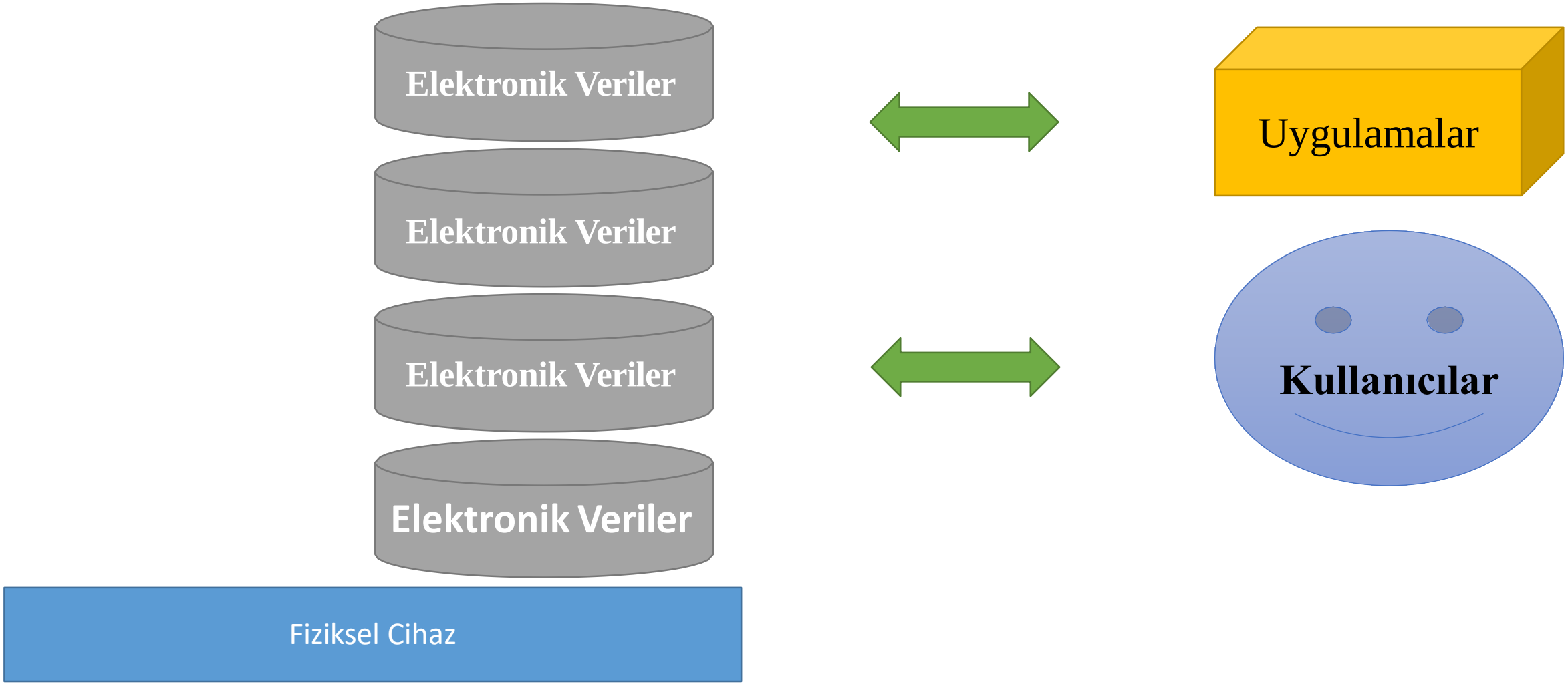
Depolama Ortamları

Depolama Ortamlarına Neden İhtiyaç Duyulur?

- Dijital veriler, uçucu ve çabuk bozulabilir bir yapıya sahiptir. Bu nedenle veri yönetimi ve organizasyonunun dikkatli bir şekilde yapılması veriye ulaşabilmek açısından en temel gereksinimlerden bir tanesidir.

- Bu veri yönetiminin ve organizyasyonun yapılmasına imkan tanıyan sistemler depolama ortamlarıdır.

Depolama Ortamları Nedir ?



Depolama Ortamları Nedir ?

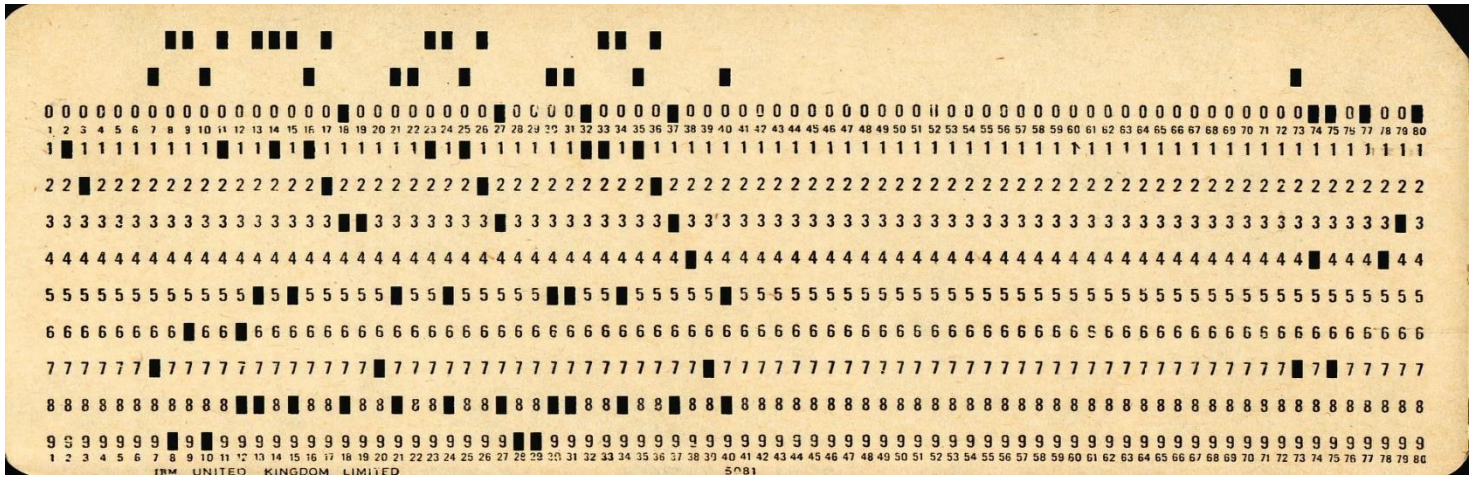
- Bilgisayarlarda, bir depolama ortamı, uygulamalar ve kullanıcılar için elektronik verileri alan ve saklayan, diğer taraftanda verileri geri almak için hazır hale getiren fiziksel bir cihazdır.

- Depolama ortamı, bir bilgisayarın veya başka bir aygıtın içinde olabilir veya bir sisteme doğrudan veya bir ağ üzerinden harici olarak bağlı olabilir .

- ***Bu terimin çoğul hali depolama ortamıdır.***

Depolama Ortamlarının Ortaya Çıkışı?

- Depolama ortamının ilk biçimleri, içine delikler açılmış bilgisayar kağıt bantlarını içeriyordu.
- Her delik tek bir veri bitine karşılık geliyordu. Bir kağıt şerit okuyucu, deliği yorumlar ve bir sayıya dönüştürür.
- Delikli kartlar da veri depolamanın ilk günlerinde yaygın olarak kullanıldı ve bir zamanlar dünyanın dijital bilgilerinin çoğunu depoladı.

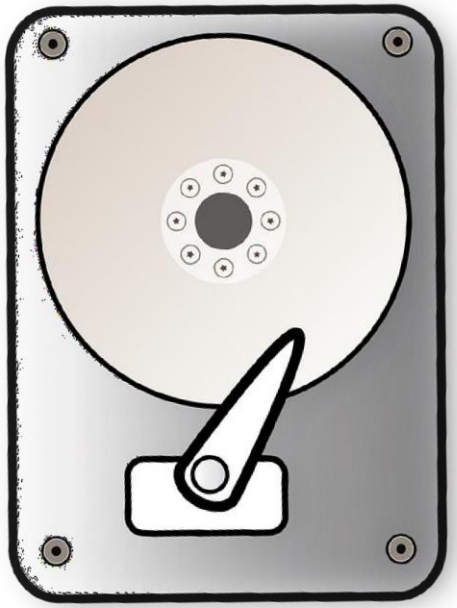


*Resim Kaynak (https://en.wikipedia.org/wiki/Punched_card)

Depolama Ortamlarının Ortaya Çıkışı?

- Kağıt bant ve delikli kartların yerini manyetik bant aldı ve bu da sonunda manyetik disklerin ortaya çıkışına yol açtı.

- Sabit disk sürücüleri (HDD'ler) ve katı hal sürücüleri (SSD'ler) artık birincil depolama biçimleridir.



Depolama Ortamlarının Ortaya Çıkışı?

1725 - Delikli kartlar

1846 - Delikli kartuş

1946 - Selectron tüpü - 32-512 bayt

1947 - Williams tüpü - 0.0625KB

1950 - Silindir bellek - 10KB

1951 - Manyetik kartuş (UNISERVO) - inç başına 128B

1956 - Sabit disk (IBM 350) - 4.4MB

1969 - Disket - 80KB

1972 - Kompakt kaset - 660KB

1978 - Lazer disk - her yüze 1 saat video

1980 - IBM 3380 - 2.52GB

1980 - 5.25" disket (ST-506) - 5MB

1982 - CD - 700MB

1985 - DVD ROM - 4GB

1995 - Jazz Drive - 1GB

1996 - 7200RPM sabit disk

1999 - Microdrive - 170MB

2000 - USB bellek çubuğu 4MB - 256GB

2000 - SD kart - 32MB-64MB

2003 - Blu Ray - 25GB

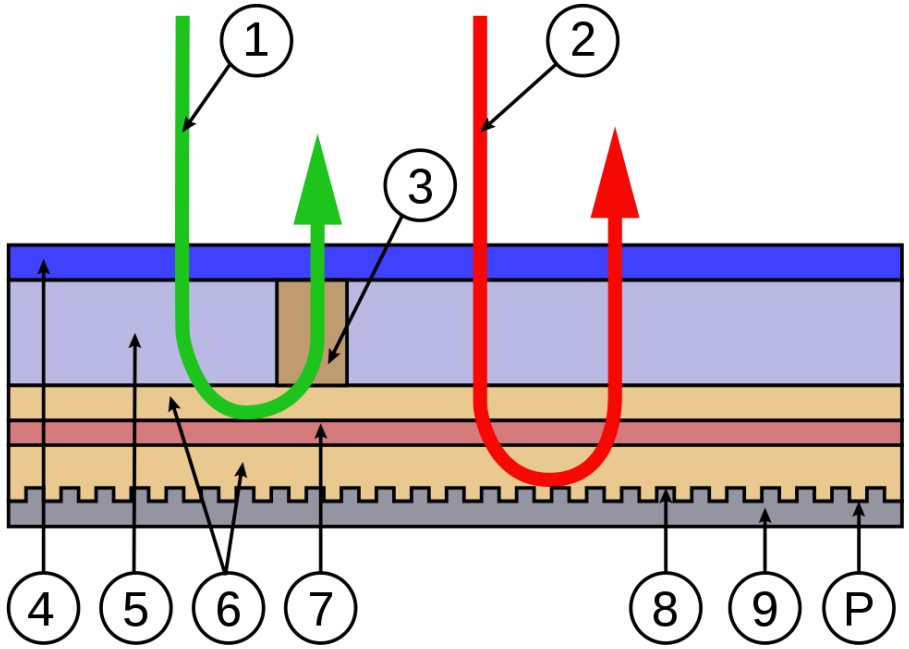
2006 - Bulut depolama

2008 - Katı hal sürücüsü (SSD) - 64GB

2009 - Holographic Versatile Disc - 39TB

*kaynak (https://www.chip.com.tr/haber/depolama-ortamlarinin-tarihcesi_47044.html)

Holographic Versatile Disc ?



*Resim Kaynak (https://en.wikipedia.org/wiki/Holographic_Versatile_Disc#/media/File:HVDstruct.svg)

Farklı Depolama Ortamları

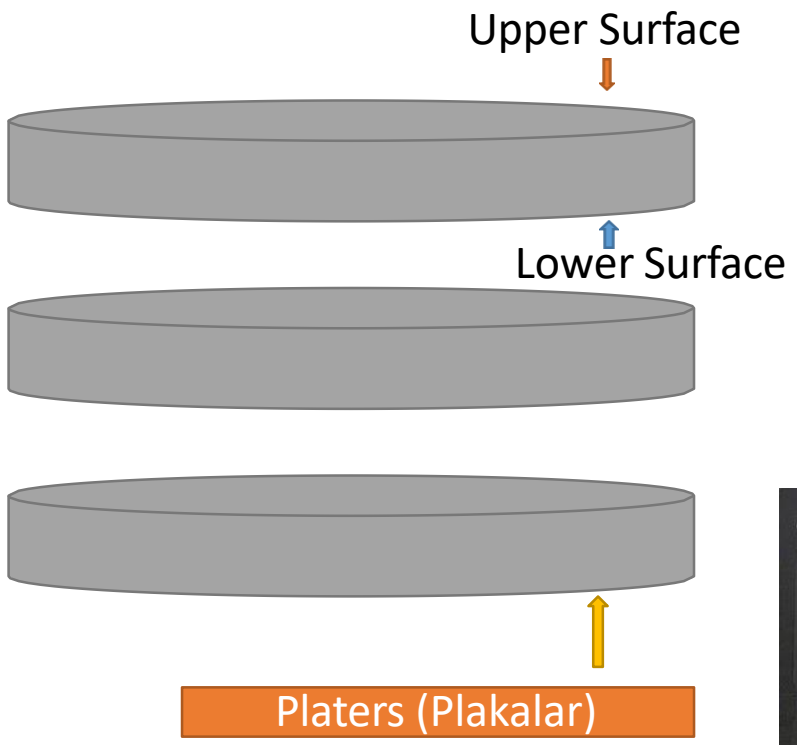
1-Sabit Diskler

Verileri depolamak için kullanabileceğimiz depolama aygıtlarıdır. Modern bilgisayarların çoğu, büyük miktarda veri depolamak için sabit diskler kullanır.

Bir sabit diskin mimarisi, aşağıdakileri içeren birkaç fiziksel bileşenden oluşur:

- Plakalar (Platters)
- Mil (Spindel)
- Okuma/Yazma kafaları
- İz (Track)
- Sektörler

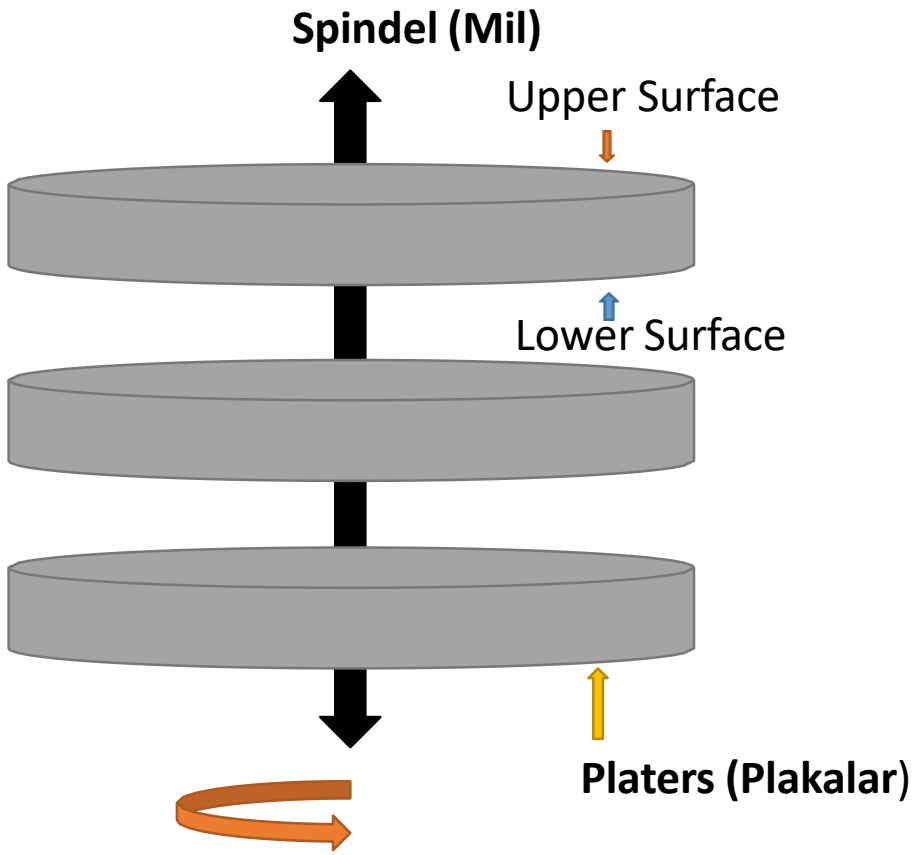
Plakalar (Platers)



Sabit diskler, eş merkezli bir disk yığını olarak düzenlenmiştir. Tek bir diske plaka denir.

Her tabak iki yüzeyden oluşur: bir alt ve bir üst yüzey.

Spindel

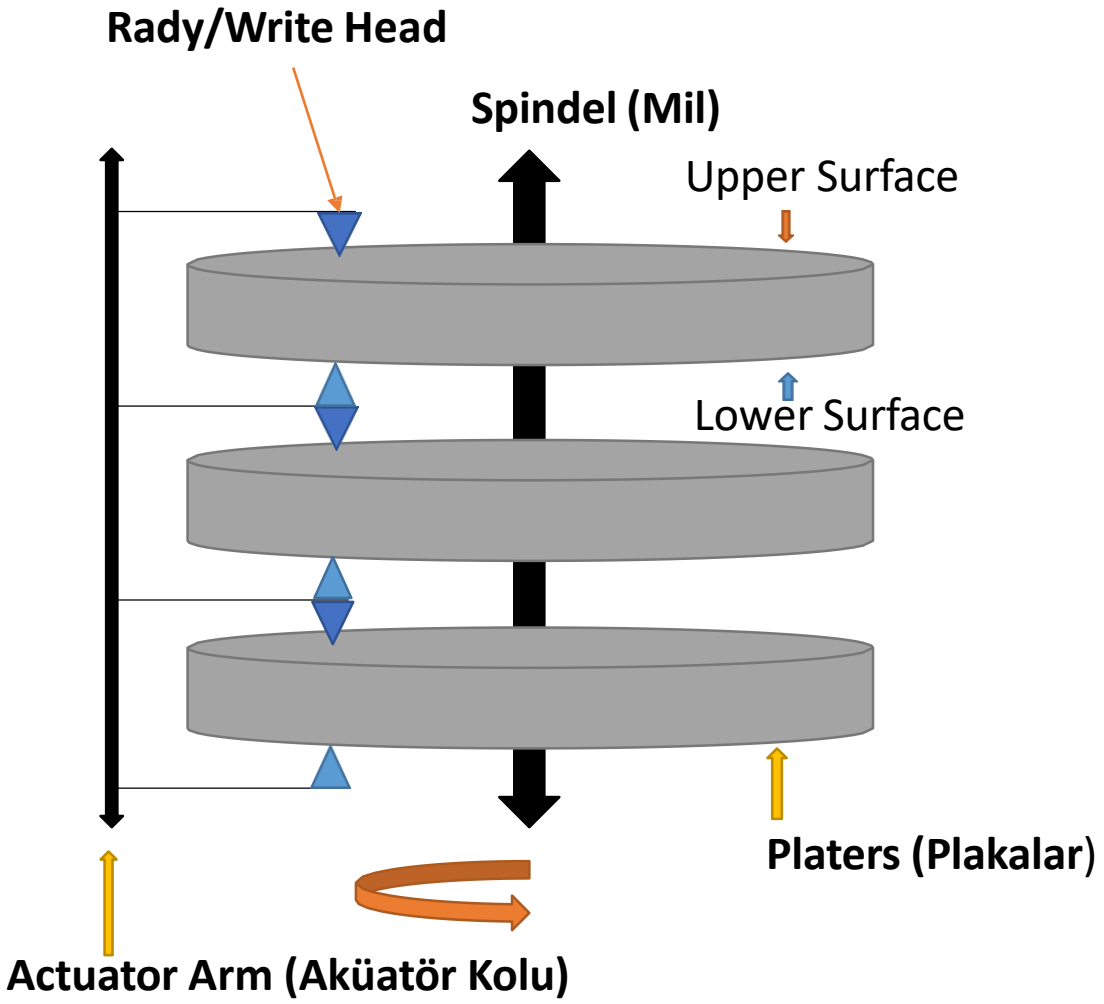


Sabit disk içindeki plakalar, plakaların ortasından geçen bir mil ile birbirine bağlanır.

Mil, ekseni boyunca tek yönlü olarak hareket eder (saat yönünde veya saat yönünün tersine).

Milin hareketi plakaların da dönmesine neden olur.

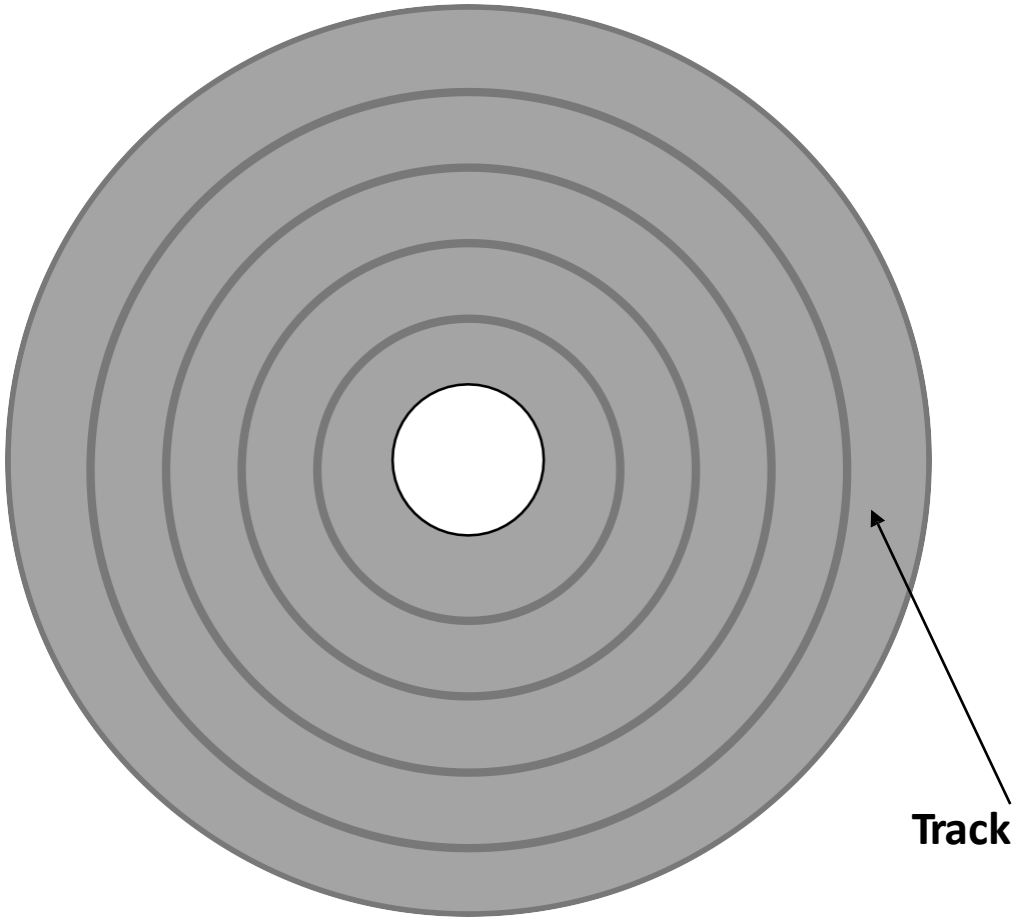
Okuma Yazma Kafası



Bir plaka üzerindeki her yüzey , diske veri okumak veya yazmak için kullanılan bir okuma/yazma kafası içerir.

Okuma/yazma kafaları bir tabağın yüzeyi boyunca ileri geri hareket edebilir. Okuma/yazma kafaları sırayla tek bir aktüatör koluna bağlanır .

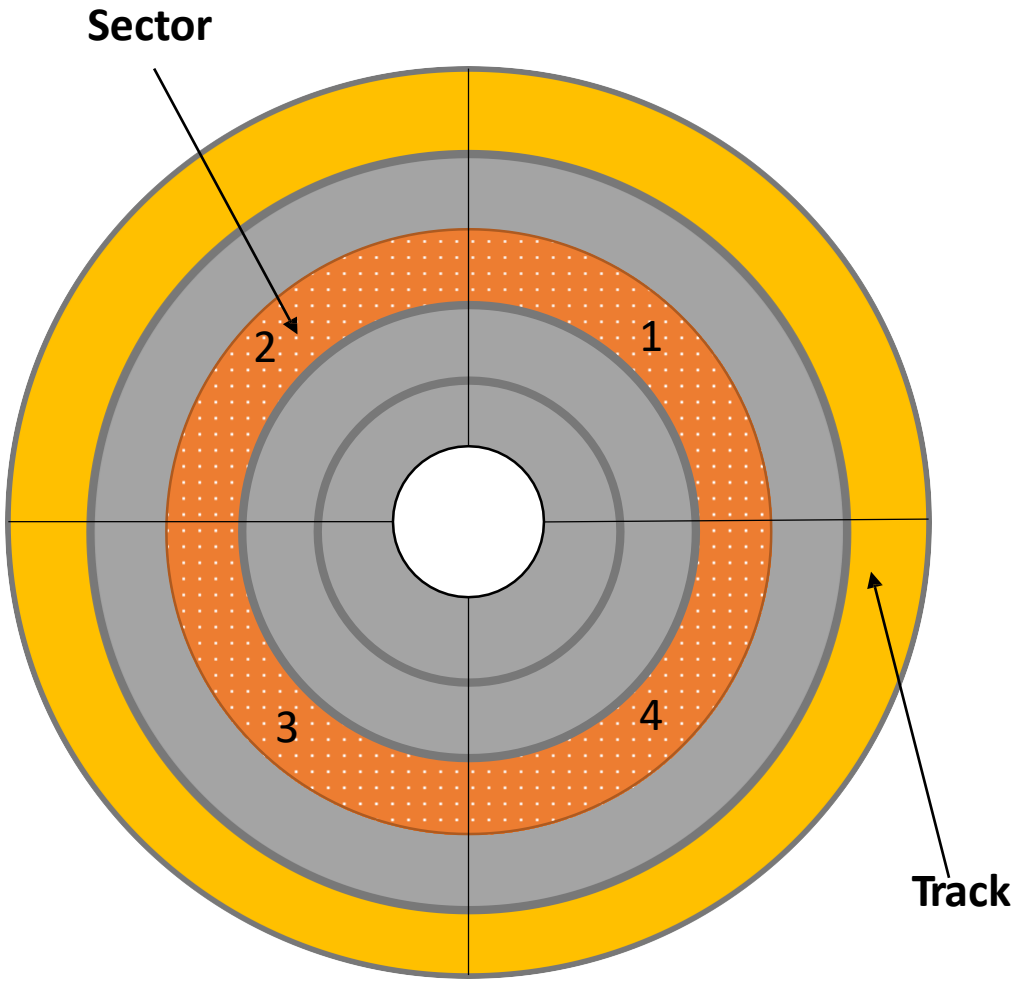
İz (Track)



Bir tabağın her yüzeyi sabit sayıda raydan oluşur . Bunlar, tabağın merkezine doğru hareket ettikçe çevresi azalan bir tabağın yüzeyindeki dairesel alanlardır.

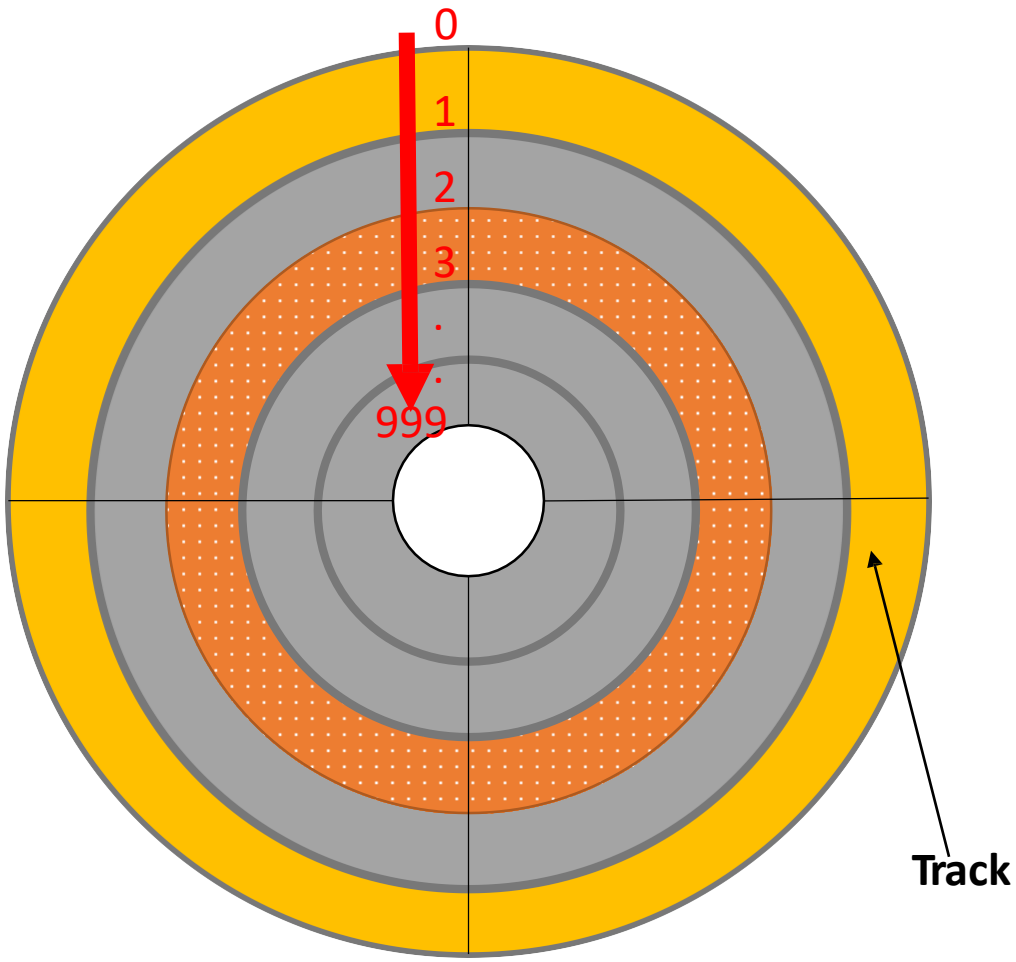
Veriler önce en dıştaki ize yazılır.

Sektör (Sector)



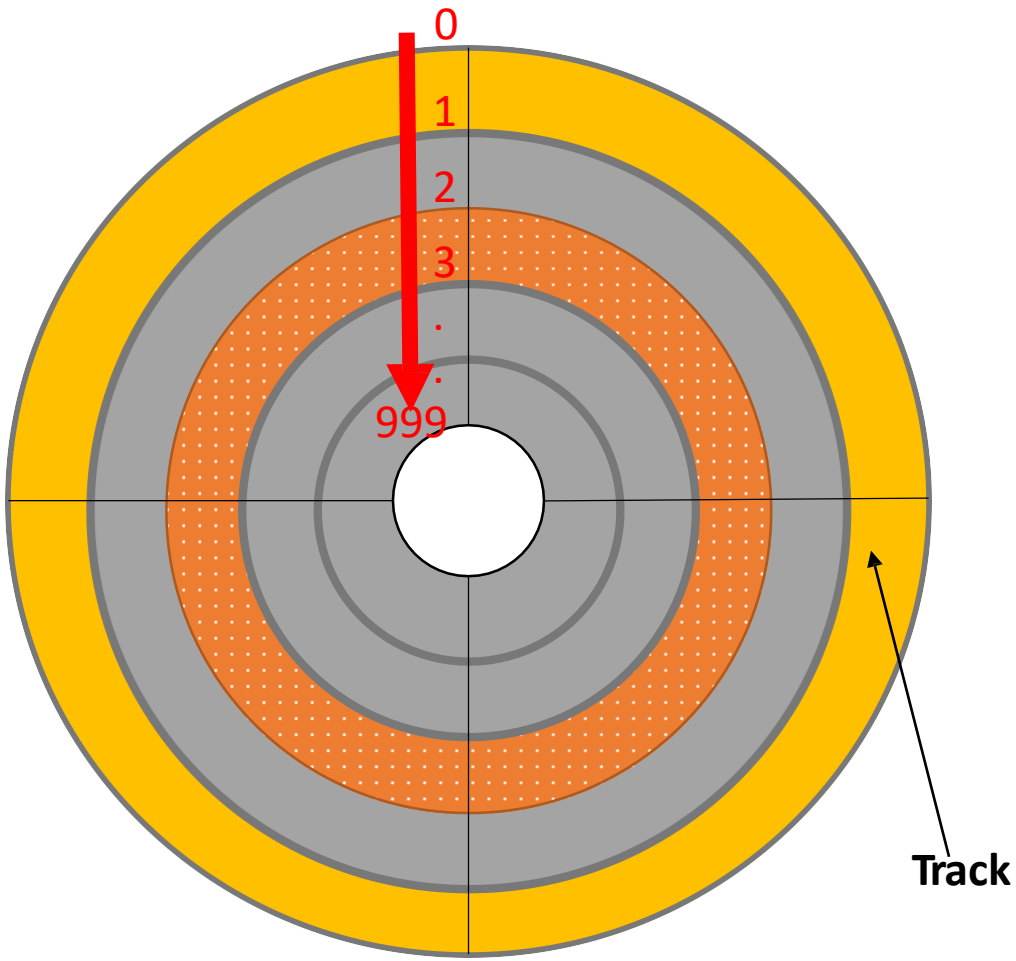
- Her iz sabit sayıda sektöre bölünmüştür.
- Sektörler, hat bölümlerini böler ve verileri depolar.

Sabit Disk Geometrisi ?

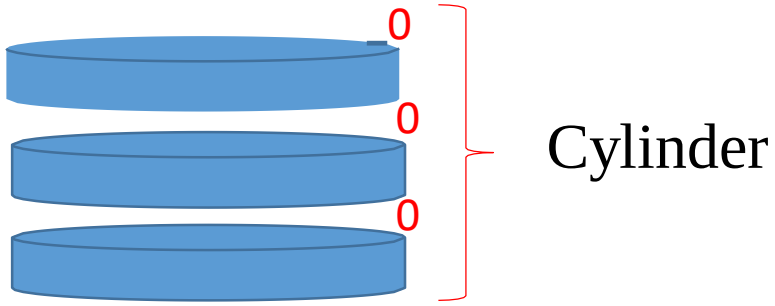


- Track'lar dışarıdan içe doğru 0'dan başlayarak numaralandırılır.
- Örnek, her bir plaka 1000 parçaya ayırdığımızda, her bir plakanın dışındaki ilk track 0, en iç track ise 999 olur.

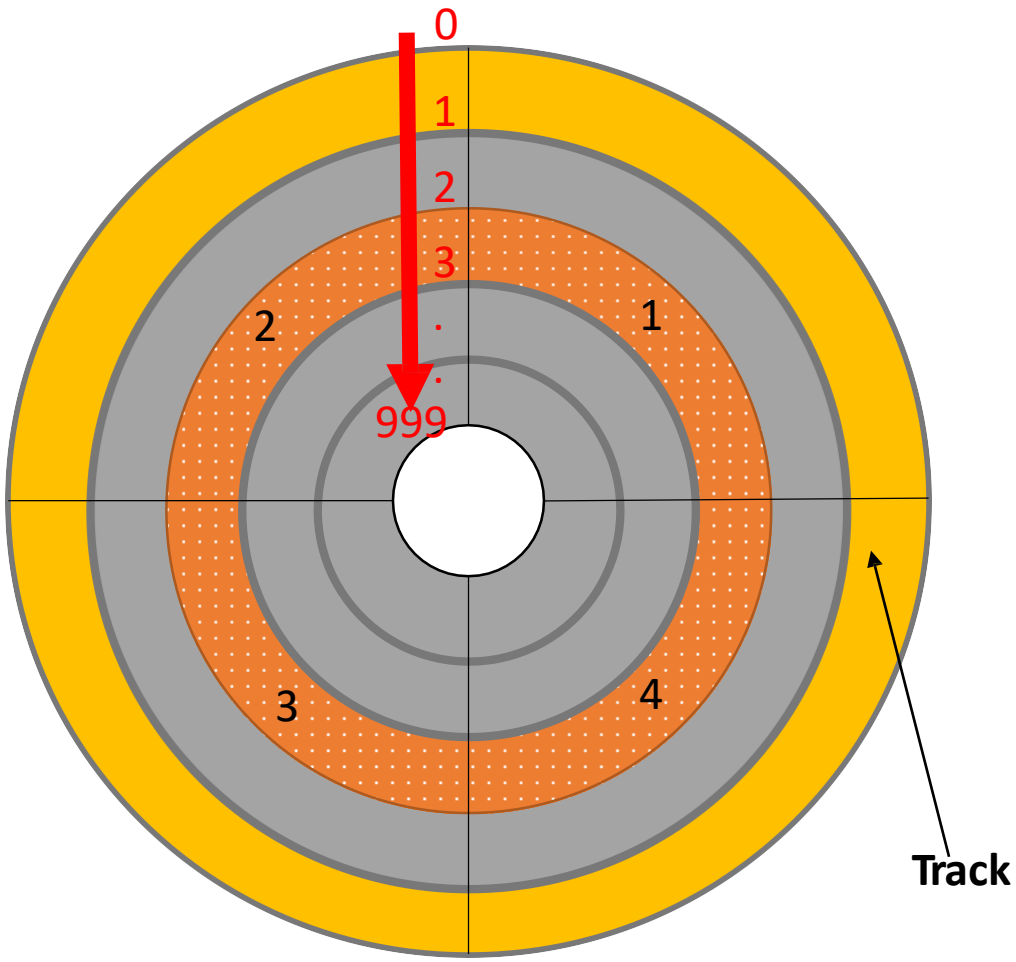
Sabit Disk Geometrisi ?



- Her plaka üzerinde bulunan Track'lara alt alta aynı adres atanır. Belirli bir adresteki tüm parçaları tanımlamak için Cylinder kullanılır.

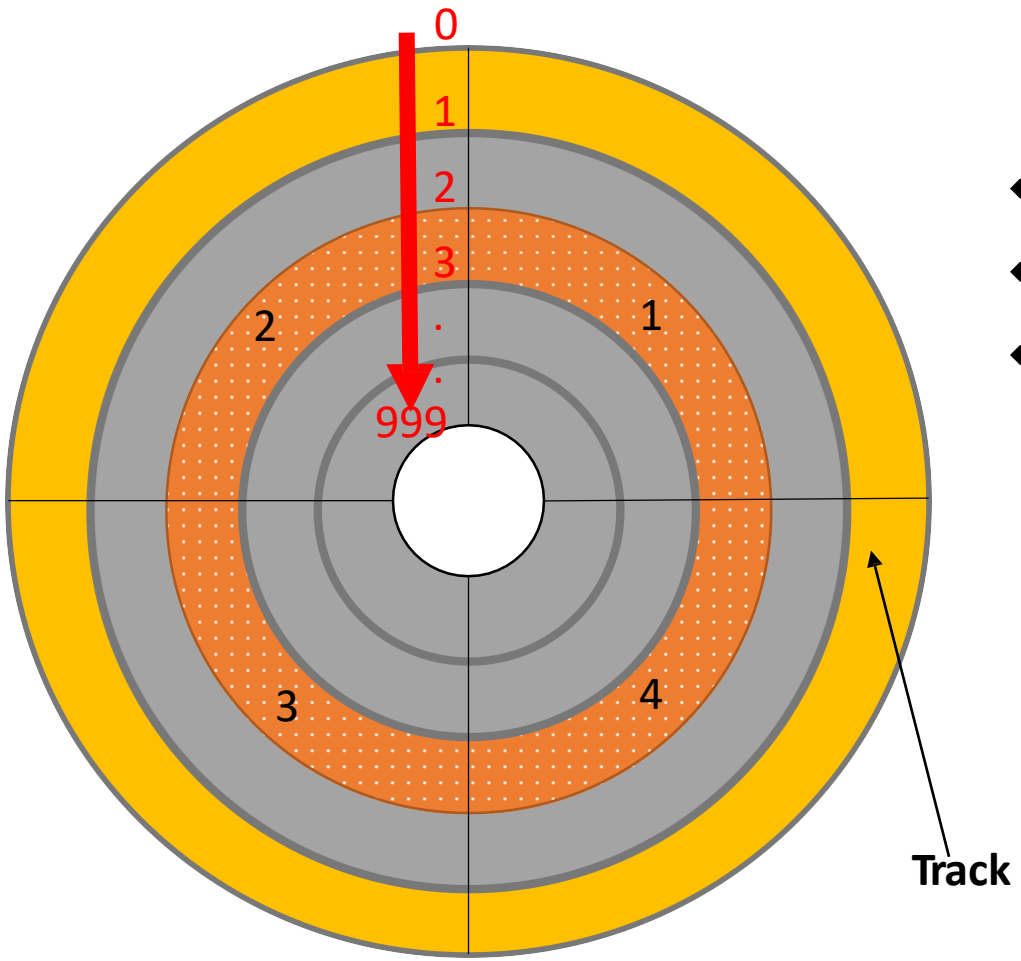


Sabit Disk Geometrisi ?

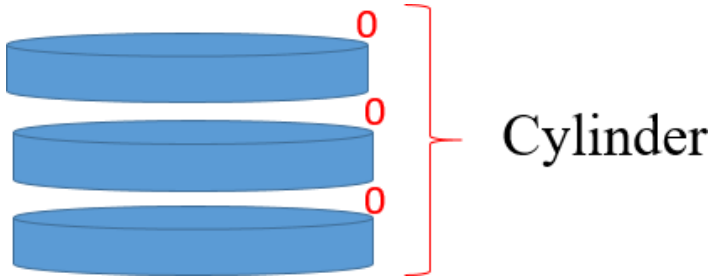


- Her bir track, sabit diskteki en küçük adreslenebilir depolama birimi olan ve genellikle 512 byte olan sektörlere ayrılmıştır.
- Her sektörde her bir track için 1'den başlayarak bir adres verilir.

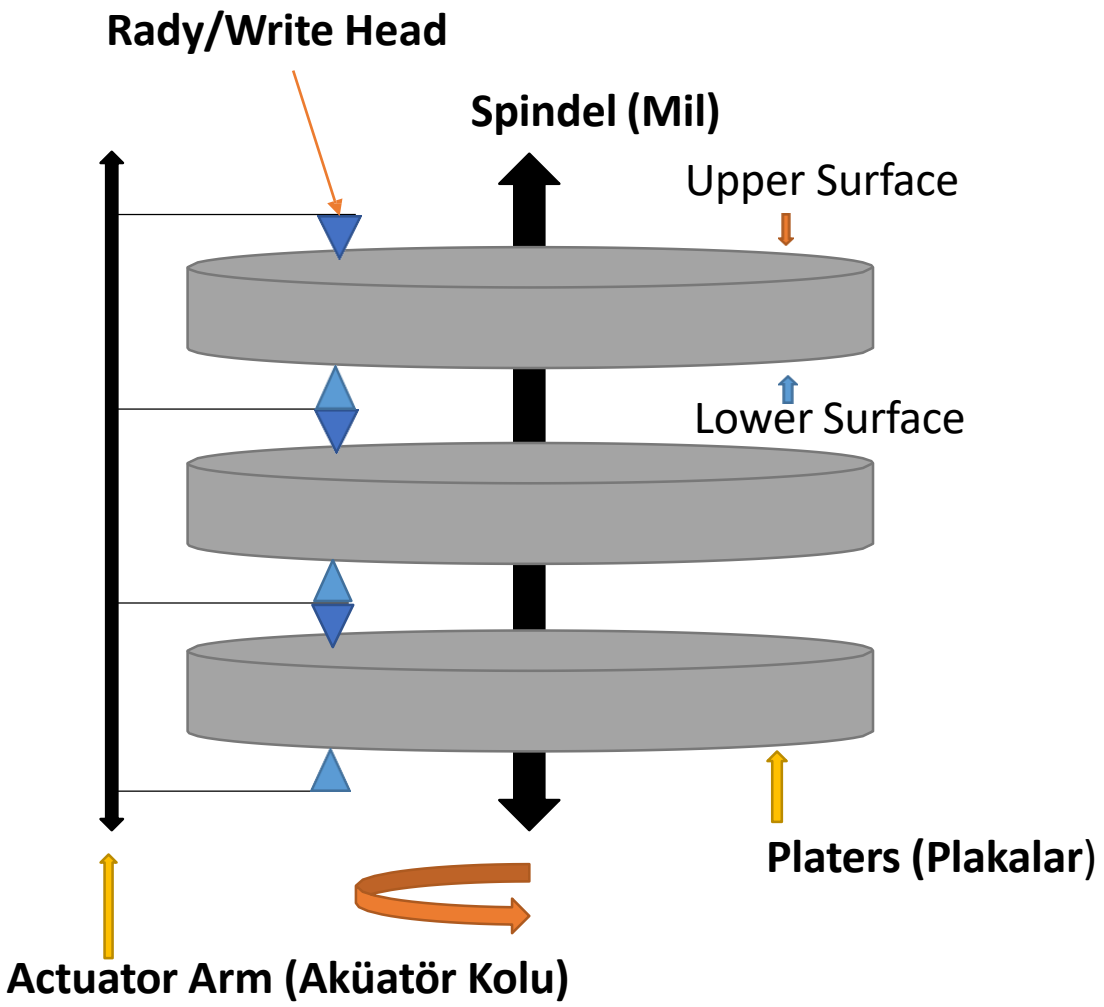
Sabit Disk Geometrisi ?

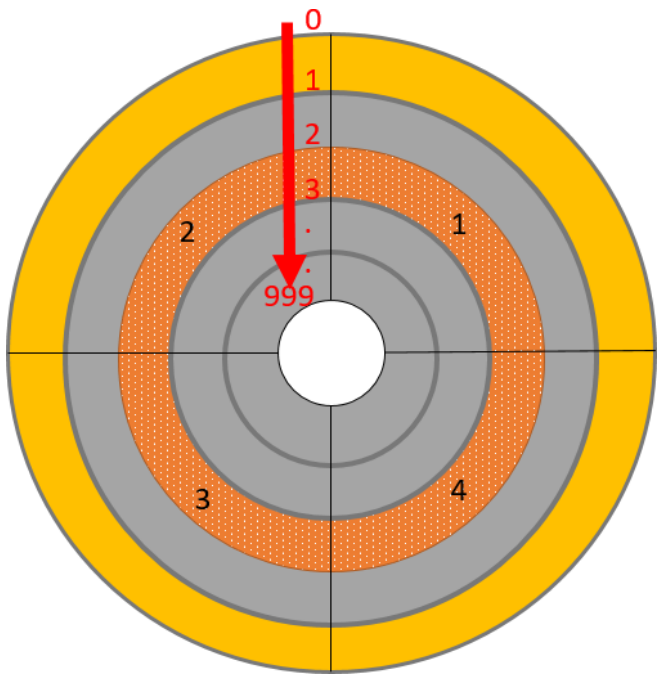


- Silindir adresi (C),
- Plaka ve kenarı alacak baş numarasını (H),
- Track üzerindeki sektörü göstermek için (S),



CHS


Rady/Write Head
Spindel (Mil)
Upper Surface
Lower Surface
Platers (Plakalar)
Actuator Arm (Aküatör Kolu)


0
1
2
3
999
1
2
3
4

Veri Yazılması ?

- Verilerin hard disk üzerine yazılabilmesi için öncelikle **hard disk üzerindeki yapının adreslemiş olması gerekir.**

- **Hard diske veri yazıldığında** ve veri çağırıldığında **veriyi organize etmek için kullanacağı adresleme tipi CHS'dir**

CHS Nedir ?

- CHS: Cylinder, Head ve Sector ‘un kısaltmasıdır ve bir hard disk üzerindeki verinin yerini belirtmek için kullanılan adresleme yöntemidir.

CHS Adresleme (Eski Bios'larda) ?

C: Silindir 10 bit (Geçerli aralık 0 ile 1023 silindir arasındadır)
H: Kafa 8 bit (Geçerli aralık 0 ile 255 silindir arasındadır)
S: Sektör 6 bit. (Geçerli aralık 1 ile 63 sektör arasındadır)

Adreslenebilir Boyut=(Sektör Boyutu)×(Sektör Sayısı)×(Kafa Sayısı)×(Silindir Sayısı)
512×63×256×1024=8064 MB disk boyutu sağlar.

Aslında Max Boyut 8 GB'dır ancak Track Üzerindeki sektör 1 den başladığı için boyutunu hesaptan düşersek

8x1024=8192 MB
1 Sektör 512 byte x 256 =128 MB'dır. → **8192-128=8064 MB**

Geçerli Aralıklar Fiziksel Gerçekleri Yansıtırmı?

- **128 plakalı (0-255 kafalı) sabit disk yoktur. Bu maksimum değerler**, bir zamanlar BIOS tarafından sabit diski adreslemek için kullanıldı.

- **Sabit disk denetleyicisi** bu değerleri daha sonra dahili olarak **gerçek özelliklere dönüştürerek sınırlı adresleme boyutunu arttırdı.**

- **C** ve **H** değerleri **0'dan başlar**; ve **S** **değeri 1'dedir.** Bir sabit disk için ilk sektör bu nedenle CHS adresi 0/0/1'de bulunur.

CHS Adresleme (Ata Spesifikasyonu) ?

C: Silindir 16 bit (Geçerli aralık 0 ile 65535 silindir arasındadır)
H: Kafa 4 bit (Geçerli aralık 0 ile 15 kafa arasındadır)
S: Sektör 8 bit. (Geçerli aralık 1 ile 255 sektör arasındadır)

Adreslenebilir Boyut=(Sektör Boyutu)×(Sektör Sayısı)×(Kafa Sayısı)×(Silindir Sayısı) **512×255×16×65536=130560** MB disk boyutu sağlar.

Alternatif Bir Adresleme Varmı ?

- Bilim ve teknolojinin hızla gelişmesiyle birlikte sabit disklerin kapasitesi büyük ölçüde artmıştır.
- Mevcut kapasiteler 8 GB içeren adresleme aralığının çok ötesindedir.
- Bu nedenle, 8 GB'ın üzerindeki bir alan adreslemek için CHS adres biti, 24'ten 28'e genişletilerek 128 GB'a ulaşan adresleme alanı elde edilmiştir.

LBA - Mantıksal Blok Adresleme

- LBA, sektörleri adreslemek için tamamen yeni bir yöntem sunar.
- LBA ile sabit disk, 0'dan başlayarak mevcut blokları basitçe sayarak adreslenir. Yani ***silindire, kafaya ve sektöre atıfta bulunmak yerine her sektöre benzersiz bir numara atar.***
- Bu yöntem ile 48 bit LBA adresleme modu kullanılarak adresleme alanı 144.000.000 GB'a kadar ulaşabilir.

LBA - Mantıksal Blok Adresleme

- 8 GB büyük sabit diskler için, uyumluluk nedenleriyle, önyükleme için, CHS adreslemesini desteği devam etmektedir.
- Modern BIOS'lar, Int13h kesme uzantılarını kullanarak Mantıksal Blok Adresleme (LBA) moduna geçebilir.
- LBA kullanılarak, sabit disk, 0'dan başlayarak mevcut blokları basitçe sayan tek, büyük bir aygıt olarak adreslenir. Dolayısıyla bir LBA bloğu, CHS adresleme kullanan bir sektöre karşılık gelir.

LBA - Mantıksal Blok Adresleme

- LBA 0 adresi, CHS 0,0,1 adresine ve LBA 1 adresi, CHS adresi 0,0,2 ye karşılık gelmektedir.

- Track içindeki tüm sektörler kullanıldığında, aynı silindirdeki bir sonraki kafadaki ilk sektör CHS adresi 0,1,1 olacaktır.

- ***Bazı sistemlerin halen CHS adreslerini kullandığından bu nedenle CHS'den LBA'ya dönüşüm yapabilmeyi bilmek gereklidir.***

CHS'dan LBA'ye Dönüşüm Formülü

LBA = (((C * TH) + H) * TS) + S – 1

C = Sektörün Silindir#
TH = Geçerli BIOS çevirisine göre sabit sürücünün Toplam Kafaları
TS = Sabit sürücünün Toplam Sektör sayısı
H = Sektörün kafası#
S = Sektörün Sektörü#

ÖRNEK: Disk Geometrisi 1020 16 63 olan CHS adresi 3/2/1 olan verinin LBA adresi nedir ?

CHS 3/2/1= ((3× 16)+2)× 63 + (1-1) = 3150 LBA'dır

LBA'dan CHS'ye Dönüşüm Formülü

- C , H ve S silindir numarası, kafa numarası ve sektör numarasıdır.
- LBA mantıksal blok adresidir
- HPC , silindir başına maksimum kafa sayısıdır (disk sürücüsü tarafından rapor edilir, genellikle 28 bit LBA için 16)
- SPT , iz başına maksimum sektör sayısıdır (disk sürücüsü tarafından rapor edilir, genellikle 28 bit LBA için 63)

ÖRNEK: LBA = 3150 ardesini CHS formatına dönüştürelim.

C = LBA ÷ (HPC × SPT)

C = 3150 ÷ (16 × 63) = 3.125 = 3

H = (LBA ÷ SPT) mod HPC

H= (3150 ÷ 63) mod 16

H= 50 mod 16 = 2

S = (LBA mod SPT) + 1

S = (3150 mod 63) + 1

S = (0) + 1 = 1

CHS= 3 2 1

LBA'dan CHS'ye Dönüşüm Formülü

- LBA adresleri, **aşağıdaki formülle CHS demetlerine eşlenebilir ("mod" modulo işlemidir , yani kalandır ve "÷" tamsayı bölümüdür , yani herhangi bir kesirli parçanın atıldığı bölümün bölümüdür) :**

$$C = LBA \div (HPC \times SPT)$$
$$H = (LBA \div SPT) \bmod HPC$$
$$S = (LBA \bmod SPT) + 1$$

ÖRNEK: LBA = 3150 ardesini CHS formatına dönüştürelim.

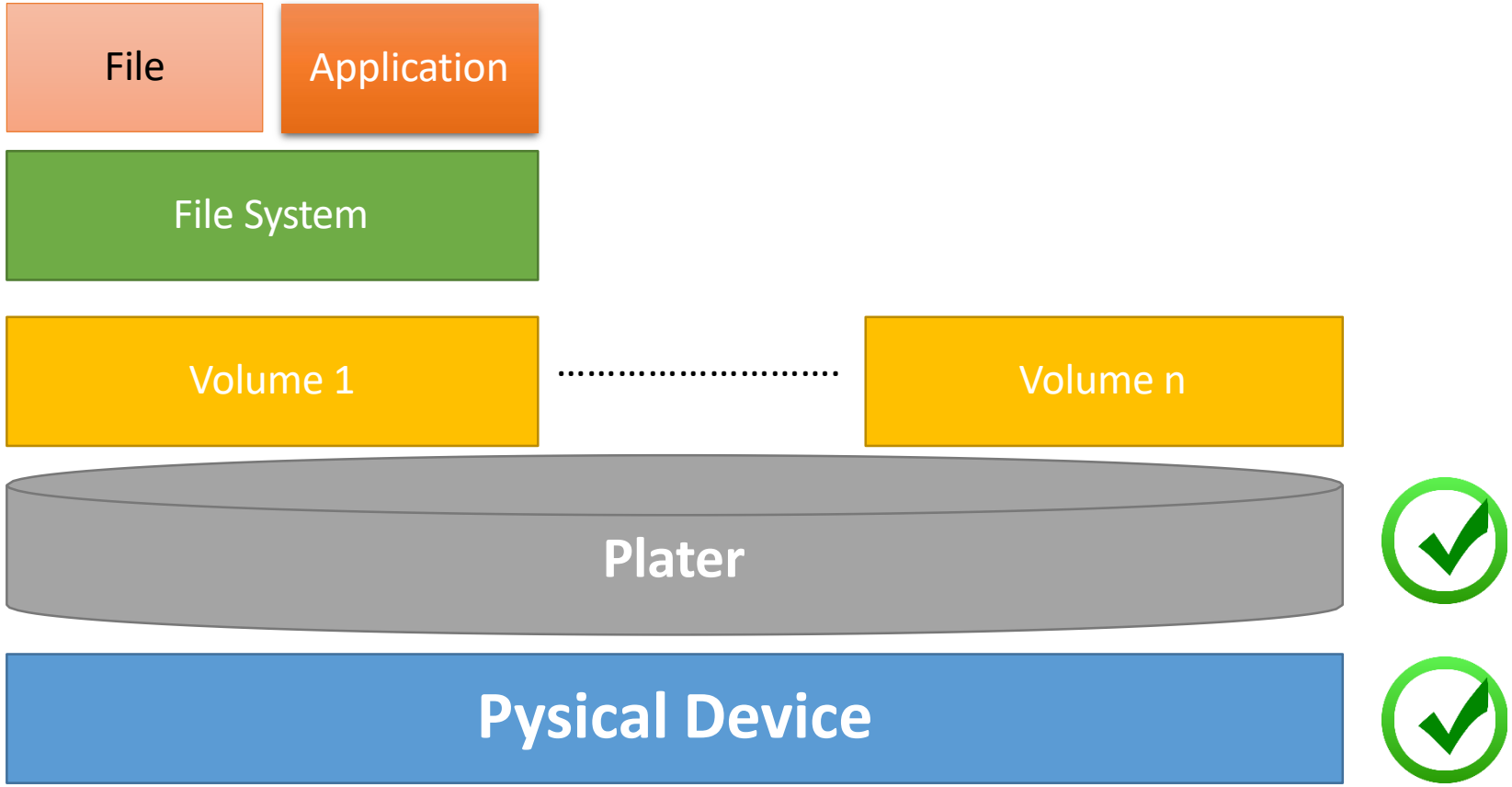
C = LBA ÷ (HPC × SPT)
C = 3150 ÷ (16 × 63) = 3.125 = 3

H = (LBA ÷ SPT) mod HPC
H= (3150 ÷ 63) mod 16
H= 50 mod 16 = 2

S = (LBA mod SPT) + 1
S = (3150 mod 63) + 1
S = (0) + 1 = 1

CHS= 3 2 1

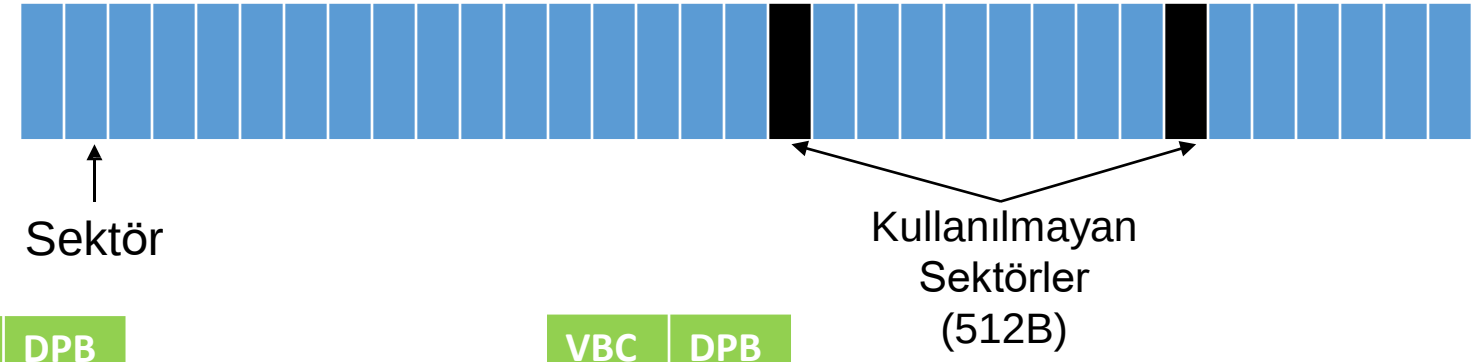
SABİT DİSK YAPISI



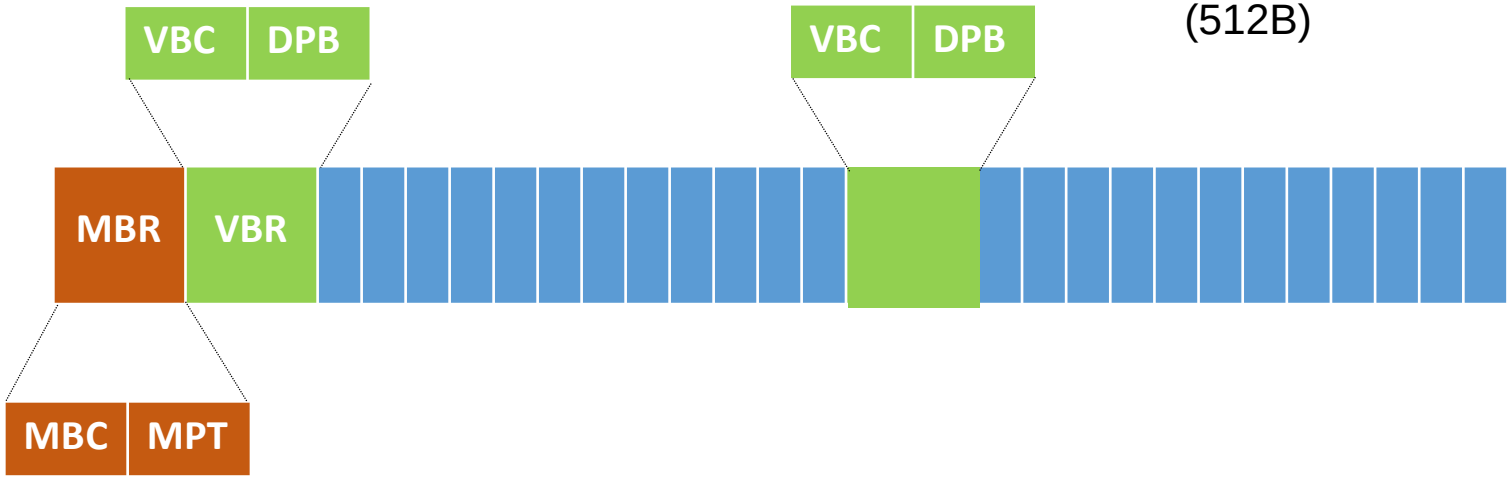
SABİT DİSK YAPISI

- Blank Media

- Low-Level Format

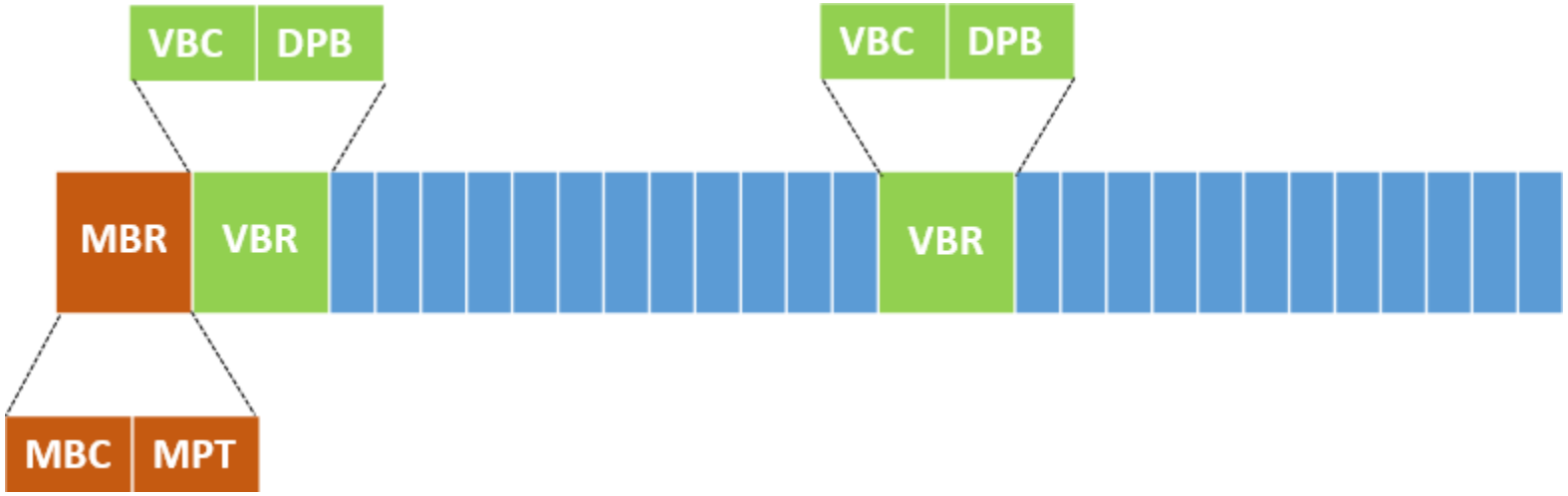


- Partition



SABİT DİSKLERİN MANTIKSAL YAPISI

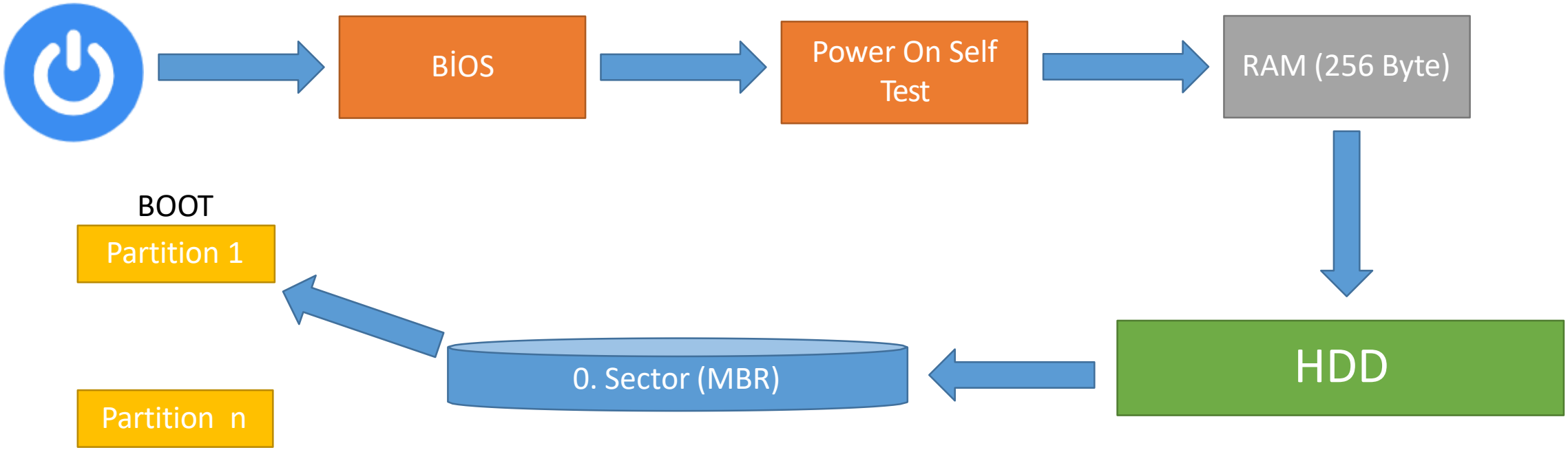
- Partition



MBR = master boot record
MBC = master boot code
MPT = master partition table

VBR = volume boot record
VBC = volume boot code
DPB = disk parameter block

ÖNYÜKLEME SÜRECİ



MBR (Master Boot Record)

- Ana Önyükleme Kaydı (MBR), bir sabit diskin veya çıkarılabilir bir sürücünün ilk sektöründeki bilgidir .

- Bilgisayarın ana deposunda İşletim sisteminin (OS) yüklü bulunduğu bölüm bilgilerini tutar.

- MBR ayrıca, disk üzerindeki bölümlerin adreslerini içeren bir tabloyu da içerisinde barındırır.

MBR (Master Boot Record)

Master Boot Code (MBC)

Master Partition Table (MPT)

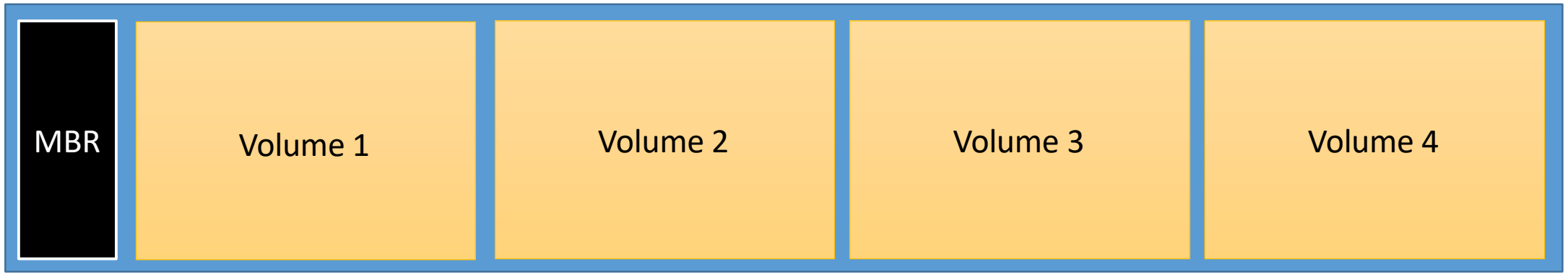
```
0000000000 33 C0 8E D0 BC 00 7C 8E-C0 8E D8 BE 00 7C BF 00  3À·Ð¼·|·À·Ø¾·|¿
0000000010 06 B9 00 02 FC F3 A4 50-68 1C 06 CB FB B9 04 00  ·¹··üó¤Ph··Ëû¹··
0000000020 BD BE 07 80 7E 00 00 7C-0B 0F 85 0E 01 83 C5 10  ½¾··~··|·····Å·
0000000030 E2 F1 CD 18 88 56 00 55-C6 46 11 05 C6 46 10 00  âñÍ··V·UÆF··ÆF··
0000000040 B4 41 BB AA 55 CD 13 5D-72 0F 81 FB 55 AA 75 09  ´A»ªUÍ·]r··ûUªu·
0000000050 F7 C1 01 00 74 03 FE 46-10 66 60 80 7E 10 00 74  ÷Á··t·þF·f`·~··t
0000000060 26 66 68 00 00 00 00 66-FF 76 08 68 00 00 68 00  &fh····fÿv·h··h·
0000000070 7C 68 01 00 68 10 00 B4-42 8A 56 00 8B F4 CD 13  |h··h··´B·V··ôÍ·
0000000080 9F 83 C4 10 9E EB 14 B8-01 02 BB 00 7C 8A 56 00  ··Ä··ë·¸··»·|·V·
0000000090 8A 76 01 8A 4E 02 8A 6E-03 CD 13 66 61 73 1C FE  ·v··N··n·Í·fas·þ
00000000a0 4E 11 75 0C 80 7E 00 80-0F 84 8A 00 B2 80 EB 84  N·u··~·····ª·ë·
00000000b0 55 32 E4 8A 56 00 CD 13-5D EB 9E 81 3E FE 7D 55  U2ä·V·Í·]ë··>þ}U
00000000c0 AA 75 6E FF 76 00 E8 8D-00 75 17 FA B0 D1 E6 64  ªunÿv·è··u·ú°Ñæd
00000000d0 E8 83 00 B0 DF E6 60 E8-7C 00 B0 FF E6 64 E8 75  è··°ßæ`è|·°ÿædèu
00000000e0 00 FB B8 00 BB CD 1A 66-23 C0 75 3B 66 81 FB 54  ·û¸·»Í·f#Àu;f·ûT
00000000f0 43 50 41 75 32 81 F9 02-01 72 2C 66 68 07 BB 00  CPAu2·ù··r,fh·»·
0000000100 00 66 68 00 02 00 00 66-68 08 00 00 00 66 53 66  ·fh····fh····fSf
0000000110 53 66 55 66 68 00 00 00-00 66 68 00 7C 00 00 66  SfUfh····fh·|··f
0000000120 61 68 00 00 07 CD 1A 5A-32 F6 EA 00 7C 00 00 CD  ah···Í·Z2öê·|··Í
0000000130 18 A0 B7 07 EB 08 A0 B6-07 EB 03 A0 B5 07 32 E4  ·····ë·¶·ë··µ·2ä
0000000140 05 00 07 8B F0 AC 3C 00-74 09 BB 07 00 B4 0E CD  ····ð¬<·t·»··´·Í
0000000150 10 EB F2 F4 EB FD 2B C9-E4 64 EB 00 24 02 E0 F8  ·ëòôëý+Éädë·$·àø
0000000160 24 02 C3 49 6E 76 61 6C-69 64 20 70 61 72 74 69  $·ÃInvalid parti
0000000170 74 69 6F 6E 20 74 61 62-6C 65 00 45 72 72 6F 72  tion table·Error
0000000180 20 6C 6F 61 64 69 6E 67-20 6F 70 65 72 61 74 69   loading operati
0000000190 6E 67 20 73 79 73 74 65-6D 00 4D 69 73 73 69 6E  ng system·Missin
00000001a0 67 20 6F 70 65 72 61 74-69 6E 67 20 73 79 73 74  g operating syst
00000001b0 65 6D 00 00 00 63 7B 9A-C1 C3 01 92 78 78 80 20  em···c{·ÁÃ··xx··
00000001c0 21 00 07 1D 17 46 00 08-00 00 00 28 11 00 00 1D  !····F·····(····
00000001d0 18 46 07 FE FF FF 00 30-11 00 00 60 90 12 00 00  ·F·þÿÿ·0···`····
00000001e0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ····************
00000001f0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 55 AA  ··************U·
```

signature

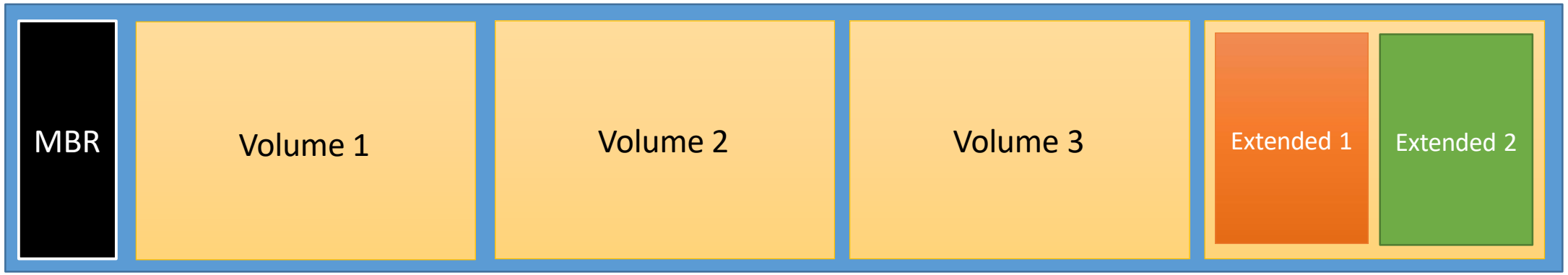
MBT (Master Boot Table)

byte'lar (16 Byte)	Açıklama
0. Byte	''80'' İşletim Sistemi Var ''00'' Bootable İşletim Sitemi Yok
1-3 Byte	CHS formatında ilk sektörün adresi
4.Byte	Dosya Sistemi Türü Hakkında Bilgi Verir. Örneğin: ''07'' NTFS , ''0B'' FAT32
5-7 Byte	CHS formatında sektör adresi
8-11 Byte	LBA formatında ilk sektör adresi
12-15 Byte	LBA formatında Total sektör sayısı

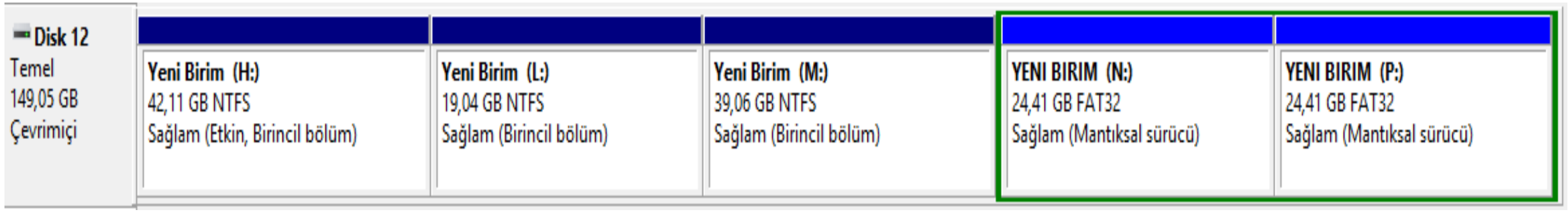
Extended Partition (Genişletilmiş Bölüm)



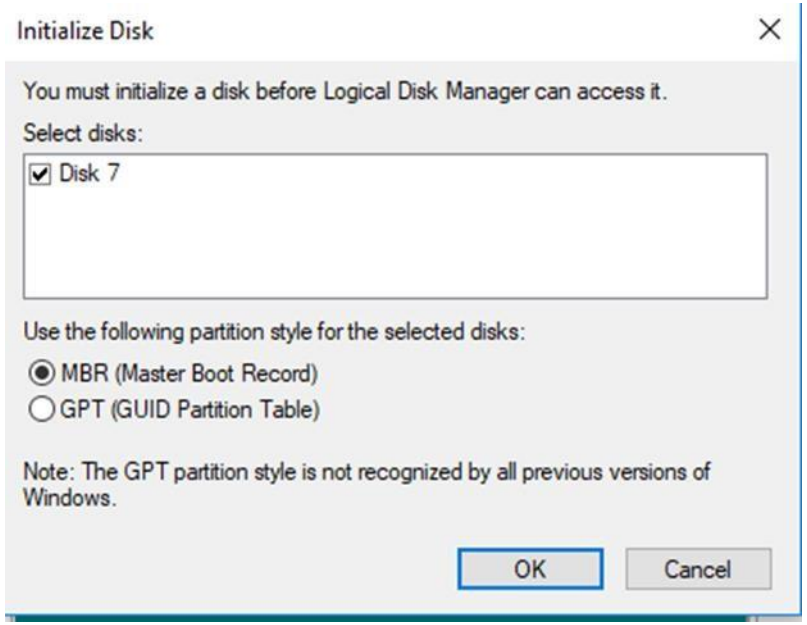
Extended Partition (Genişletilmiş Bölüm)



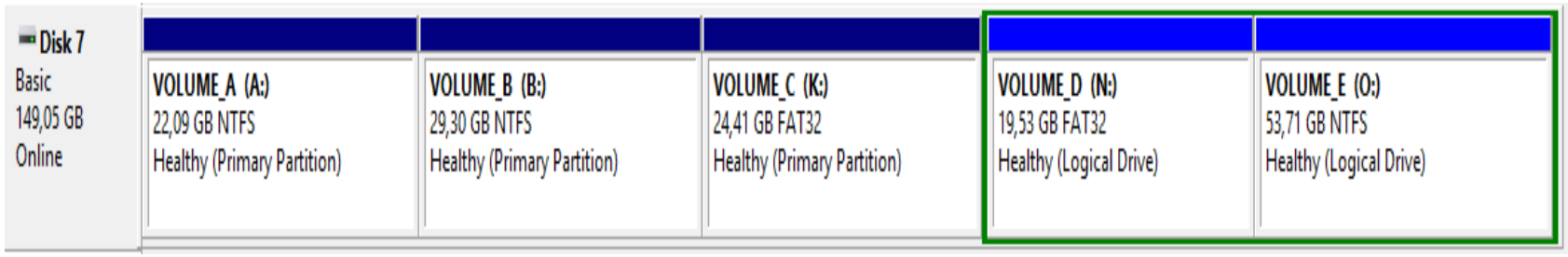
Extended Partition (Genişletilmiş Bölüm)

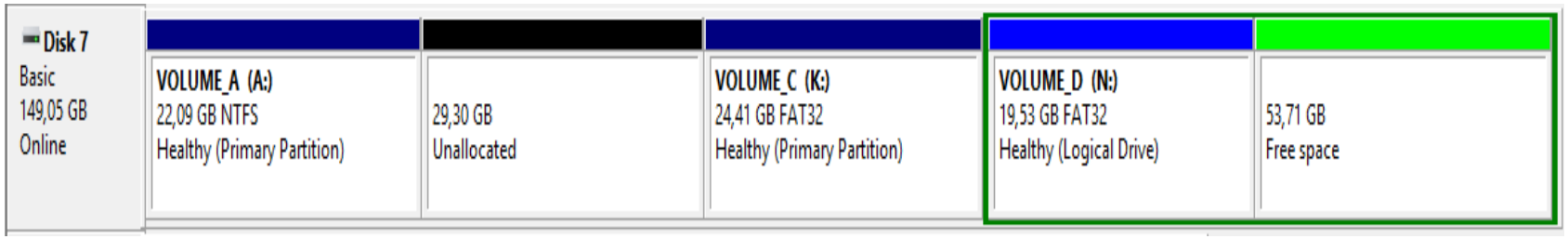


Örnek Genişletilmiş Bölüm Oluşturma



Örnek Genişletilmiş Bölüm Oluşturma





Extended Partition (Genişletilmiş Bölüm)

ID	Type	Start Sector	Total Sectors	Size
07	NTFS	2.048	46.336.000	22,1 GB
0c	FAT32X	107.778.048	51.200.000	24,4 GB
0c	FAT32X	158.980.033	40.960.000	19,5 GB

Extended Partition (Genişletilmiş Bölüm)

ID	Type	Start Sector	Total Sectors	Size
07	NTFS	2.048	46.336.000	22,1 GB
00	Recovered	46.337.985	61.440.062	29,3 GB
0c	FAT32X	107.778.048	51.200.000	24,4 GB
0c	FAT32X	158.980.033	40.960.000	19,5 GB

Extended Partition (Genişletilmiş Bölüm)

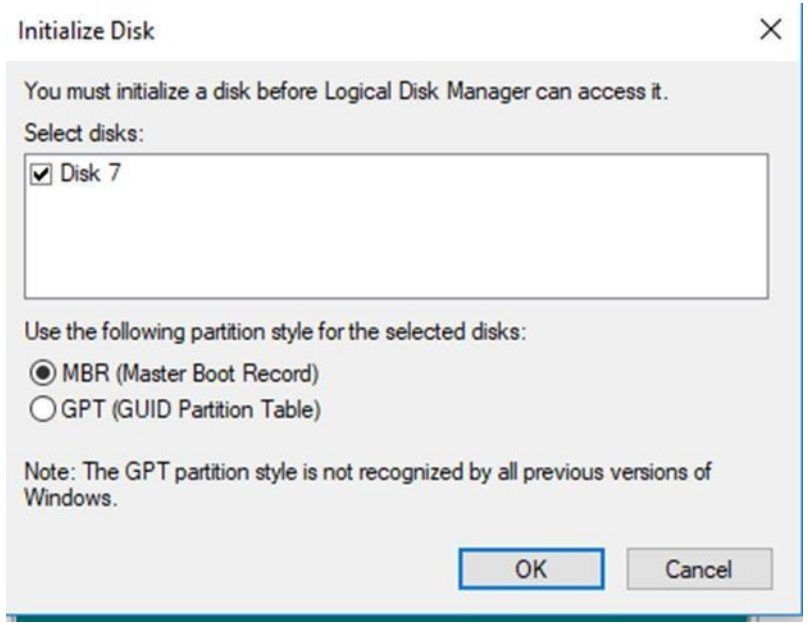
ID	Type	Start Sector	Total Sectors	Size
07	NTFS	2.048	46.336.000	22,1 GB
00	Recovered	46.337.985	61.440.062	29,3 GB
0c	FAT32X	107.778.048	51.200.000	24,4 GB
0c	FAT32X	158.980.033	40.960.000	19,5 GB
00	Recovered	199.942.081	112.635.966	53,7 GB

GPT (Global Unic İdentities Partition Table)

- GUID Bölüm Tablosu, Intel tabanlı bilgisayarlarda yaygın olan eski Ana Önyükleme Kaydı (MBR) bölümleme şemasını genişleten yeni bir disk mimarisidir.

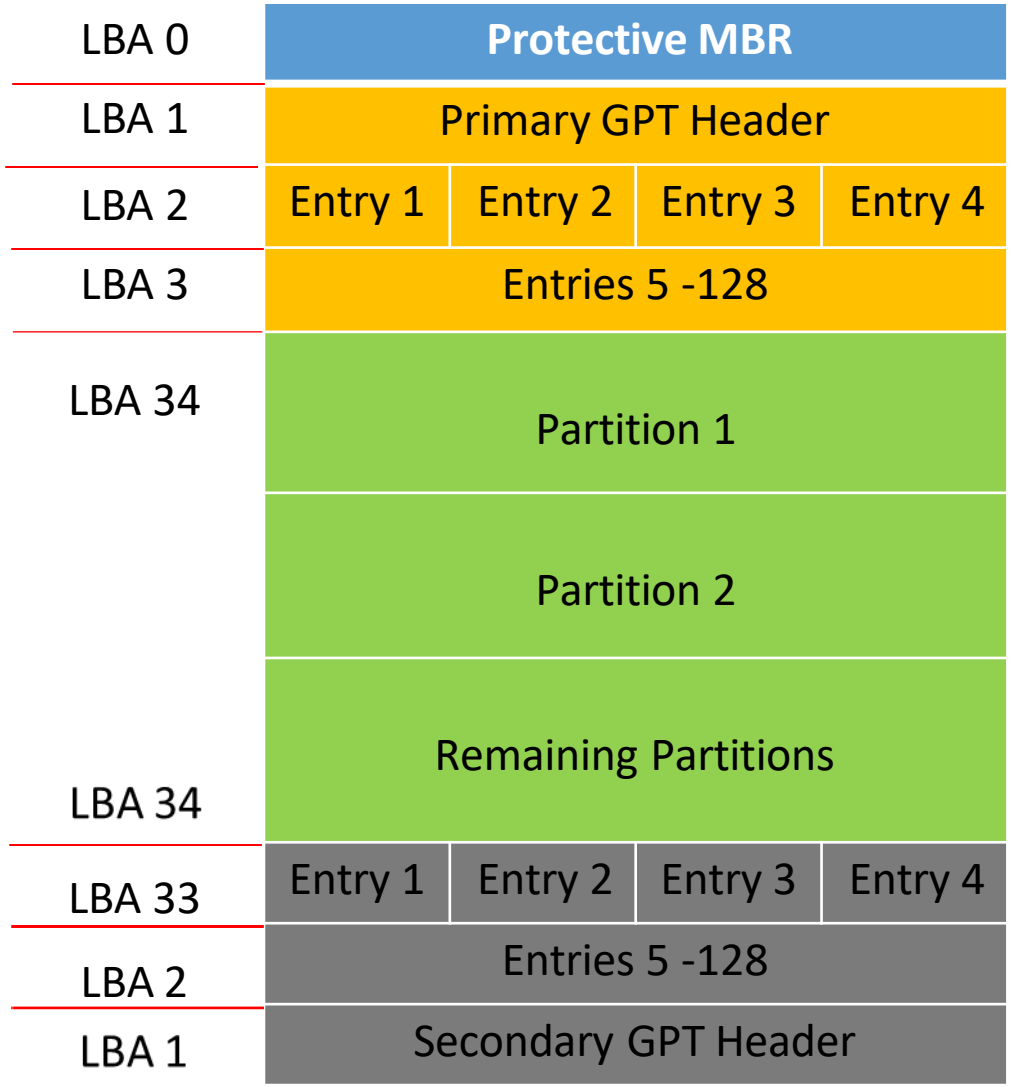
- BİOS, MBR Kullanırken EFI, GPT kullanır.

- MBR, diski tanımlamak ve boot etmek için MBC kullanırken, EFI bunu tek başına yapabilir. Ancak geriye dönük uyuşmayı sağlamak için GPT başlığında MBR içerir



EFI (Extensible Firmware Interface), bilgisayarın başlangıç aşamasında (boot) donanım ve işletim sistemi arasında bir köprü olarak görev yapan **bir tür firmware veya arayüzdür**. **EFI, bilgisayarın donanımını başlatır** ve **işletim sistemi yüklenmeden önce temel sistem ayarlarını yapılandırır.**

GPT (Global Unic İdentities Partition Table)



LBA 0

```
0000000000 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000010 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000020 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000030 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000040 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000050 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000060 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000070 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000080 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000090 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000000a0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000000b0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000000c0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000000d0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000000e0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000000f0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000100 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000110 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000120 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000130 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000140 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000150 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000160 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000170 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000180 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
0000000190 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000001a0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000001b0 00 00 00 00 00 00 00 00-04 F4 AC C2 00 00 00 00
00000001c0 02 00 EE FE FF 33 01 00-00 00 FF FF FF FF 00 00
00000001d0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000001e0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000001f0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 55 AA
```

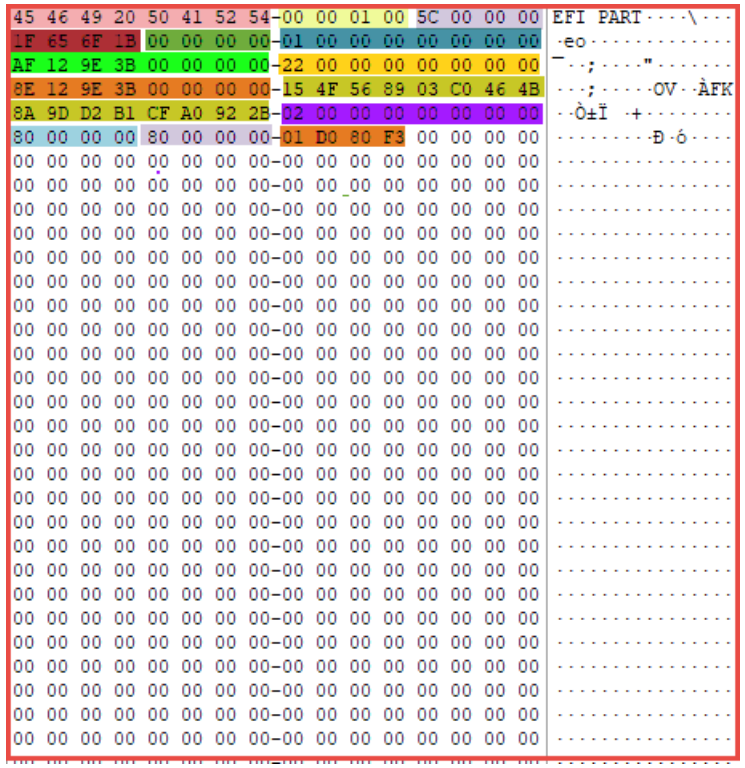
Protective MBR	
440 Byt	UEFI Tarafından Kullanılmayan Alan
440-443	Kullanılmamış ve Sıfıra ayarlanmış alan
443-445	Kullanılmamış ve Sıfıra ayarlanmış alan
446-509	Kullanılmayan ancak efı bölümüne işaret eden yanlızca bir bölüm kaydı olan MBR bölüm kaydına ayarlanmıştır
510-511	MBR bölüm imzası

```
00000001b0  00 00 00 00 00 00 00 00-04 F4 AC C2 00 00 00 00
00000001c0  02 00 EE FE FF 33 01 00-00 00 FF FF FF FF 00 00
00000001d0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000001e0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00
00000001f0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 55 AA
```

Protective MBR	
00	UEFI Tarafından Göz ardı edilen bir değerdir
1-3	Başlangıç LBA alanına karşılık gelir 00 02 00 olarak ayarlanır.
4	EE EFI bölüm türünü belirtir.
5-7	Diskteki son mantıksal bölümün CHS adresini temsil eder, bu mümkün değilse FF FF FF olarak temsil edilir.
8-11	00 00 00 01 olarak belirtililir GPT bölüm başlığının yerini belirtir.
12-15	Diskin Boyutu bu alanda gösterilmeyecek kadar büyükse FF FF FF FF olarak ayarlanır.

LBA 1

0-7	8 Byte	" EFI BÖLÜM İmzası " , 45 46 49 20 50 41 52 54
8-11	4 Byte	Revizyon (GPT sürüm 1.0 için (en az UEFI sürüm 2.3.1 aracılığıyla), değer 00 00 01 00'dir)
12-15	4 Byte	Little endian olarak başlık boyutu (bayt cinsinden, genellikle 5C 00 00 00 = 92 bayt anlamına gelir)
16-19	4 Byte	Başlığın CRC32'si (0'dan başlık boyutuna), bu alan hesaplama sırasında sıfırlanır
20-23	4 Byte	Rezerve ; sıfır olmalı
24-31	8 Byte	Mevcut LBA (bu başlık kopyasının yeri)
32-39	8 Byte	Yedek LBA (diğer başlık kopyasının yeri)
40-47	8 Byte	Bölümler için ilk kullanılabilir LBA (birincil bölüm tablosu son LBA + 1)
48-55	8 Byte	Kullanılabilir son LBA (ilk ikincil bölüm tablosu LBA - 1)
56-71	16 Byte	Disk GUID'si (UNIX'lerde UUID olarak da anılır)
72-79	8 Byte	LBA'dan başlayan bölüm girişleri (birincil kopyada her zaman 2)
80-83	4 Byte	Bölüm girişlerinin sayısı (128 adet girdi)
84-87	4 Byte	Bölüm girişlerinin sayısı Bölüm girişinin boyutu (genellikle 128)
88-91	4 Byte	bölüm dizisinin CRC 32
92	*	Rezerve ; bloğun geri kalanı için sıfır olmalıdır (512 baytlık bir LBA için 420 bayt)

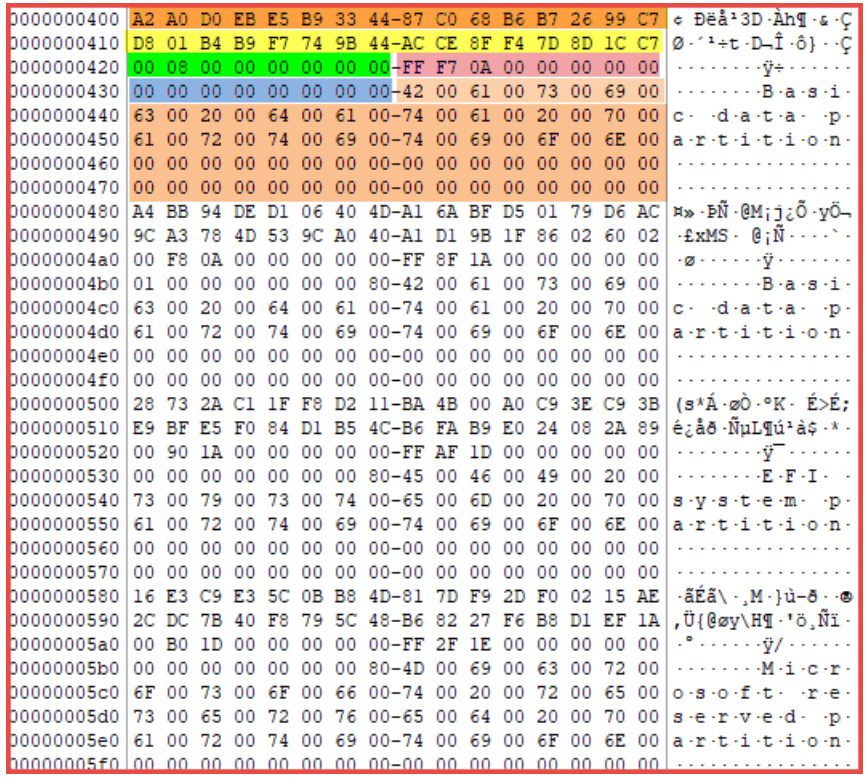


LBA 1

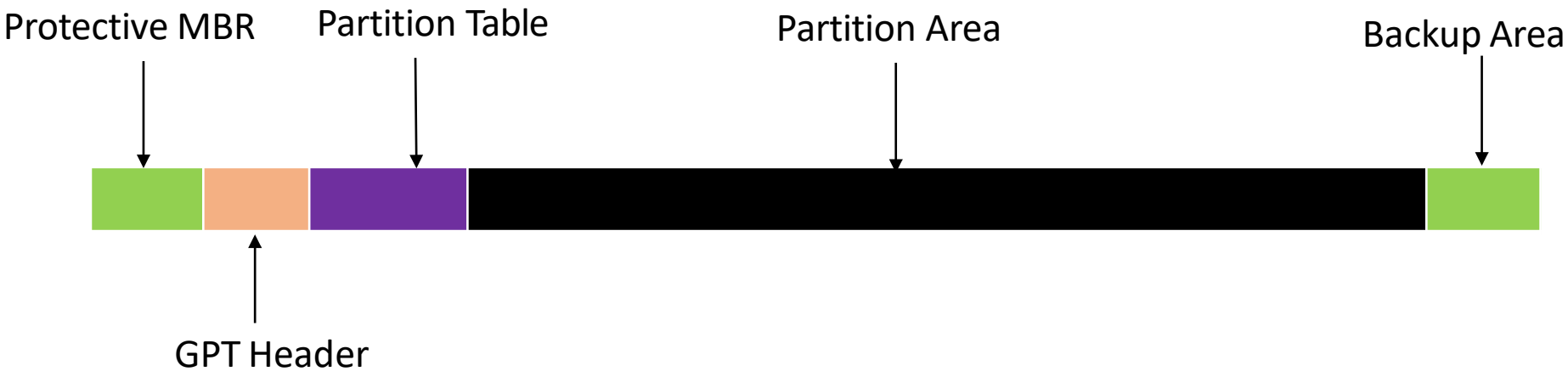
- Bölüm Tablo Başlıkları kullanıcının disk üzerinde faydalanabileceği blokları tanımlar.
- Bu tabloyu oluşturan bölümlendirme girdilerinin sayısı ve boyutlarını da tanımlar.
- 64 bit Windows Server 2003'lerde 128 adet bölümlendirme girdisi vardır ve hepsi 128 bayt büyüklüğündedir. Bu yüzden 128 tane bölümlendirme yapılabilir.
- Başlık diskin GUID'ini içerir.
- Başlık kendi boyutunu, yerini (her zaman LBA1) ve ikincil GPT başlık ve tablosunun yerini kaydeder.
- Ayrıca kendisi ve tablo için EFI tarafından boot sarında onaylanacak CRC32 sağlama toplamını (checksum) da içerir. ***EFI bu kontrolü yaptığı için hex düzenleyicilerle GPT'nin içeriği değiştirilmemeli. Böyle değişimler sağlama toplamının (checksum) hata vermesine neden olacaktır. Böyle bir durumda EFI ikincil GPT'nin üzerine yazacak, o da aynı hatayı verirse diske erişemeyecektir.***

LBA 2

Partition Type GUID	0-15	16 Byte	Bu Bölümün amacını ve türünü tanımlayan benzersiz kimlik değeridir. Sıfır değeri, bu bölüm girişinin kullanılmadığını tanımlar.
Unique Partition GUID	16-31	16 Byte	Her bölüm girişi için benzersiz bir GUID değeridir. Şimdiye kadar oluşturulan her bölümün benzersiz bir GUID'si olacaktır. Bu GUID, GPT Bölüm Girişi oluşturulduğunda atanmalıdır. GPT Bölümü Girişi, GPT Başlığındaki NumberOfPartition Girişleri daha geniş bir adres aralığı içerecek şekilde artırıldığında oluşturulur.
Starting LBA	32-39	8 Byte	Bu giriş tarafından tanımlanan bölümün LBA'sının başlangıcı
Ending LBA	40-47	8 Byte	Bu girdi tarafından tanımlanan bölümün LBA'sının sonu .
Attirubutes	48-55	8 Byte	Öznitelik bitleri, UEFI tarafından ayrılmış tüm bitler
Partition Name	56-127	72 Byte	Bölümün okunabilir bir adını içeren boş sonlandırılmış dize,
Reserve	128	Bölüm Girişi Boyutu - 128	Varsa GPT PartitionEntry'nin geri kalanı UEFI tarafından ayrılmıştır ve sıfır olmalıdır.



GPT GENEL YAPISI



MBR VE GPT FARKLARI

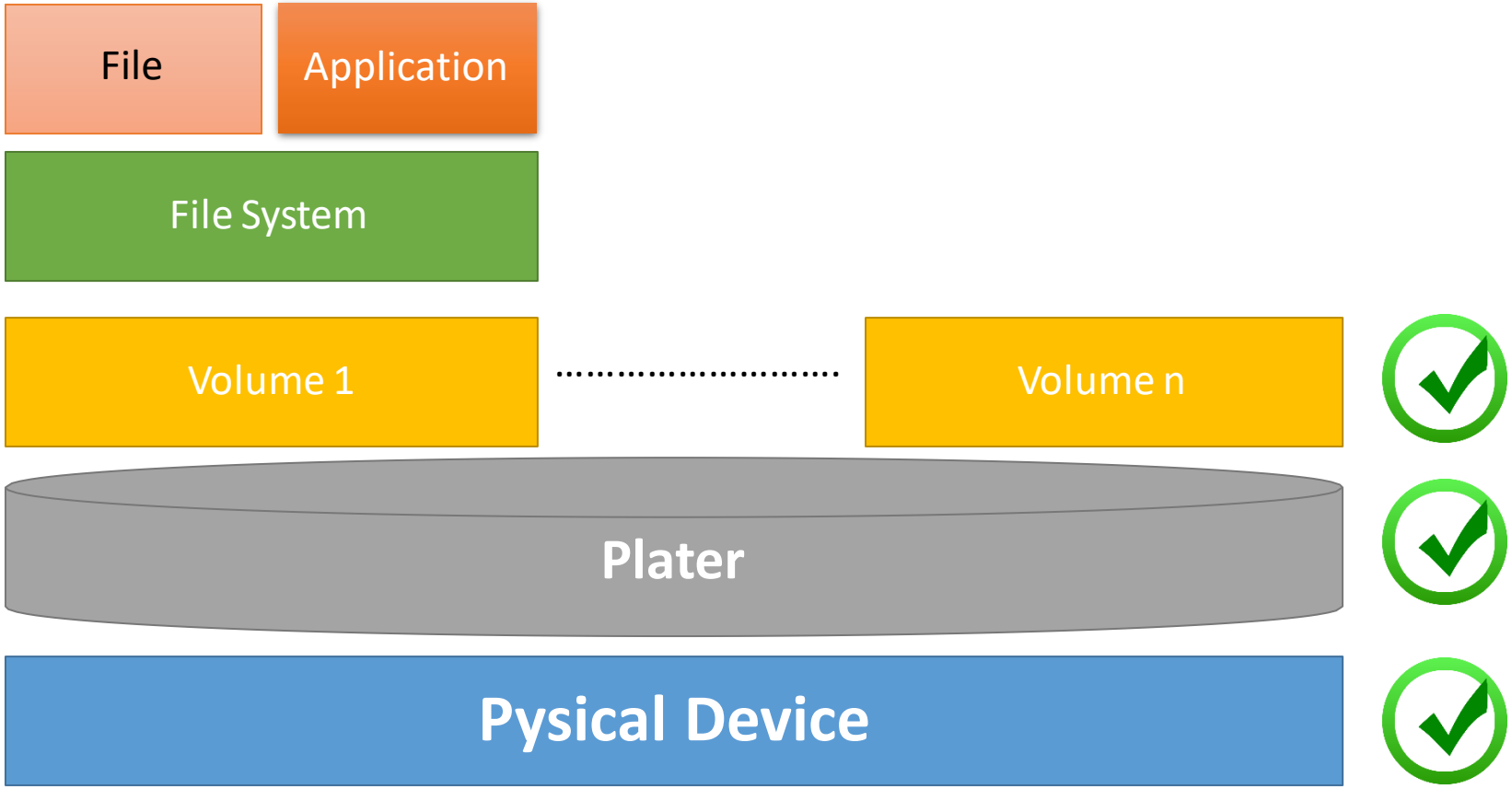
- MBR yalnızca 2 TB HDD alanını işleyebilir + yalnızca dört adede kadar birincil bölüm oluşturur. GPT diskleri 9.4 Zeta Bayta kadar boyutu destekler, + genişletilmiş bir bölüm oluşturmaya gerek kalmadan bir GPT sürücüsünde 128 bölüm oluşturabilirsiniz.

- GPT'nin bir kopyayı ikincil yedek olarak tutmasından dolayı GPT daha güvenlidir.

- UEFI nedeniyle, GPT önyükleme işlemleri MBR'den çok daha hızlıdır.

DOSYA SİSTEMLERİ

DOSYA SİSTEM YAPISI



DOSYA SİSTEM YAPILARI

- Bir dosya sistemi olmadan, depolama aygıtı içerisindeki arka arkaya depolanmış büyük bir veri yığınını işletim sistemi ayırt edemezdi.

- Dosya sistemi terimi, adını, belgeleri dosya olarak tuttuğumuz ve dizinlere koyduğumuz eski kağıt tabanlı veri yönetim sistemlerinden alır.

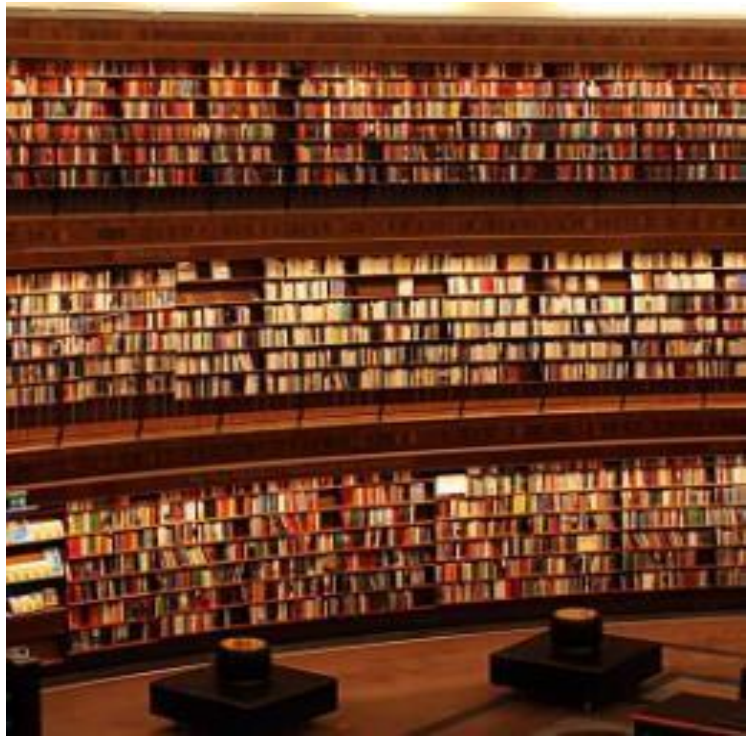
DOSYA SİSTEM YAPILARI

- Her yere dağılmış kağıt yığınları olan bir oda düşünün. Dosya sistemi olmayan bir depolama aygıtı da aynı durumda olacaktır ve bu işe yaramaz bir elektronik aygıt olacaktır.

DOSYA SİSTEM YAPILARI

- Yine de bir dosya sistemi sadece bir defter tutmaya benzemez.

- Alan yönetimi, meta verileri, veri şifreleme, dosya erişim kontrolü ve veri bütünlüğü de dosya sisteminin sorumluluklarıdır.



DOSYA SİSTEMİ NEDİR?

- Sabit Disk yapısı üzerinde dosyaların saklanma biçimini ifade eden yapılardır.
- Her işletim sistemi, dosyaları yönetmek için belirli bir dosya sistemi kullanır.

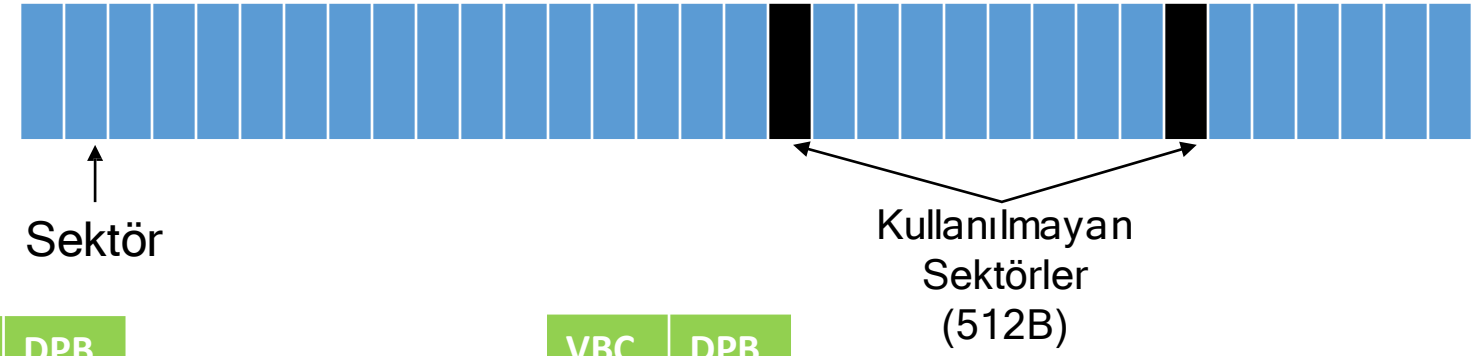
DOSYA SİSTEMİNİN GÖREVLERİ NELERDİR ?

- Sabit Disk Üzerinde ;
  - Dosya İsimlerini,
  - Dosyaların Konum Bilgilerini,
  - Meta-Data Bilgilerini,
  - Attribute Bilgilerini,
  - Fragmantasyon Bilgilerini saklamak ve bu bilgiler üzerinde yapılan değişiklikleri kayıt etmektir.

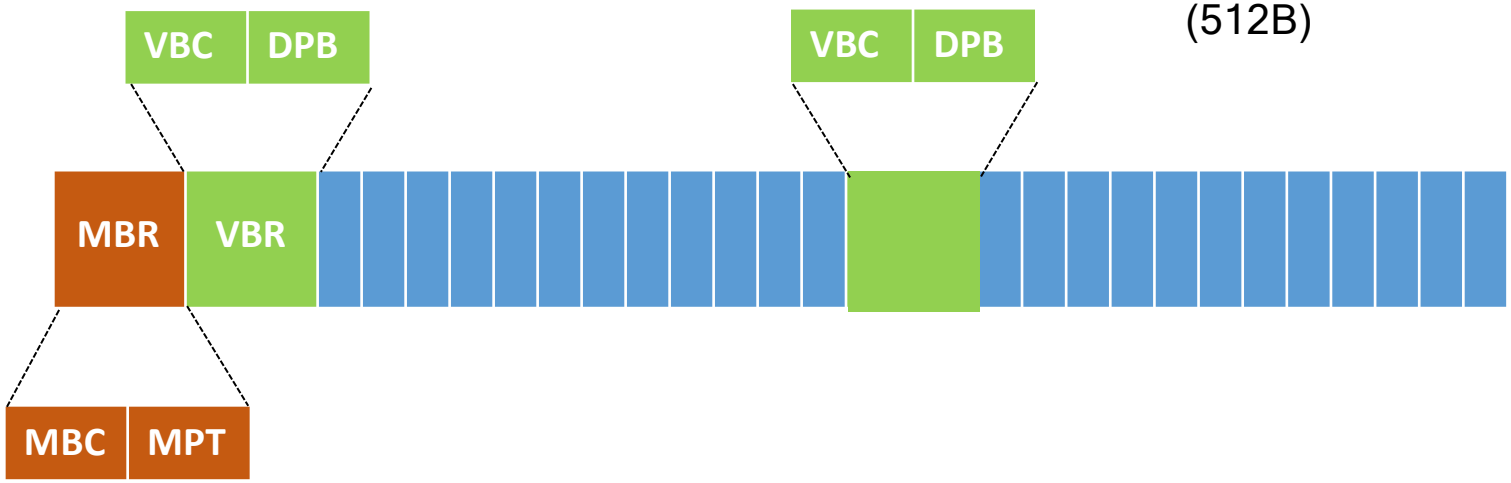
Dosya Sistemi Veri Kategorileri

- Blank Media

- Low-Level Format



- Partition



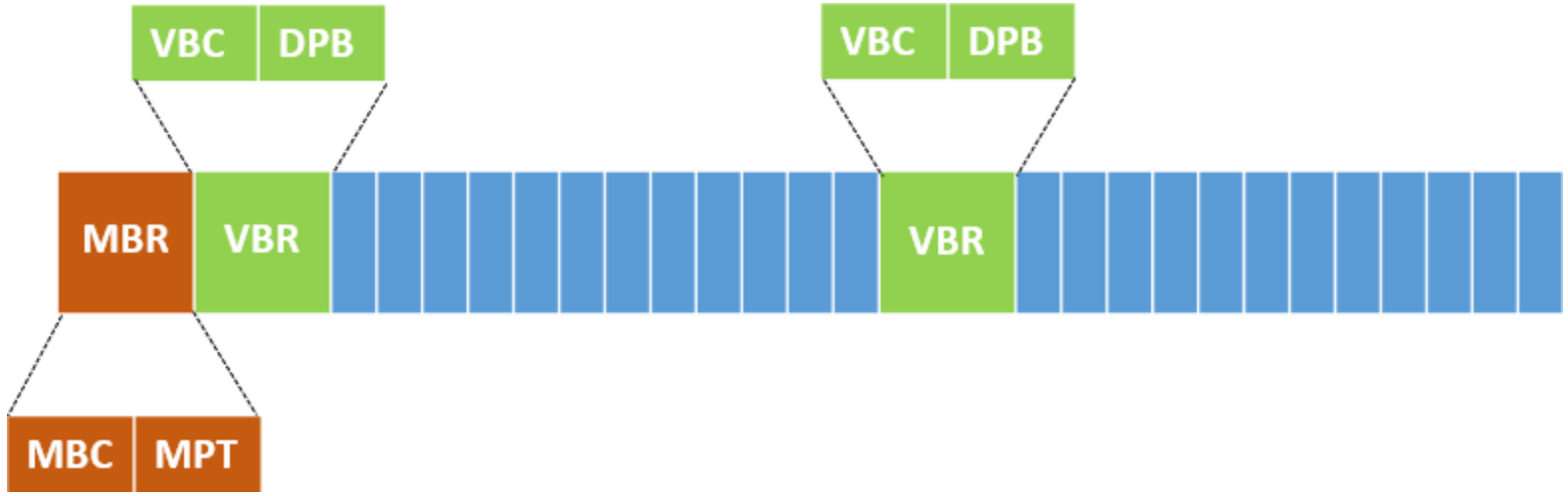
Dosya Sistem Kategorisi

Windows XP ve öncesi = 63
Windows Vista ve Sonrası = 2048
Linux Sitemlerde = 63

Not: Extended Partitionlar arasında 63 sektörlük bir boş alan bulunur.

Dosya Sistem Kategorisi

- Partition



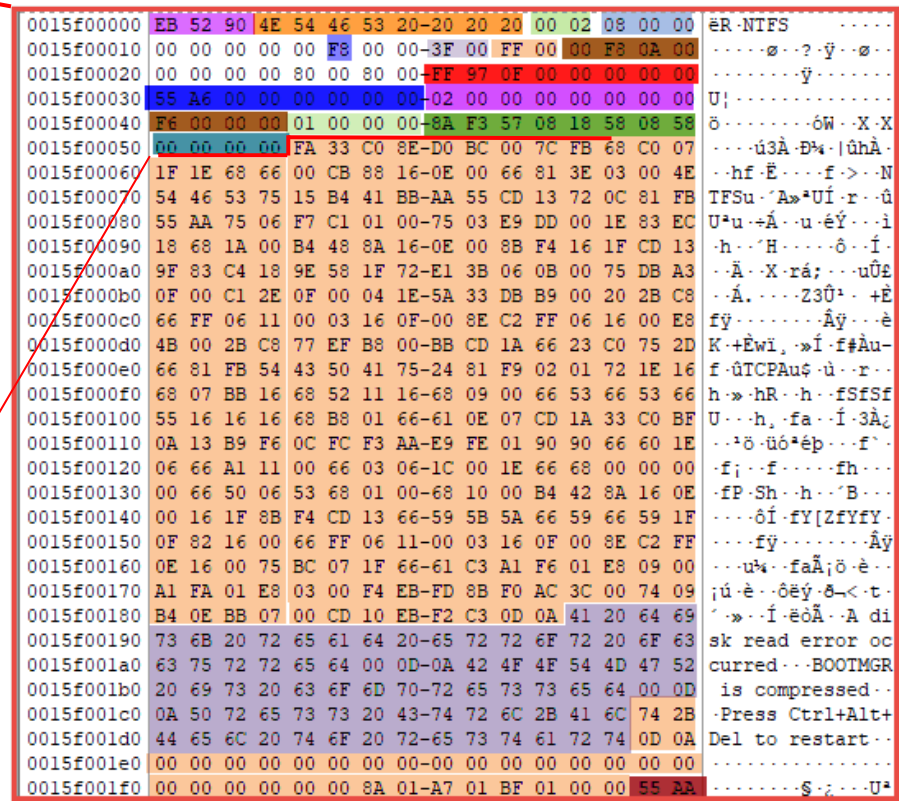
MBR = master boot record
MBC = master boot code
MPT = master partition table

VBR = volume boot record
VBC = volume boot code
DPB = disk parameter block

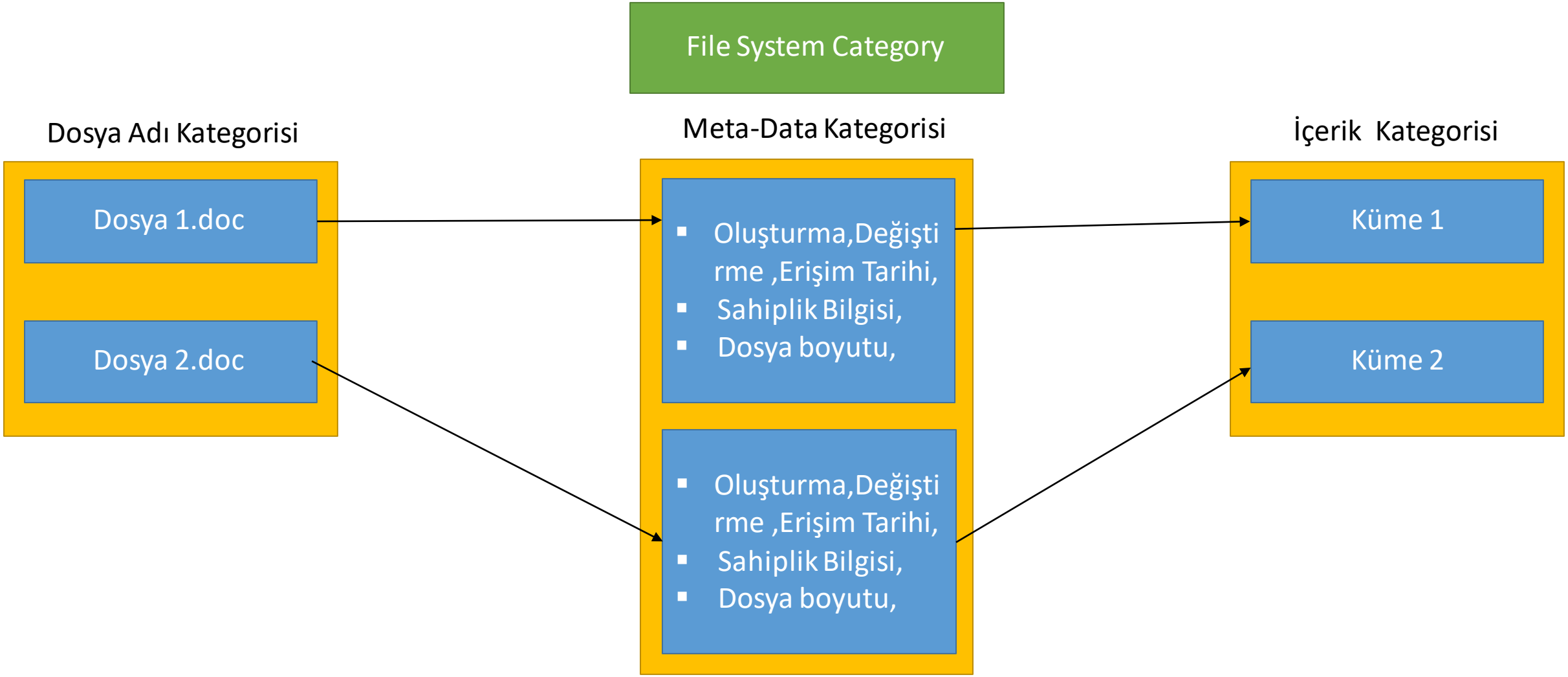
VBR (NTFS Dosya Sistemi)

Atlama talimatı (Jump İnstruction)	jmp 0x00000054
OEM kimliği	NTFS
Sektör başına bayt	0x200
Küme başına sektörler	0x08
Ayrılmış sektörler	0x00
Medya tanımlayıcı	0xF8
Parça başına sektörler	0x3F
Kafa sayısı	0xFF
Gizli sektörler	0xAF800
Toplam sektörler	0xF97FF
MFT ilk küme	0xA655
MFT ayna ilk küme	0x02
MFT kaydı başına küme sayısı	0xF6
Dizin bloğu başına kümeler	0x01
Volume Seri No	0x8AF3570818589858
Sağlama toplamı	0x00000000
HATA MESAJI	A disk read error occurred. BOOTMGR is compressed. Press Ctrl+Alt+Del to restart
İMZA DEĞERİ	0xAA55

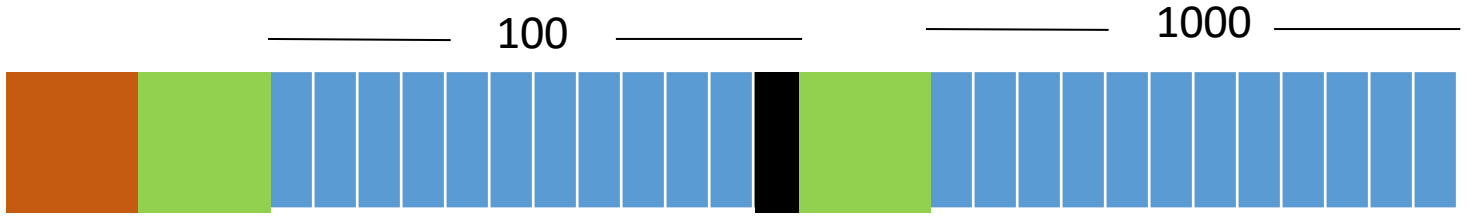
disk parameter block



DOSYA SİSTEMİ VERİ KATEGORİLERİ



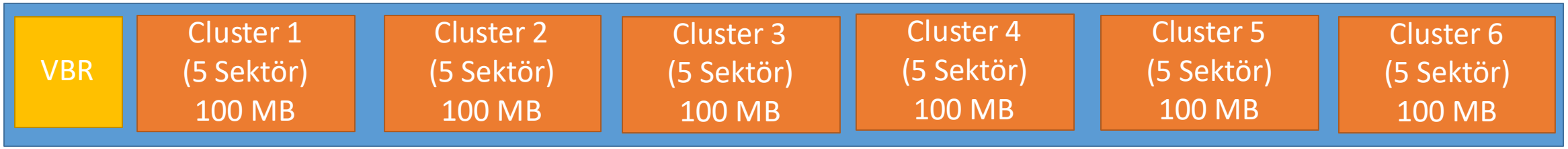
İçerik Kategorisi



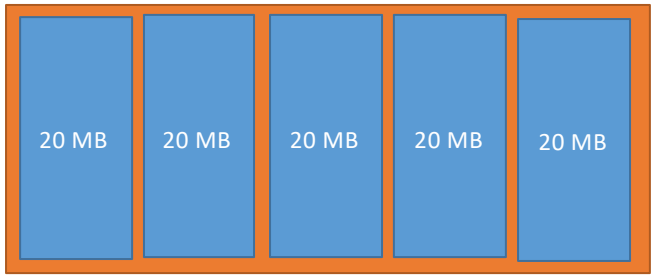
- 4 Bit lik adresleme yöntemi kullanalım
- 2^4=16 adet adresleme yapılabilir.
- 100÷16 = 6 kalan 4
- 6x16=96
- 4 slak alan (volüme slack)

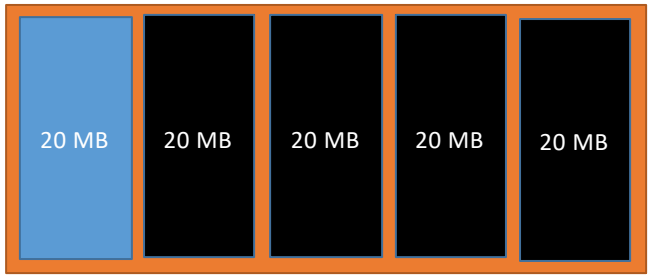
- 8 Bit lik adresleme yöntemi kullanalım
- 2^8=256 adet adresleme yapılabilir.
- 1000÷256 = 3 kalan 232
- 3x256=768
- 4x250=1000 , 250 adresleme yapılabilir

İçerik Kategorisi



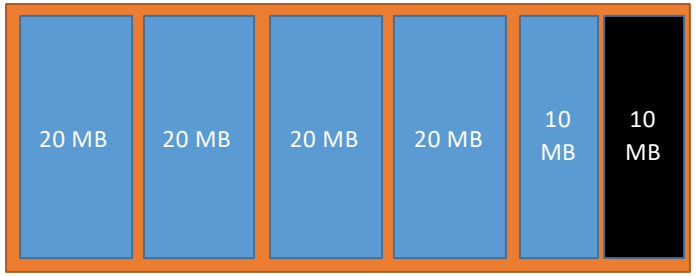
1. Durum = Örnek.txt (120 MB)





File Slack

2. Durum = Örnek.txt (90 MB)



Ram Slack

Metadata Kategorisi

Bir nesnenin veya bir bilgi parçasın kendisini ifade etmez. Ancak bir nesnenin veya bilgi parçasının içeriğini tanımlar.

Bunu farklı şekillerde ifade edelim;

- Metadata = Verilerle ilgili veriler
- Metadata = Bir bilgi nesnesini tanımlamak ve temsil etmek için kullanılan bir dizi veri
- Metadata = Depolanan verileri tanımlayan belgeler

Örneğin, bir e-postanın içinde içerik veya "bilgi" vardır, ancak meta verileri gönderildiği zaman, gönderen, konu vb. olabilir.

MetaData Türleri

Üç ana meta veri türü vardır:

- **Tanımlayıcı meta veriler:** Bir dosyanın içeriğiyle ilgilidir ve genellikle yazar, başlık, özet ve anahtar sözcükleri içerir.
- **Yapısal meta veriler:** Farklı veriler arasındaki ilişki ve bunların nasıl bir araya getirildikleri ile ilgilidir.
- **İdari meta veriler:** Sahiplik, hak yönetimi ve hangi programın kullanıldığı gibi diğer teknik bilgilerle ilgilidir.

Dosya Adı Kategorisi

Dosya adı kategorisi, her dosyanın ve dizinin adını depolayan veri yapılarını içerir.

FİLE ALLOCATION TABLE (FAT) DOSYA SİSTEMİ

FAT Dosya Sistemi: orijinal olarak küçük diskler ve basit klasör yapıları için tasarlanmış basit bir dosya sistemidir.

- Az sayıda veri yapısı türüne sahiptir.
- Genellikle USB, Hafıza Kartı, Dijital Kayıt Cihazlarında dosya formatı olarak sıkça tercih edilir.
- Linux ve Windows işletim sistemleri tarafından desteklenir.

FİLE ALLOCATION TABLE (FAT) DOSYA SİSTEMİ

FAT Dosya Sistemi Verileri organize etmek için iki tür kayıt tutar.

1. **File Allocation Table (FAT):** Hangi Cluster'ların boş yada dolu olduğu bilgisi ile veri fragmantasyonu ile ilgili kayıtları tutar.

2. **Directory Entry (DIR):** Veri fragmantasyonu dışındaki bilgiler burada tutulur.

Not: FAT dosya sistemi içerisindeki her bir dizin ve dosya cluster içerisinde tutulmaktadır.

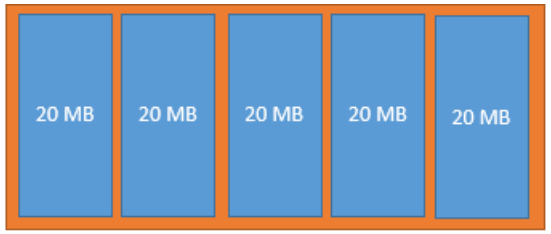
CLUSTER VE BLOCK

- Cluster ve Block'lar adreslemede kullanılan temel veri yapılarıdır ve iki'nin katları şeklinde) sektörlerin bir araya gelmesi ile oluşur.

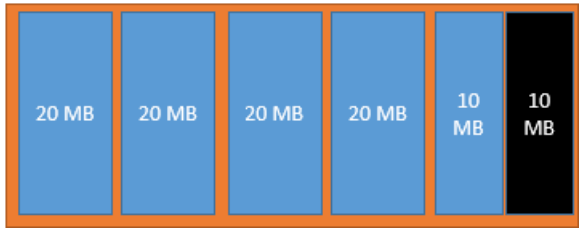
- Windows'ta cluster kullanılırken Linux işletim sistemlerinde Block kullanılır.

- Bir dosya yada dizin birden fazla block yada cluster'a yazılabilir.

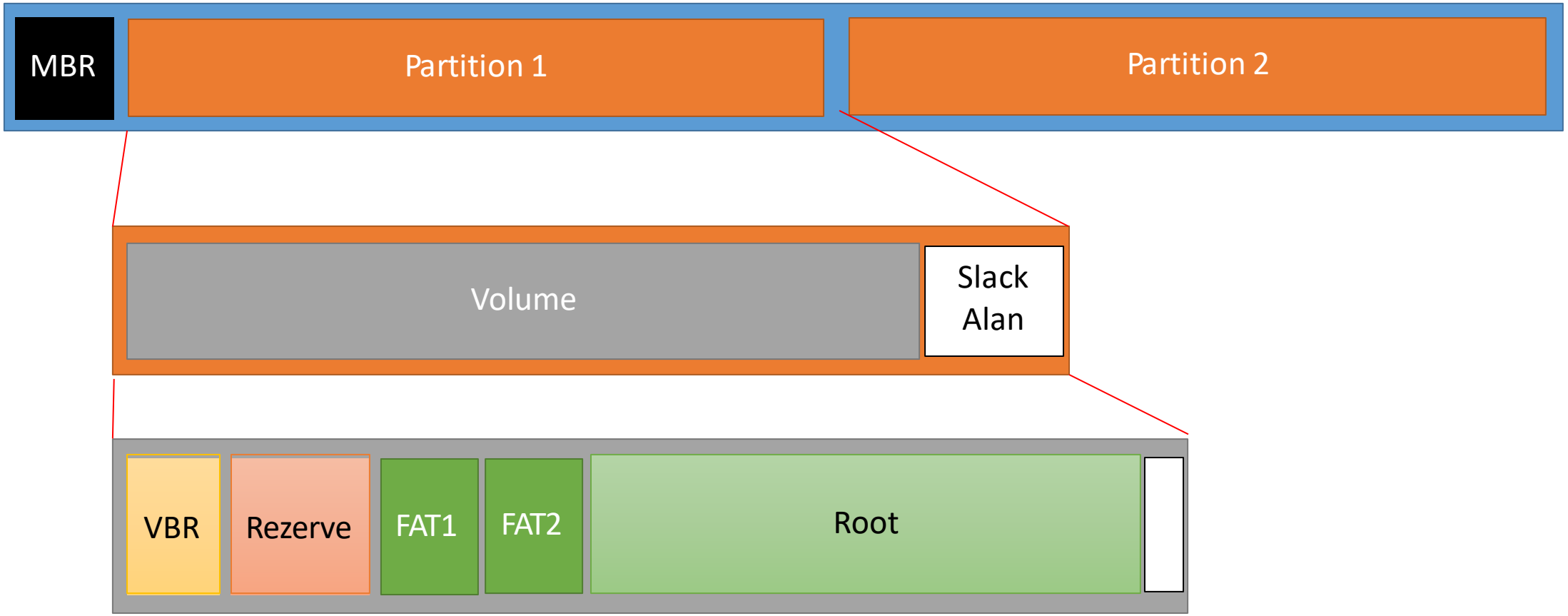
Örnek.txt (120 MB)



Örnek.txt (90 MB)



FAT DOSYA SİSTEMİ YAPISI



FAT VBR YAPISI

Bayt	Amaç
0-2	Önyükleme koduna atlamak için derleme kodu talimatları (önyüklenebilir bölümde zorunludur)
3-10	ASCII'de OEM adı
11-12	Sektör başına bayt (512, 1024, 2048 veya 4096)
13	Küme başına sektör sayısı (2'nin katı olmalı ve küme boyutu <=32 KB olmalıdır)
14-15	Sektörlerde ayrılmış alanın büyüklüğü
16	FAT sayısı (genellikle 2)
17-18	Kök dizindeki maksimum dosya sayısı (FAT12/16; FAT32 için 0)
19-20	Dosya sistemindeki sektör sayısı; 2 B yeterince büyük değilse, 0'a ayarlayın ve aşağıdaki 32-35 baytlarda 4 B değerini kullanın
21	Ortam türü (0xf0=çıkarılabilir disk, 0xf8=sabit disk)
22-23	FAT12/16 için sektörler halinde her bir FAT'nin boyutu; FAT32 için 0
24-25	Depolama aygıtındaki parça başına sektörler
26-27	Depolama aygıtındaki kafa sayısı
28-31	Başlangıç bölümünden önceki sektör sayısı
32-35	Dosya sistemindeki sektör sayısı; yukarıdaki 2B alanı (bayt 19-20) sıfır değilse bu alan 0 olacaktır.

```
000000000  EB 58 90 4D 53 44 4F 53-35 2E 30 00 02 10 E2 08  ëX·MSDOS5.0···â·
000000010  02 00 00 00 00 F8 00 00-3F 00 FF 00 40 00 00 00  ·····ø··?·ÿ·@···
000000020  C0 F7 DC 01 8F 3B 00 00-00 00 00 00 02 00 00 00  À÷Ü··;··********
000000030  01 00 06 00 00 00 00 00-00 00 00 00 00 00 00 00  ****************
000000040  80 00 29 F8 63 6B 7A 4E-4F 20 4E 41 4D 45 20 20  ··)øckzNO NAME
000000050  20 20 46 41 54 33 32 20-20 20 33 C9 8E D1 BC F4    FAT32   3É·Ñ¼ô
000000060  7B 8E C1 8E D9 BD 00 7C-88 56 40 88 4E 02 8A 56  {·Á·Ù½·|·V@·N··V
000000070  40 B4 41 BB AA 55 CD 13-72 10 81 FB 55 AA 75 0A  @´A»ªUÍ·r··ûUªu·
000000080  F6 C1 01 74 05 FE 46 02-EB 2D 8A 56 40 B4 08 CD  öÁ·t·þF·ë-·V@´·Í
000000090  13 73 05 B9 FF FF 8A F1-66 0F B6 C6 40 66 0F B6  ·s·¹ÿÿ·ñf·¶Æ@f·¶
0000000a0  D1 80 E2 3F F7 E2 86 CD-C0 ED 06 41 66 0F B7 C9  Ñ·â?÷â·ÍÀí·Af··É
0000000b0  66 F7 E1 66 89 46 F8 83-7E 16 00 75 39 83 7E 2A  f÷áf·Fø·~··u9·~*
0000000c0  00 77 33 66 8B 46 1C 66-83 C0 0C BB 00 80 B9 01  ·w3f·F·f·À·»··¹·
0000000d0  00 E8 2C 00 E9 A8 03 A1-F8 7D 80 C4 7C 8B F0 AC  ·è,·é¨·¡ø}·Ä|·ð¬
0000000e0  84 C0 74 17 3C FF 74 09-B4 0E BB 07 00 CD 10 EB  ·Àt·<ÿt··´·»··Í·ë
0000000f0  EE A1 FA 7D EB E4 A1 7D-80 EB DF 98 CD 16 CD 19  î¡ú}ëä¡}·ëß·Í·Í·
000000100  66 60 80 7E 02 00 0F 84-20 00 66 6A 00 66 50 06  f`·~····  ·fj·fP·
000000110  53 66 68 10 00 01 00 B4-42 8A 56 40 8B F4 CD 13  Sfh····´B·V@·ôÍ·
000000120  66 58 66 58 66 58 66 58-EB 33 66 3B 46 F8 72 03  fXfXfXfXë3f;Før·
000000130  F9 EB 2A 66 33 D2 66 0F-B7 4E 18 66 F7 F1 FE C2  ùë*f3Òf··N·f÷ñþÂ
000000140  8A CA 66 8B D0 66 C1 EA-10 F7 76 1A 86 D6 8A 56  ·Êf·ÐfÁê·÷v··Ö·V
000000150  40 8A E8 C0 E4 06 0A CC-B8 01 02 CD 13 66 61 0F  @·èÀä··Ì¸··Í·fa·
000000160  82 74 FF 81 C3 00 02 66-40 49 75 94 C3 42 4F 4F  ·tÿ·Ã··f@Iu·ÃBOO
000000170  54 4D 47 52 20 20 20 20-00 00 00 00 00 00 00 00  TMGR    ********
000000180  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ****************
000000190  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ****************
0000001a0  00 00 00 00 00 00 00 00-00 00 00 00 0D 0A 44 69  ··************Di
0000001b0  73 6B 20 65 72 72 6F 72-FF 0D 0A 50 72 65 73 73  sk errorÿ··Press
0000001c0  20 61 6E 79 20 6B 65 79-20 74 6F 20 72 65 73 74   any key to rest
0000001d0  61 72 74 0D 0A 00 00 00-00 00 00 00 00 00 00 00  art·************
0000001e0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ****************
0000001f0  00 00 00 00 00 00 00 00-AC 01 B9 01 00 00 55 AA  ********¬·¹···Uª
```

FAT YAPISI

- Dosya Fragmantasyonunu sağlar.
- FAT, bir kümenin bir dosyaya tahsis edilip edilmediğini ve tahsis ettiği bir sonraki kümenin nasıl tanımlanacağını göstererek çalışır.
- Bir FAT32 girişi 4 bayt uzunluğundadır.

2	3	4	FF FF FF 0F	5	FF FF FF 0F	7

ROOT YAPISI

- Dizin verileri 32 baytlık kayıtlar halinde düzenlenir.

- Herhangi bir sektör tam olarak 16 kayıt tutar ve hiçbir dizin kaydı sektör sınırını asla geçemez.

- **Dört tür 32 baytlık dizin kaydı vardır:**

1. **Kısa dosya adıyla normal kayıt** - Nitelik normaldir
2. **Uzun dosya adı metni** - Attrib, dört tür bitin tümüne sahiptir
3. **Kullanılmayan** - İlk bayt 0xE5
4. **Dizinin sonu** - İlk bayt sıfırdır

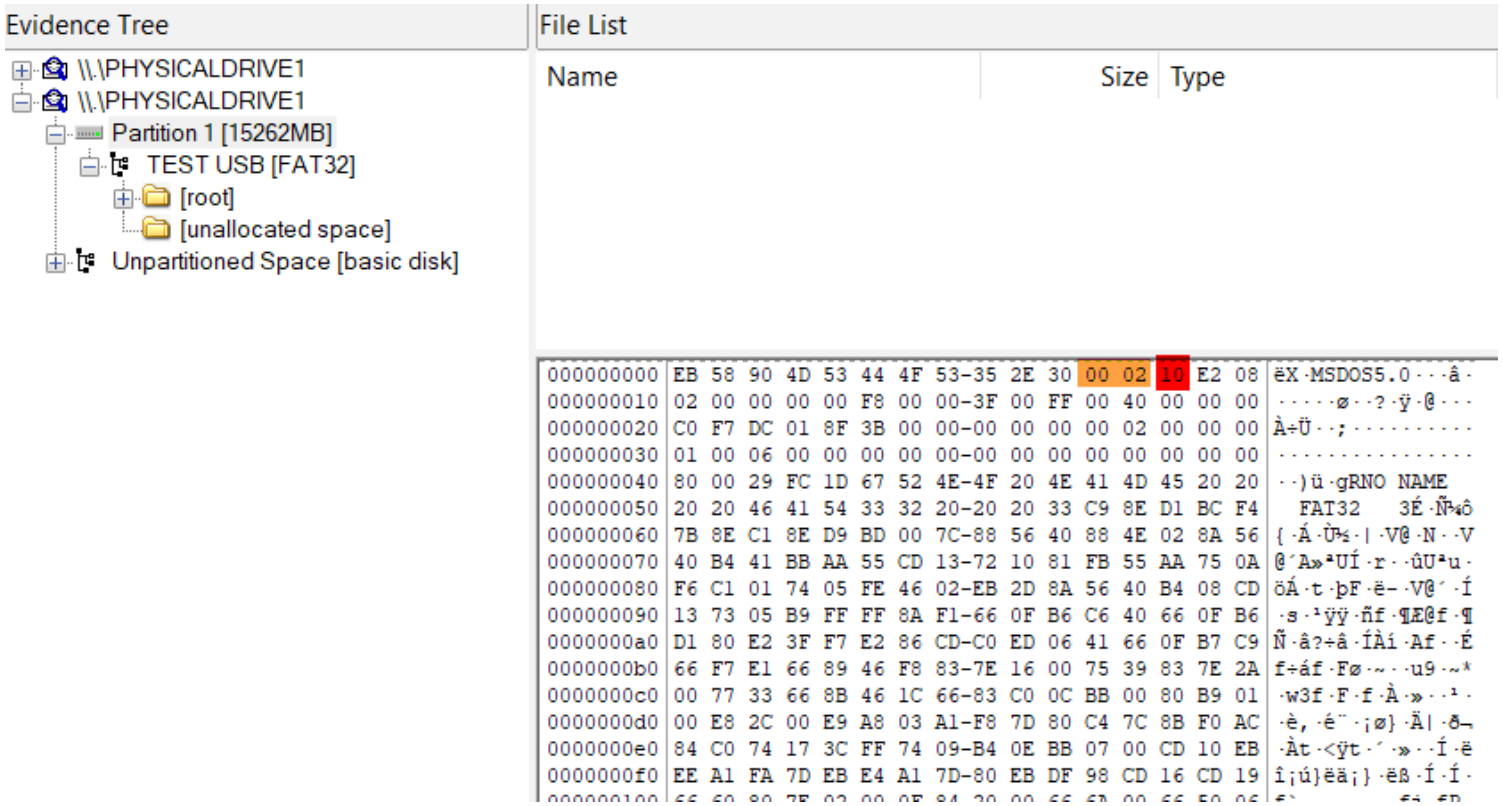
ROOT YAPISI

```
0080 54 45 53 54 20 20 20 20-4A 50 47 20 10 41 7D 04 TEST    JPG ·A}·
0090 5E 55 5E 55 00 00 D2 02-5E 55 05 00 EB A8 00 00 ^U^U··Ò·^U··ë¨··
```

- **32 bayt uzunluğundaki kayıt tablosu**

Ofset (Onaltılı)	Boyut (Bayt)	Tanım
0x00	8	Dosya adı (ilk bayt durum baytıdır)
0x08	3	Dosya uzantısı
0x0B	1	Dosya Öznitelik Baytı (11 Nitelik - bir bitvektör. Bit 0: salt okunur. Bit 1: gizli. Bit 2: sistem dosyası. Bit 3: birim etiketi. Bit 4: alt dizin Bit 5: arşiv. Bit 6-7: kullanılmamış.
0x0C	1	Küçük Harf Bayrakları
0x0D	1	Milisaniye Olarak Dosya Oluşturma Süresi
0x0E	2	Dosya Oluşturma Süresi (0-4 bit saniye, 5-10 bit dakika, 11-15 bit saat)
0x10	2	Dosya Oluşturma Tarihi (0-4 bit gün, 5-8 bit ay, 9-15 bit yıl)
0x12	2	Son Erişim Tarihi (0-4 bit gün, 5-8 bit ay, 9-15 bit yıl)
0x14	2	High Word Başlangıç Kümesi (sıfır ise yalnızca Low Word kullanılır)
0x16	2	Dosyaya Son Yazma Zamanı (5/6/5 bit, saat/dakika/çift saniye için)
0x18	2	Dosyaya Son Yazma Tarihi Tarih (7/4/5 bit, 1980'den beri/ay/gün için)
0x1A	2	Düşük Kelime Başlangıç Kümesi (Başlangıç kümesi (boş bir dosya için 0)
0x1C	4	Dosya boyutu (Bayt cinsinden dosya boyutu)

ÖRNEK UYGULAMA



- **VBR okunarak sektör ve küme boyutu hesaplanması**

Ofset (Onaltılı)	Boyut (Bayt)	Tanım	Değer	Decimal
0x0B	2	Sektör Başına Bayt	00 02	512
0x0D	1	Küme Başına Sektörler	10	16

ROOT YAPISI

```
0080 54 45 53 54 20 20 20 20-4A 50 47 20 10 41 7D 04 TEST    JPG ·A}·
0090 5E 55 5E 55 00 00 D2 02-5E 55 05 00 EB A8 00 00 ^U^U··Ò·^U··ë¨··
```

- **Attiributes 11.byte**

Attribute 20 = 00100000

0	0	0	0	0	0	0	1	Read only	Sadece oku
0	0	0	0	0	0	1	0	Hidden File	Gizlenmiş. Normal dizin görünümlerinden dosyaları veya dizinleri gizler.
0	0	0	0	0	1	0	0	System File	Dosyanın sisteme ait olduğunu ve fiziksel olarak taşınmaması gerektiğini belirtir
0	0	0	0	1	0	0	0	Volume ID	Volume Id'si
0	0	0	0	1	1	1	1	Long File Name	Uzun Dosya Adı
0	0	0	1	0	0	0	0	Directory	Bir Dizinin Kapsayıcı olduğunu gösterir
0	0	1	0	0	0	0	0	Archive	Yedeklemeye yardımcı uygulamalar tarafından kullanılır. FAT sürücüsü tarafından ayarlanan yeni dosya oluşturma, değiştirme veya yeniden adlandırma işlemi yapılır. Yedekleme programları, yedeklenecek dosyayı kolayca bulabilir ve yedekleme işleminden sonra özniteliği temizler.

ROOT YAPISI

```
0080 54 45 53 54 20 20 20 20-4A 50 47 20 10 41 7D 04 TEST    JPG ·A}·
0090 5E 55 5E 55 00 00 D2 02-5E 55 05 00 EB A8 00 00 ^U^U··Ò·^U··ë¨··
```

- **Dosya Oluşturma Tarihi : 18-19.byte**

Oluşturma Tarihi 5E 55(55 5E) = 0101010101011110

Byte 19								Byte 18							
55								5E							
0	1	0	1	0	1	0	1	0	1	0	1	1	1	1	0

0 1 0 1 0 1 0	1 0 1 0	1 1 1 1 0
42 + 1980 =2022	10	30
Yıl + 1980	Ay	Gün

ÖRNEK UYGULAMA

Evidence Tree

- \\.\PHYSICALDRIVE1
- \\.\PHYSICALDRIVE1
  - Partition 1 [15262MB]
    - TEST USB [FAT32]
      - [root]
      - [unallocated space]
  - Unpartitioned Space [basic disk]

File List

Name	Size	Type
System Volume Information	8	Directory
!EST2.GIF	354	Regular File
!EST2.GIF.FileSlack	7	File Slack
TEST.JPG	43	Regular File
TEST.JPG.FileSlack	6	File Slack

```
0000 54 45 53 54 20 55 53 42-20 20 20 08 00 00 00 00 TEST USB     ·····
0010 00 00 00 00 00 00 78 04-5E 55 00 00 00 00 00 00 ·····x·^U·····
0020 42 20 00 49 00 6E 00 66-00 6F 00 0F 00 72 72 00 B ·I·n·f·o···rr·
0030 6D 00 61 00 74 00 69 00-6F 00 00 00 6E 00 00 00 m·a·t·i·o···n···
0040 01 53 00 79 00 73 00 74-00 65 00 0F 00 72 6D 00 ·S·y·s·t·e···rm·
0050 20 00 56 00 6F 00 6C 00-75 00 00 00 6D 00 65 00  ·V·o·l·u···m·e·
0060 53 59 53 54 45 4D 7E 31-20 20 20 16 00 8A 77 04 SYSTEM~1    ···w·
0070 5E 55 5E 55 00 00 78 04-5E 55 03 00 00 00 00 00 ^U^U··x·^U·····
0080 54 45 53 54 20 20 20 20-4A 50 47 20 10 41 7D 04 TEST    JPG ·A}·
0090 5E 55 5E 55 00 00 D2 02-5E 55 05 00 EB A8 00 00 ^U^U··Ò·^U··ë¨··
00a0 E5 45 53 54 32 20 20 20-47 49 46 20 00 BA 8A 04 åEST2   GIF ·º··
00b0 5E 55 5E 55 00 00 51 AE-49 55 0B 00 7F 87 05 00 ^U^U··Q®IU·····
00c0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 ····————————
00d0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 ····————————
00e0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 ····————————
00f0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 ····————————
0100 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 ····————————
```

- **Silinmiş Dosya Görseli TEST2.GIF**

ÖRNEK UYGULAMA

```
00a0  E5 45 53 54 32 20 20 20-47 49 46 20 00 BA 8A 04  åEST2   GIF .º..
00b0  5E 55 5E 55 00 00 51 AE-49 55 0B 00 7F 87 05 00  ^U^U..Q®IU......
```

Ofset (Onaltılı)	Boyut (Bayt)	Tanım
0x00	8	Dosya adı (ilk bayt durum baytıdır)
0x08	3	Dosya uzantısı
0x0B	1	Dosya Öznitelik Baytı
0x0C	1	Küçük Harf Bayrakları
0x0D	1	Milisaniye Olarak Dosya Oluşturma Süresi
0x0E	2	Dosya Oluşturma Süresi
0x10	2	Dosya Oluşturma Tarihi
0x12	2	Son Erişim Tarihi
0x14	2	High Word Başlangıç Kümesi (sıfır ise yalnızca Low Word kullanılır)
0x16	2	Dosyaya Son Yazma Zamanı
0x18	2	Dosyaya Son Yazma Tarihi
0x1A	2	Düşük Kelime Başlangıç Kümesi
0x1C	4	Dosya boyutu

KONUM

```
00a0 | E5 45 53 54 32 20 20 20-47 49 46 20 00 BA 8A 04 | åEST2   GIF ·º··
00b0 | 5E 55 5E 55 00 00 51 AE-49 55 0B 00 7F 87 05 00 | ^U^U··Q®IU·····
```

Ofset (Onaltılı)	Boyut (Bayt)	Tanım
0x00	8	Dosya adı (ilk bayt durum baytıdır)
0x08	3	Dosya uzantısı
0x0B	1	Dosya Öznitelik Baytı
0x0C	1	Küçük Harf Bayrakları
0x0D	1	Milisaniye Olarak Dosya Oluşturma Süresi
0x0E	2	Dosya Oluşturma Süresi
0x10	2	Dosya Oluşturma Tarihi
0x12	2	Son Erişim Tarihi
0x14	2	High Word Başlangıç Kümesi (sıfır ise yalnızca Low Word kullanılır)
0x16	2	Dosyaya Son Yazma Zamanı
0x18	2	Dosyaya Son Yazma Tarihi
0x1A	2	Low Word (Düşük Kelime) Başlangıç Kümesi
0x1C	4	Dosya boyutu

Konum

High Word Başlangıç Kümesi **0x0000'dır**, yani başlangıç kümesini bulmak için sadece 0x1A ofsetindeki düşük kelime değerine ihtiyacımız vardır. 0x1A uzaklığındaki değer 0x0B00'dır ve bu küçük endian olarak okunan **0x000B**'tür ve ondalık sayıya dönüştürüldüğünde bize dosyanın **küme 11'de** başladığını söyler.

DOSYA BOYUTU

```
00a0 | E5 45 53 54 32 20 20 20-47 49 46 20 00 BA 8A 04 | åEST2   GIF ·º··
00b0 | 5E 55 5E 55 00 00 51 AE-49 55 0B 00 7F 87 05 00 | ^U^U··Q®IU......
```

Dosya boyutu

- 0x1C = 7F 87 05 00 ofsetindeki değer. Bu, 0x0005877F olan ve Windows'ta programcı hesap makinesi kullanılarak (onaltılık aralık işlevine sahip olan) küçük-endian olarak okunur ve mantıksal boyut olan **362367 bayttır.**

- **Küme Boyutu= Sektör boyutu x Kume Başına Düşen Sektör**
  = 512 x 16 =8192

- Dosyanın boyutunu küme boyutuna bölünce 362367/8192 = 44,23 olacaktır.

- Bu sayıyı da 45 kümeye yuvarladığımız küme sayısını belirlemiş oluyoruz.

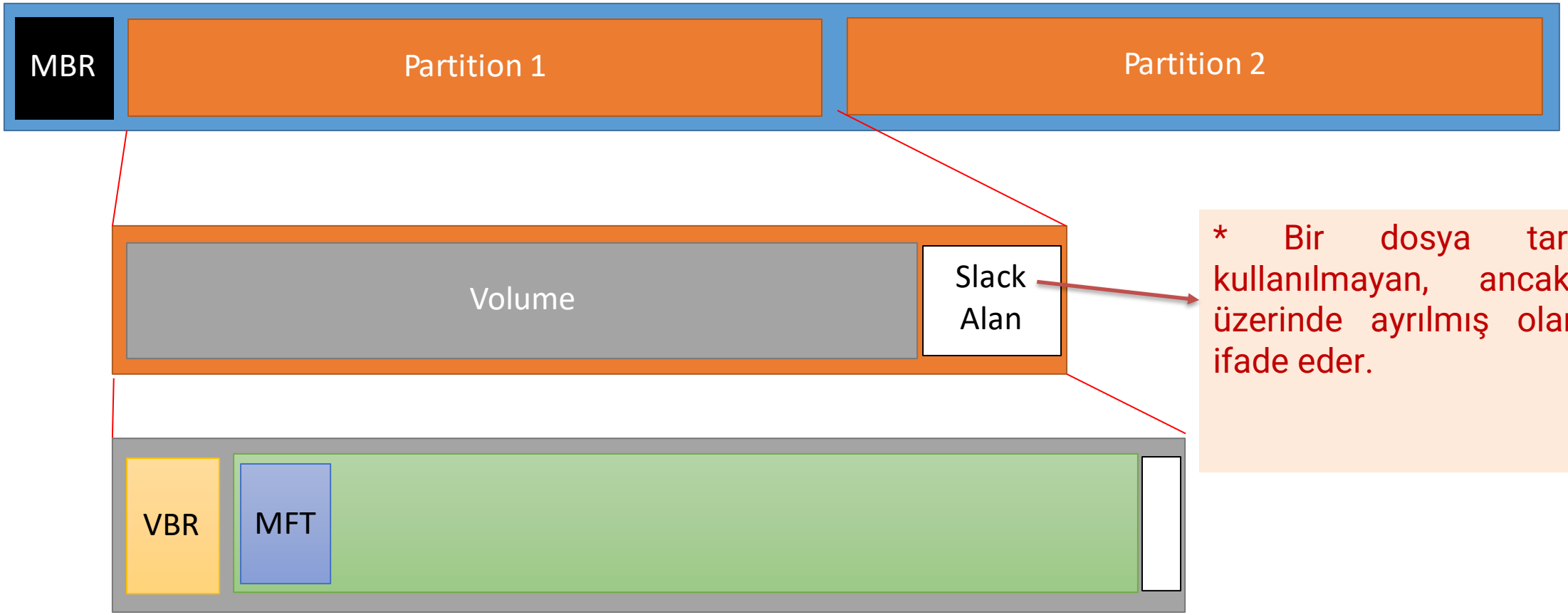
FSSTAT ARACI

```
C:\Users\vfv\Desktop\TEST>fsstat -f fat -o 64 image.001
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: FAT32

OEM Name: MSDOS5.0
Volume ID: 0x52671dfc
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory): TEST USB
File System Type Label: FAT32
Next Free Sector (FS Info): 32912
Free Sector Count (FS Info): 31225648

Sectors before file system: 64

File System Layout (in sectors)
Total Range: 0 - 31258559
* Reserved: 0 - 2273
** Boot Sector: 0
** FS Info Sector: 1
** Backup Boot Sector: 6
* FAT 0: 2274 - 17520
* FAT 1: 17521 - 32767
* Data Area: 32768 - 31258559
** Cluster Area: 32768 - 31258559
*** Root Directory: 32768 - 32783

METADATA INFORMATION
--------------------------------------------
Range: 2 - 499612678
Root Directory: 2

CONTENT INFORMATION
--------------------------------------------
Sector Size: 512
Cluster Size: 8192
Total Cluster Range: 2 - 1951613

FAT CONTENTS (in sectors)
--------------------------------------------
32768-32783 (16) -> EOF
32784-32799 (16) -> EOF
32800-32815 (16) -> EOF
32816-32911 (96) -> EOF
```

NEW TECHONOLOGI FİLE SYSTEM (NTFS)

- FAT dosya sisteminde bulunmayan bir performans, güvenilirlik ve uyumluluk kombinasyonu sağlar.
- Kritik verilerin bütünlüğü için önemli olan veri erişim kontrolünü ve sahiplik ayrıcalıklarını da destekler.
- Temel olarak, birimdeki her şey bir dosyadır ve bir dosyadaki her şey, veri özniteliğinden güvenlik özniteliğine, dosya adı özniteliğine kadar bir özniteliktir.
- Ayrılan bir NTFS birimindeki her sektör bir dosyaya aittir. Dosya sistemi meta verileri (dosya sisteminin kendisini tanımlayan bilgiler) bile bir dosyanın parçasıdır.

NTFS DOSYA SİSTEMİ YAPISI



* Bir dosya tarafından kullanılmayan, ancak disk üzerinde ayrılmış olan alanı ifade eder.

NTFS VBR YAPISI

0-2 Bayt	Jump komutu (Montaj kodu) önyükleme koduna atlamak için (önyüklenebilir bölümde zorunludur.)
3-10 Bayt	OEM adı (OEM Kimliği)
11-12 Bayt	Sektör başına Bayt
13 Bayt	Küme başına sektör
14-15 Bayt	Rezerve
16-20 Bayt	Kullanılmayan
21 Bayt	Ortam Tanımlayıcısı
22-23 Bayt	Kullanılmayan
24-31 Bayt	Kullanılmayan
32-35 Bayt	Kullanılmayan
36-39 Bayt	Kullanılmayan
40-47 Bayt	Dosya Sistemindeki Toplam Sectör
48-55 Bayt	$MFT'nin başlangıç küme adresi
56-63 Bayt	$MFT Mirror $DATA özniteliğinin başlangıç küme adresi
64 Bayt	MFT girişinin boyutu
65-67 Bayt	Kullanılmayan
68 Bayt	İndeks kaydının boyutu (dizin girişi)
69-71 Bayt	Kullanılmayan
72-79 Bayt	Seri Numarası
80-83 Bayt	Kullanılmayan
84-509 Bayt	Ön Yükleme Kodu
510-511 Bayt	Bitiş İmzası

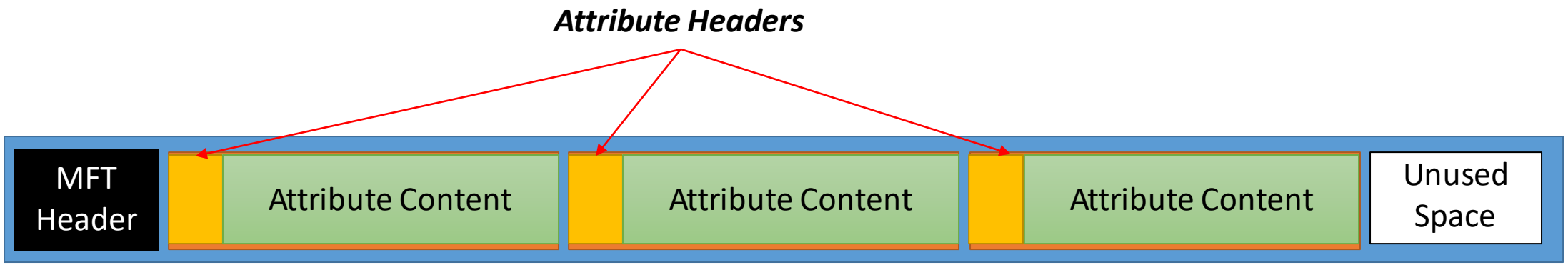
```
000000000  EB 52 90 4E 54 46 53 20-20 20 20 00 02 08 00 00  ëR·NTFS     ·····
000000010  00 00 00 00 00 F8 00 00-3F 00 FF 00 40 00 00 00  ·····ø··?·ÿ·@···
000000020  00 00 00 00 80 00 00 00-BF F7 DC 01 00 00 00 00  ·····¿÷Ü·····
000000030  00 00 0C 00 00 00 00 00-02 00 00 00 00 00 00 00  ··· ··· ··· ···
000000040  F6 00 00 00 01 00 00 00-18 EF DE EC 2B DF EC AC  ö·····ïÞì+ßì¬
000000050  00 00 00 00 FA 33 C0 8E-D0 BC 00 7C FB 68 C0 07  ····ú3À·Ð¼·|ûhÀ·
000000060  1F 1E 68 66 00 CB 88 16-0E 00 66 81 3E 03 00 4E  ··hf·Ë····f·>··N
000000070  54 46 53 75 15 B4 41 BB-AA 55 CD 13 72 0C 81 FB  TFSu·´A»ªUÍ·r··û
000000080  55 AA 75 06 F7 C1 01 00-75 03 E9 DD 00 1E 83 EC  Uªu·÷Á··u·éÝ···ì
000000090  18 68 1A 00 B4 48 8A 16-0E 00 8B F4 16 1F CD 13  ·h··´H·····ô··Í·
0000000a0  9F 83 C4 18 9E 58 1F 72-E1 3B 06 0B 00 75 DB A3  ··Ä··X·rá;···uÛ£
0000000b0  0F 00 C1 2E 0F 00 04 1E-5A 33 DB B9 00 20 2B C8  ··Á.····Z3Û¹· +È
0000000c0  66 FF 06 11 00 03 16 0F-00 8E C2 FF 06 16 00 E8  fÿ·····Âÿ···è
0000000d0  4B 00 2B C8 77 EF B8 00-BB CD 1A 66 23 C0 75 2D  K·+Èwï¸·»Í·f#Àu-
0000000e0  66 81 FB 54 43 50 41 75-24 81 F9 02 01 72 1E 16  f·ûTCPAu$·ù··r··
0000000f0  68 07 BB 16 68 52 11 16-68 09 00 66 53 66 53 66  h·»·hR··h··fSfSf
000000100  55 16 16 16 68 B8 01 66-61 0E 07 CD 1A 33 C0 BF  U···h¸·fa··Í·3À¿
000000110  0A 13 B9 F6 0C FC F3 AA-E9 FE 01 90 90 66 60 1E  ··¹ö·üóªéþ···f`·
000000120  06 66 A1 11 00 66 03 06-1C 00 1E 66 68 00 00 00  ·f¡··f·····fh···
000000130  00 66 50 06 53 68 01 00-68 10 00 B4 42 8A 16 0E  ·fP·Sh··h··´B···
000000140  00 16 1F 8B F4 CD 13 66-59 5B 5A 66 59 66 59 1F  ····ôÍ·fY[ZfYfY·
000000150  0F 82 16 00 66 FF 06 11-00 03 16 0F 00 8E C2 FF  ····fÿ·····Âÿ
000000160  0E 16 00 75 BC 07 1F 66-61 C3 A1 F6 01 E8 09 00  ···u¼··faÃ¡ö·è··
000000170  A1 FA 01 E8 03 00 F4 EB-FD 8B F0 AC 3C 00 74 09  ¡ú·è··ôëý·ð¬<·t·
000000180  B4 0E BB 07 00 CD 10 EB-F2 C3 0D 0A 41 20 64 69  ´·»··Í·ëòÃ··A di
000000190  73 6B 20 72 65 61 64 20-65 72 72 6F 72 20 6F 63  sk read error oc
0000001a0  63 75 72 72 65 64 00 0D-0A 42 4F 4F 54 4D 47 52  curred···BOOTMGR
0000001b0  20 69 73 20 63 6F 6D 70-72 65 73 73 65 64 00 0D   is compressed··
0000001c0  0A 50 72 65 73 73 20 43-74 72 6C 2B 41 6C 74 2B  ·Press Ctrl+Alt+
0000001d0  44 65 6C 20 74 6F 20 72-65 73 74 61 72 74 0D 0A  Del to restart··
0000001e0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ·· ··· ··· ···
0000001f0  00 00 00 00 00 00 8A 01-A7 01 BF 01 00 00 55 AA  ·····Š·§·¿···Uª
```

MFT (Master File Table)

- NTFS dosya sisteminin Adres defteridir.
- Sabit diskte bulunan bütün dosyaların ve dizinlerin, disk üzerinde hangi konumda bulunduğunu ve ne kadarlık bir yer kapladığını kaydeder.
- MFT her dosya ve dizin için kayıt tuttuğu gibi kendisi içinde bir kayıt tutar ve bu kayıt $MFT'dir. Bu giriş MFT 'nin disk üzerindeki konumunu tutar, MFT'nin tamamına ulaşabilmek için yine MFT kaydının okunması gerekir.
- MFT dosyasının başlangıç yeri Ön Yükleme sektöründe kayıtlıdır.

MFT YAPISI

- Her bir $MFT girişi (kayıt) 1.024 (0x400) bayt uzunluğundadır
- Küçük dosyalar bir MFT kayıtlarında tamamen bulunabilir



MFT KAYIT DEĞERLERİ 1024 BAYT

```
00c00 46 49 4C 45 30 00 03 00-38 19 80 00 00 00 00 00  FILE0···8·******
00c10 03 00 01 00 38 00 01 00-70 01 00 00 00 04 00 00  ····8···p******
00c20 00 00 00 00 00 00 00 00-06 00 00 00 03 00 00 00  ****************
00c30 02 00 00 00 00 00 00 00-10 00 00 00 60 00 00 00  ************`···
00c40 00 00 18 00 00 00 00 00-48 00 00 00 18 00 00 00  ********H*******
00c50 63 20 A2 DC F6 F1 D8 01-63 20 A2 DC F6 F1 D8 01  c ¢ÜöñØ·c ¢ÜöñØ·
00c60 63 20 A2 DC F6 F1 D8 01-63 20 A2 DC F6 F1 D8 01  c ¢ÜöñØ·c ¢ÜöñØ·
00c70 06 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ****************
00c80 00 00 00 00 01 01 00 00-00 00 00 00 00 00 00 00  ****************
00c90 00 00 00 00 00 00 00 00-30 00 00 00 68 00 00 00  ********0···h···
00ca0 00 00 18 00 00 00 01 00-50 00 00 00 18 00 01 00  ********P*******
00cb0 05 00 00 00 00 00 05 00-63 20 A2 DC F6 F1 D8 01  ********c ¢ÜöñØ·
00cc0 63 20 A2 DC F6 F1 D8 01-63 20 A2 DC F6 F1 D8 01  c ¢ÜöñØ·c ¢ÜöñØ·
00cd0 63 20 A2 DC F6 F1 D8 01-00 00 00 00 00 00 00 00  c ¢ÜöñØ********
00ce0 00 00 00 00 00 00 00 00-06 00 00 00 00 00 00 00  ****************
00cf0 07 03 24 00 56 00 6F 00-6C 00 75 00 6D 00 65 00  ··$·V·o·l·u·m·e·
00d00 60 00 00 00 28 00 00 00-00 00 18 00 00 00 04 00  `···(***********
00d10 10 00 00 00 18 00 00 00-54 00 45 00 53 00 54 00  ********T·E·S·T·
00d20 20 00 55 00 53 00 42 00-70 00 00 00 28 00 00 00   ·U·S·B·p···(···
00d30 00 00 18 00 00 00 05 00-0C 00 00 00 18 00 00 00  ****************
00d40 00 00 00 00 00 00 00 00-03 01 80 00 00 00 00 00  ****************
00d50 80 00 00 00 18 00 00 00-00 00 18 00 00 00 03 00  ****************
00d60 00 00 00 00 18 00 00 00-FF FF FF FF 00 00 00 00  ********ÿÿÿÿ····
00d70 20 02 00 00 00 00 00 00-60 00 00 00 28 00 00 00   *******`···(···
00d80 00 00 18 00 00 00 04 00-10 00 00 00 18 00 00 00  ****************
00d90 54 00 45 00 53 00 54 00-20 00 55 00 53 00 42 00  T·E·S·T· ·U·S·B·
00da0 70 00 00 00 28 00 00 00-00 00 18 00 00 00 05 00  p···(***********
00db0 0C 00 00 00 18 00 00 00-00 00 00 00 00 00 00 00  ****************
00dc0 01 02 84 00 00 00 00 00-80 00 00 00 18 00 00 00  ****************
00dd0 00 00 18 00 00 00 03 00-00 00 00 00 18 00 00 00  ****************
00de0 FF FF FF FF 00 00 00 00-00 00 00 00 00 00 00 00  ÿÿÿÿ************
00df0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 02 00  ****************
```

FİLE RECORD HEADER - 56 BYTE

$STANDARD_INFORMATION ATTRIBUTE – 96 BYTE

$FILE_NAME ATTRIBUTE

$DATA ATTRIBUTE

0000,0100, 0200, 0300 → Little Endian → 0000,0001,0002,0003

0000	Hex to Binary →	0000	Silinmiş Dosya
0001		0001	Varolan Dosya
0002		0010	Silinmiş Klasör
0003		0011	Var Olan Klasör

Tarih Bilgileri

```
00c00 46 49 4C 45 30 00 03 00-38 19 80 00 00 00 00 00  FILE0···8·····
00c10 03 00 01 00 38 00 01 00-70 01 00 00 00 04 00 00  ····8···p·····
00c20 00 00 00 00 00 00 00 00-06 00 00 00 03 00 00 00  ··————————————
00c30 02 00 00 00 00 00 00 00-10 00 00 00 60 00 00 00  ————————————`···
00c40 00 00 18 00 00 00 00 00-48 00 00 00 18 00 00 00  ————————H·····
00c50 63 20 A2 DC F6 F1 D8 01-63 20 A2 DC F6 F1 D8 01  c ¢ÜöñØ·c ¢ÜöñØ·
00c60 63 20 A2 DC F6 F1 D8 01-63 20 A2 DC F6 F1 D8 01  c ¢ÜöñØ·c ¢ÜöñØ·
00c70 06 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ··————————————
00c80 00 00 00 00 01 01 00 00-00 00 00 00 00 00 00 00  ··————————————
00c90 00 00 00 00 00 00 00 00-30 00 00 00 68 00 00 00  ————————0···h···
00ca0 00 00 18 00 00 00 01 00-50 00 00 00 18 00 01 00  ————————P·····
00cb0 05 00 00 00 00 00 05 00-63 20 A2 DC F6 F1 D8 01  ————————c ¢ÜöñØ·
00cc0 63 20 A2 DC F6 F1 D8 01-63 20 A2 DC F6 F1 D8 01  c ¢ÜöñØ·c ¢ÜöñØ·
00cd0 63 20 A2 DC F6 F1 D8 01-00 00 00 00 00 00 00 00  c ¢ÜöñØ————————
00ce0 00 00 00 00 00 00 00 00-06 00 00 00 00 00 00 00  ··————————————
00cf0 07 03 24 00 56 00 6F 00-6C 00 75 00 6D 00 65 00  ··$·V·o·l·u·m·e·
00d00 60 00 00 00 28 00 00 00-00 00 18 00 00 00 04 00  `···(···————
00d10 10 00 00 00 18 00 00 00-54 00 45 00 53 00 54 00  ————————T·E·S·T·
00d20 20 00 55 00 53 00 42 00-70 00 00 00 28 00 00 00   ·U·S·B·p···(···
00d30 00 00 18 00 00 00 05 00-0C 00 00 00 18 00 00 00  ··————————————
00d40 00 00 00 00 00 00 00 00-03 01 80 00 00 00 00 00  ··————————————
00d50 80 00 00 00 18 00 00 00-00 00 18 00 00 00 03 00  ··————————————
00d60 00 00 00 00 18 00 00 00-FF FF FF FF 00 00 00 00  ————————ÿÿÿÿ····
00d70 20 02 00 00 00 00 00 00-60 00 00 00 28 00 00 00   ·····`···(···
00d80 00 00 18 00 00 00 04 00-10 00 00 00 18 00 00 00  ··————————————
00d90 54 00 45 00 53 00 54 00-20 00 55 00 53 00 42 00  T·E·S·T· ·U·S·B·
00da0 70 00 00 00 28 00 00 00-00 00 18 00 00 00 05 00  p···(···————
00db0 0C 00 00 00 18 00 00 00-00 00 00 00 00 00 00 00  ··————————————
00dc0 01 02 84 00 00 00 00 00-80 00 00 00 18 00 00 00  ··————————————
00dd0 00 00 18 00 00 00 03 00-00 00 00 00 18 00 00 00  ··————————————
00de0 FF FF FF FF 00 00 00 00-00 00 00 00 00 00 00 00  ÿÿÿÿ————————
00df0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 02 00  ··————————————
```

- **Created Time (Oluşturma Tarihi)**
- **Modified Time (Değiştirme Tarihi)**
- **Entry Modify Time (Değiştirme Tarihi)**
- **Last Access Time (Son Erişim Tarihi)**

Dikkat !!!

- **Entry Modify Time (Değiştirme Tarihi) MFT ile ilgili bir durumdur**
- **Last Access Time (Son Erişim Tarihi) Eğer bu özellik kayıt defterinde açık değilse bu tarih değişmeyecektir. (Default Kapalı)**

Açmak İçin: Kayıt Defteri- Local Machine-Sytstem-Controlset001-Control-Filesystem-NtfsDiasebledLastAccessUbdate - Enabled

MFT ATTRIBUTE

Type	OS	Name
0x10		$STANDARD_INFORMATION
0x20		$ATTRIBUTE_LIST
0x30		$FILE_NAME
0x40	NT	$VOLUME_VERSION
0x40	2K	$OBJECT_ID
0x50		$SECURITY_DESCRIPTOR
0x60		$VOLUME_NAME
0x70		$VOLUME_INFORMATION
0x80		$DATA
0x90		$INDEX_ROOT
0xA0		$INDEX_ALLOCATION
0xB0		$BITMAP
0xC0	NT	$SYMBOLIC_LINK
0xC0	2K	$REPARSE_POINT
0xD0		$EA_INFORMATION
0xE0		$EA
0xF0	NT	$PROPERTY_SET
0x100	2K	$LOGGED_UTILITY_STREAM

FİLE RECORD HEADER - 56 BYTE

$STANDARD_INFORMATION ATTRIBUTE – 96 BYTE

$FILE_NAME ATTRIBUTE

$DATA ATTRIBUTE

MFT ATTRIBUTE OKUMAK

```
C:\Users\vfv\Desktop\4_MftRcrd-master>MFTRCRD C:\Users\vfv\Desktop\test\test.txt -d indxdump=on 1024 -s

Starting MftRcrd by Joakim Schicht
Version 1.0.0.41

Target is a File
Filesystem on C: is NTFS
File IndexNumber: 2776
BytesPerSector:  512
SectorsPerCluster:  8
ReservedSectors:  0
SectorsPerTrack:  63
NumberOfHeads:  255
HiddenSectors:  1978368
TotalSectors:  632009591
LogicalClusterNumberforthefileMFT:  786432
LogicalClusterNumberforthefileMFTMirr:  2
MFT Record Size: 1024

Record number: 2776 found at disk offset: 0x00000000C02B6000

Found attributes:
$STANDARD_INFORMATION (1)
$FILE_NAME (1)
$OBJECT_ID (1)
$DATA (1)

Record header info:
Offst to update sequence number: 48
Update sequence array size (words): 3
$LogFile sequence number (LSN): 15157047159
Sequence number: 95
Hard link count: 1
Offset to first Attribute: 115
Flags: FILE+ALLOCATED
Real size of the FILE record: 344
Allocated size of the FILE record: 1024
Base record MFT Ref: 0
Base record MFT Ref SequenceNo: 0
Next Attribute Id: 0x0006
File reference (MFT Record number): 2776
Update Sequence Number (a): 0400
Update Sequence Array (a): 00000000
```

MFTRCRD : https://github.com/jschicht/MftRcrd:

MFT ATTRIBUTE OKUMAK

```
$STANDARD_INFORMATION 1:
File Create Time (CTime): 2022-11-09 20:01:02:506:1460
File Modified Time (ATime): 2022-11-09 20:01:18:162:4737
MFT Entry modified Time (MTime): 2022-11-09 20:01:18:162:4737
File Last Access Time (RTime): 2022-11-09 20:02:11:466:3957
DOS File Permissions: archive
Max Versions: 0
Version Number: 0
Class ID: 0
Owner ID: 0
Security ID: 4716
Quota Charged: 0
USN: 3306067776

$FILE_NAME 1:
Parent MFTReference: 30136
ParentSequenceNo: 21
File Create Time (CTime): 2022-11-09 20:01:02:506:1460
File Modified Time (ATime): 2022-11-09 20:01:02:506:1460
MFT Entry modified Time (MTime): 2022-11-09 20:01:02:506:1460
File Last Access Time (RTime): 2022-11-09 20:01:02:506:1460
AllocSize: 0
RealSize: 0
EaSize: 0
Flags: archive
NameLength: 8
NameType: DOS+WIN32
NameSpace: 3
FileName: test.txt
```

MFT **ATTRIBUTE** OKUMAK

```
$OBJECT_ID 1:
ObjectId: {878E95FC-6046-11ED-A29A-701AB8017213}
ObjectId Version: 1
ObjectId Timestamp: 2022-11-09 15:52:35:6599292
ObjectId TimestampDec: 138873019556599292
ObjectId ClockSeq: 41626
ObjectId Node: 70-1A-B8-01-72-13
BirthVolumeId: NOT PRESENT
BirthObjectId: NOT PRESENT
DomainId: NOT PRESENT

Dump of $OBJECT_ID (1)
0000    40 00 00 00 28 00 00 00  00 00 00 00 00 00 05 00   @...(...........
0010    10 00 00 00 18 00 00 00  fc 95 8e 87 46 60 ed 11   ............F`..
0020    a2 9a 70 1a b8 01 72 13                            ..p...r.


$DATA 1:
Non-resident flag: 00
Name length: 0
Flags:
Attribute Id: 0001
Resident - Length of the Attribute: 6
Resident - Offset to the Attribute: 24
Resident - Indexed flag: 0
The Attribute's Name:

Dump of $DATA (1)
0000    80 00 00 00 20 00 00 00  00 00 18 00 00 00 01 00   .... ...........
0010    06 00 00 00 18 00 00 00  64 65 6e 65 6d 65 47 11   ........denemeG.
```

Boyut < 700-800 byte

MFT **ATTRIBUTE** OKUMAK

boot.ini için bir $MFT kod çözümünü boşaltma örneği:
MFTRCRD C:\boot.ini -d indxdump=off 1024 -s

Bir $MFT kod çözme + $MFT kaydının ve $MFT'nin kendisinin C: sürücüsünden bireysel özniteliklerinin dökümü için örnek:
MFTRCRD C:0 -a indxdump=off 1024 -s

D: sürücüsünden $LogFile için bir $MFT kaydı çözme örneği :
MFTRCRD D:2 -d indxdump=off 1024 -s

C: üzerindeki kök dizini için çözümlenmiş INDX kayıtlarının hexdump dökümü için örnek, C:\ adlı 'klasöre' eşdeğer
MFTRCRD C:5 -d indxdump=on 1024 -s

MFT **ATTRIBUTE** OKUMAK

- 0x100000 birim ofsetinden ve 4096 baytlık bir kayıt boyutuna sahip bir $MFT kaydı kod çözme işleminin dökümü için örnek:

  MFTRCRD C?0x100000 -d indxdump=off 4096 -s

- 4096 baytlık bir kayıt boyutuyla 0x100000 birim ofsetinden bir $MFT kaydı kod çözme örneği boşaltma ve ikili kaydı geçerli dizine kaydetme örneği:

  MFTRCRD C?0x100000 -d indxdump=off 4096 -w

MFT KAYITLARI

- MFT içerisinde bir dizi kayıt tutulur.
- Bu Kayıtların ilki MFT'nin kendisidir.
- MFT içerisinde ilk 16 Kayıt meta – data dosyaları için rezerve edilmiştir.
- Metadata dosyaları $ ile başlar ve ilk harfi her zaman büyüktür.
- Dosyalar diskin kök dizininde saklanır.

MFT KAYITLARI

File Name	MFT Record	Dosyanın Amacı
$Mft	0	Bir NTFS birimindeki her dosya ve klasör için bir temel dosya kaydı içerir. Bir dosya veya klasör için ayırma bilgisi tek bir kayda sığmayacak kadar büyükse, diğer dosya kayıtları da tahsis edilir. ***(Tüm Dosya İndexlerini saklar)***
$MftMirr	1	MFT'nin ilk dört kaydının kopya görüntüsü. Bu dosya, tek sektörlü bir arıza durumunda MFT'ye erişimi garanti eder. ***(İlk 4 kaydın yedeği)***
$LogFile	2	NTFS kurtarılabilirliği için kullanılan işlem adımlarının bir listesini içerir. Günlük dosyası boyutu, birim boyutuna bağlıdır ve 4 MB'a kadar büyük olabilir. Windows NT/2000 tarafından bir sistem arızasından sonra tutarlılığı NTFS'ye geri yüklemek için kullanılır. ***(İşlem log kayıtlarını tutar)***
$Volume	3	Birim etiketi ve birim sürümü gibi birim hakkında bilgiler içerir.
$AttrDef	4	Öznitelik adları, numaraları ve açıklamalarından oluşan bir tablo.
$(root)	5	Kök klasör.
$Bitmap	6	Hangi kümelerin kullanımda olduğunu ve boş olduğunu gösteren bir haritadır.
$Boot	7	Birimi monte etmek için kullanılan BPB'yi ve birim önyüklenebilirse kullanılan ek önyükleme yükleyici kodunu içerir. ***(Boot Kaydını saklar)***

MFT KAYITLARI

$BadClus	8	Birim için bozuk kümeler içeriyor.
$Secure	9	Bir birim içindeki tüm dosyalar için benzersiz güvenlik tanımlayıcıları içerir.
$Upcase	10	Küçük harfli karakterleri eşleşen Unicode büyük harfli karakterlere dönüştürür.
$Extend	11	***İlave Meta Data Bilgilerini İçeren dosyadır.***
	12-15	Gelecekte kullanılmak üzere rezerve edilmiştir.
$Quota	24	Birim alanında kullanıcı tarafından atanan kota sınırlarını içerir.
$ObjId	25	Dosya nesne kimliklerini içerir. **(Dosyalara verilen tek ID)**
$Reparse	26	Bu dosya, yeniden ayrıştırma noktası verilerini içeren birimdeki dosyalar ve klasörler hakkında bilgi içerir. ***(Yeniden Çözümleme Noktası Bilgisi)***